

AWS Cloud Interview Guide



CLOUD WITH RAJ
www.cloudwithraj.com

Table of Contents

- AWS Services for Interviews
- Microservice with ALB
- Event Driven Architecture (Basic)
- Microservice Vs Event Driven
- Event Driven Architectures (Advanced)
- Three-Tier Architecture
- Availability Zone & Data Center
- Lambda Cheaper Than EC2?
- RPO Vs RTO
- IP Address Vs URL
- DevOps Phases
- CI Vs CD Vs CD
- Kubernetes Tools Landscape
- Traditional CICD Vs GitOps
- Platform Vs Developer Team
- Scaling EC2 Vs Lambda Vs EKS
- Kubernetes (EKS) Scaling
- Gen AI Layers
- Prompt Engineering Vs RAG Vs Fine Tuning
- RAG with Bedrock
- EKS Upgrade With Karpenter
- EKS Upgrade With Karpenter - Advanced
- Container Lifecyle - Local to Cloud
- Gen AI Multi Model Invocation
- Git Workflow
- DevSecOps Workflow
- What Happens When You Type an URL
- AWS Well Architected Framework
- AWS Migration Tools
- Multi-site Active Active DR
- Kubernetes Node Pod Container Relationship
- Running Batch Workloads on AWS
- STAR Interview Format
- Behavioral Framework
- Hybrid Cloud Architecture
- How Karpenter Saves Money
- API Gateway Auth
- Serverless Web Application
- EDA with Kubernetes
- EDA with SNS, SQS, Kubernetes
- EKS Auto (Re:Invent 2024)
- Aurora DSQL (Re:Invent 2024)
- S3 Tables (Re:Invent 2024)
- Docker Vs Kubernetes
- System Design Trade-Offs
- Top 3 Popular Design
- Three Tier with Microservice
- Pod Container Sidecar
- Karpenter Bin Pack Granular Control
- Monolith Vs Microservice
- EventBridge Cross Account
- Kubernetes Tech Stack
- Lambda Transform
- Microservice Tech Stack
- Live Streaming
- Live Streaming with Ads
- API gateway Validation
- 10 AWS Services for 90% Interviews
- SA Vs Cloud Engineer Vs SRE
- EKS Dashboard
- Canary Deployment with API Gateway
- AWS Gen AI Landscape
- Agentic AI Evolution
- Agentic AI
- Multi Agent Pattern - Human In The Loop (HITL)
- Challenges Before MCP
- A2A
- A2A + MCP
- RAG Replaced by MCP?
- Agentic RAG
- Agentic RAG with MCP
- MCP + RAG + A2A
- Remote MCP Server
- AWS Strands Agent
- AWS Strands Code
- Bedrock Agents Vs Strands
- Bedrock Agentcore
- Kubernetes Ingress
- NGINX Ingress Deprecation Impact
- ALB Vs API Gateway
- API Gateway Transformation
- API Gateway Private Integration Change (Re:Invent 2025)
- Dynamo Index Change (Re:Invent 2025)
- Lambda Managed Instance (Re:Invent 2025)
- Multi Cloud Interconnect (Re:Invent 2025)
- Agentcore Gateway Enhancements (Re:Invent 2025)
- Canary Vs Blue/Green Vs Rolling Deployment

More To Be Added...

Raj's Bio

- Former Principal SA at 
- Former Distinguished Cloud Architect at 
- 20+ Years of IT experience
- Designed and implemented multiple world-scale projects with official AWS blogs on them
- Bestselling author (60,000+ paid students)
- Presented highly-rated talks at major events
- LinkedIn Top Systems Design Voice

Lighthouse projects designed : Dr. B. Covid Vaccine Registration, Collins Flight Systems used by 100+ airlines, Freddie Mac Datacenter to AWS Migration, and more

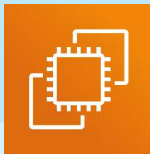
Please follow me on socials:



Important AWS Services for Interviews



Compute



EC2



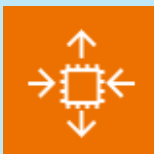
Lambda



Elastic Kubernetes Service



ECS



Auto Scaling

Storage



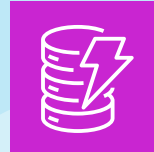
S3



EBS



RDS



DynamoDB



ElastiCache

Network



VPC



Load Balancer



API Gateway



AWS Global Infrastructure
(Region, AZ)

Security



KMS



IAM



WAF



Shield



GuardDuty



Secrets
Manager



Config

Gen AI



BedRock



Q



SageMaker

Migration



DMS



Migration Hub



Application Migration
Service



Application Discovery
Service

Event Driven



SNS



SQS



EventBridge



Step Functions

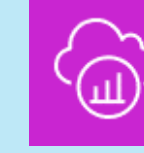
Observability



CloudWatch



CloudTrail



X-Ray

Cost Optimization



Compute
Optimizer



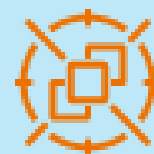
CloudWatch
Insights



Cost Explorer



Budget



Spot Instance



Reserve
Instance
Reporting



Savings Plan

Analytics



Glue



EMR



Athena



QuickSight



Kinesis



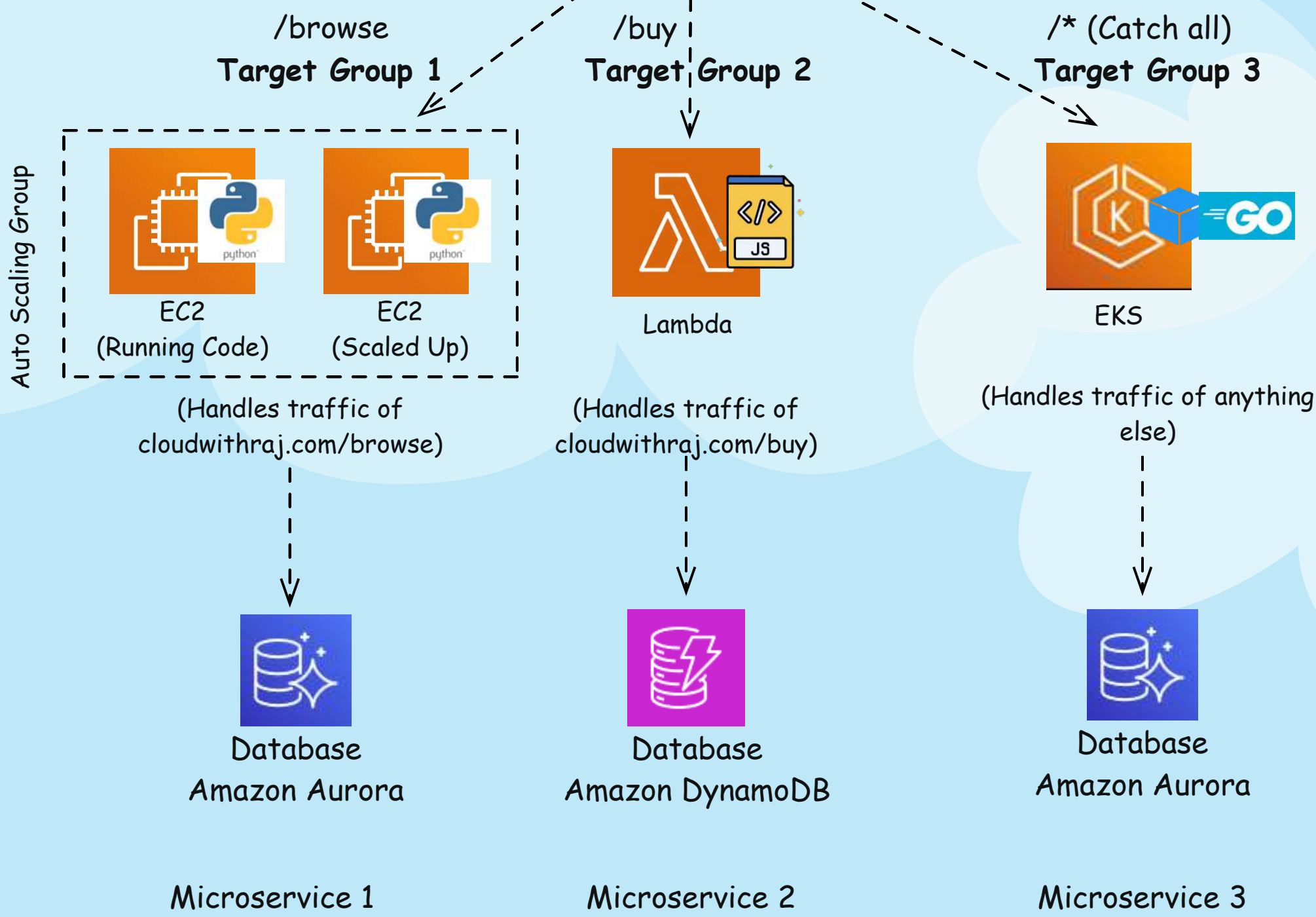
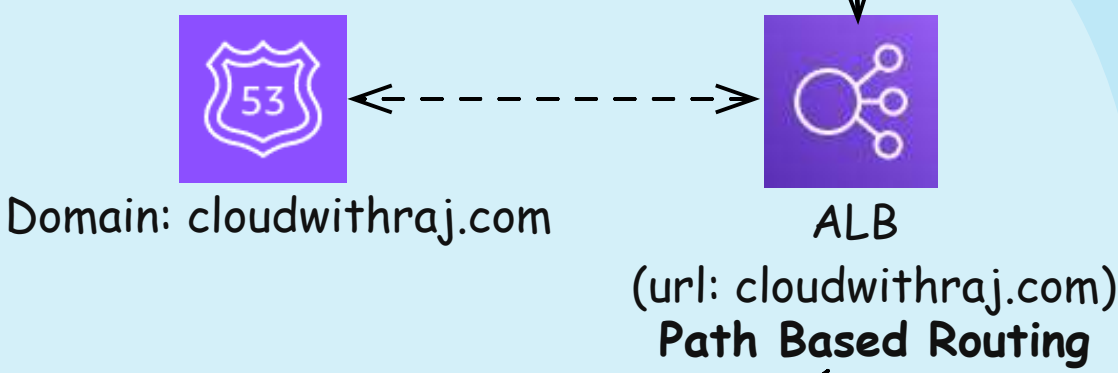
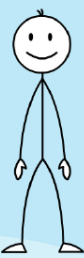
Redshift

DevOps

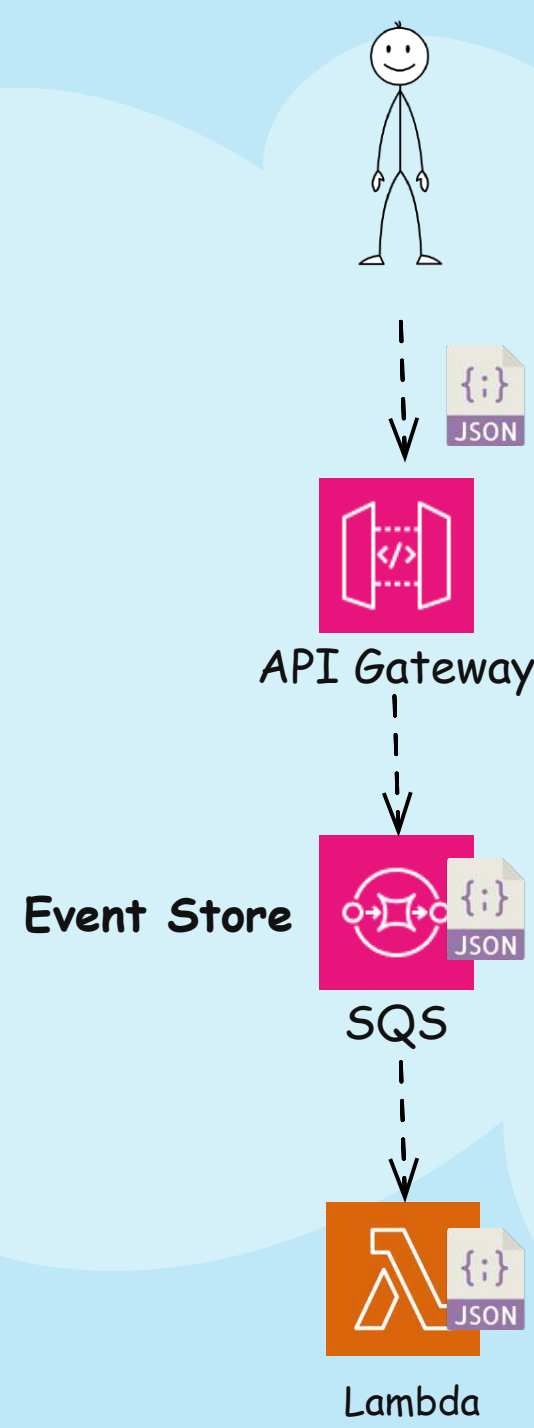


CloudFormation

Microservices with ALB



Event Driven Architecture (Basic)

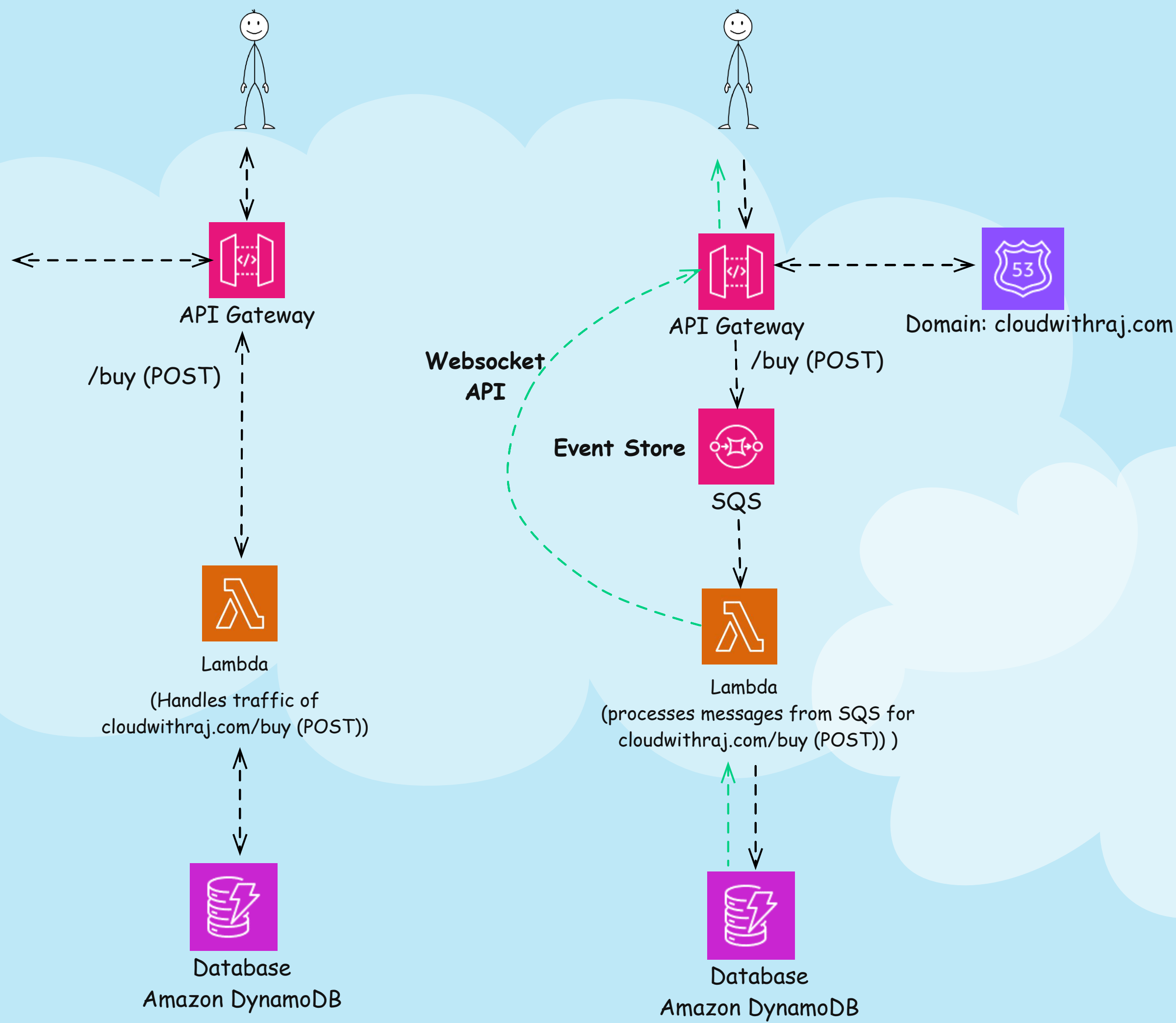


An event-driven architecture decouples the producer and processor. In this example producer (human) invokes an API, and send information in JSON payload. API Gateway puts it into an event store (SQS), and the processor (Lambda) picks it up and processes it. Note that, the API gateway and Lambda can scale (and managed/deployed) independently

Benefits of an event-driven architecture

1. Scale and fail independently - By decoupling your services, they are only aware of the event router, not each other. This means that your services are interoperable, but if one service has a failure, the rest will keep running. The event router acts as an elastic buffer that will accommodate surges in workloads.
2. Develop with agility - You no longer need to write custom code to poll, filter, and route events; the event router will automatically filter and push events to consumers. The router also removes the need for heavy coordination between producer and consumer services, speeding up your development process.
3. Audit with ease - An event router acts as a centralized location to audit your application and define policies. These policies can restrict who can publish and subscribe to a router and control which users and resources have permission to access your data. You can also encrypt your events both in transit and at rest.
4. Cut costs - Event-driven architectures are push-based, so everything happens on-demand as the event presents itself in the router. This way, you're not paying for continuous polling to check for an event. This means less network bandwidth consumption, less CPU utilization, less idle fleet capacity, and less SSL/TLS handshakes.

Microservice Vs Event Driven



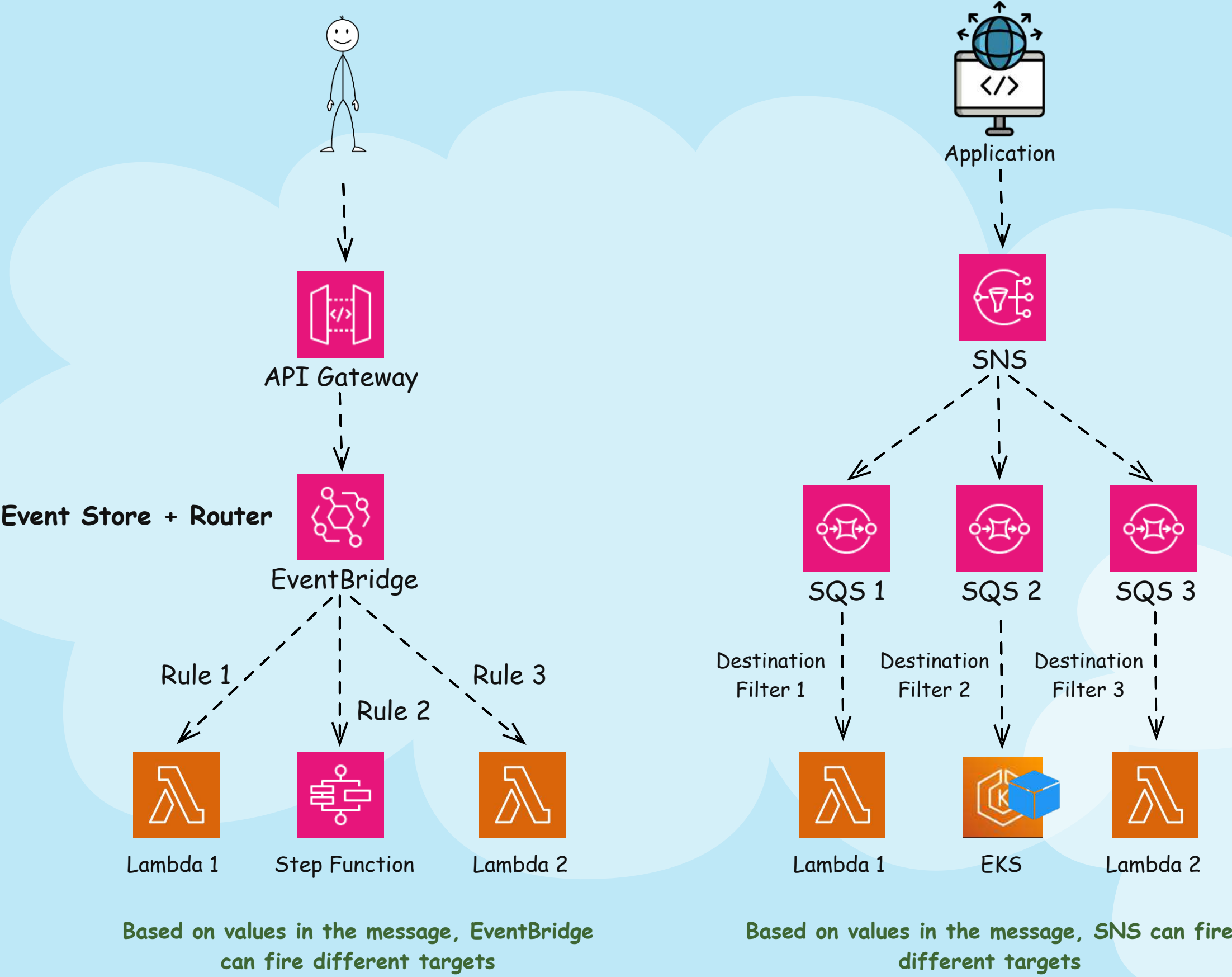
Microservice

Microservice with Event Driven

The main differences are:

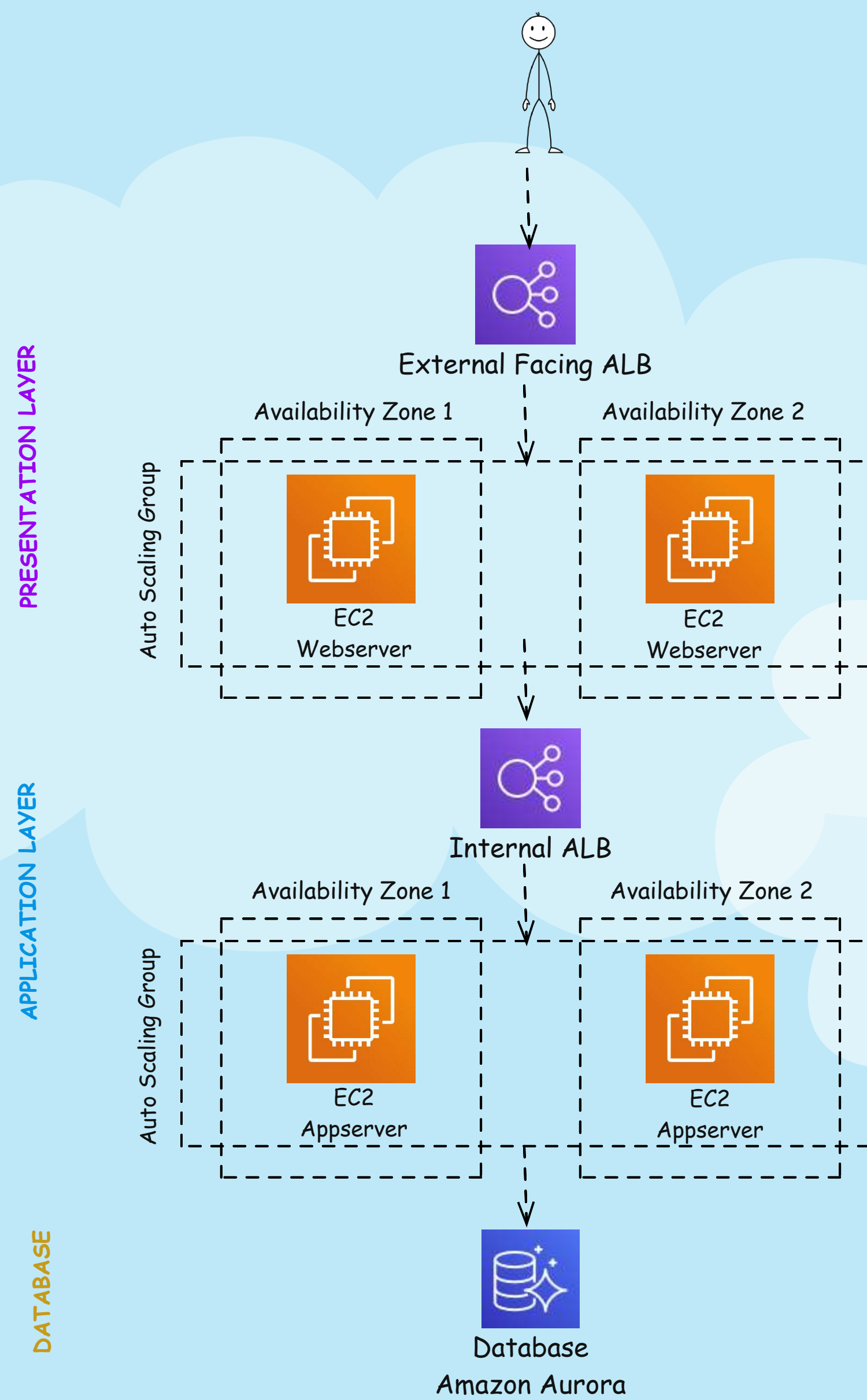
1. Traditional microservice is synchronous i.e. the request and response happens with the same invocation. Whereas with Event Driven, User gets a confirmation that message is inserted into SQS. But he doesn't get the response from the actual message processing by the Lambda in the same invocation. Instead the backend Lambda needs to send response out, in this case, using websocket APIs, to the user. Or the user can query the status afterwards
2. With EDA, API Gateway, and Lambda/Database can scale independently. Lambda can consume messages at a rate not to overwhelm the database
3. With EDA retries are built in. With microservices, if Lambda fails, user need to send the request again. With EDA, once the message is in SQS, even if Lambda fails, SQS will automatically retry

Event Driven Architecture (Advanced)



SNS Vs SQS Vs EventBridge Detailed Video:

3 Tier Architecture



1. First layer is presentation layer. Customers consume the application using this layer. Generally, this is where the front end runs. For example - amazon.com website. This is implemented using an external facing load balancer distributing traffic to VMs (EC2s) running webserver.

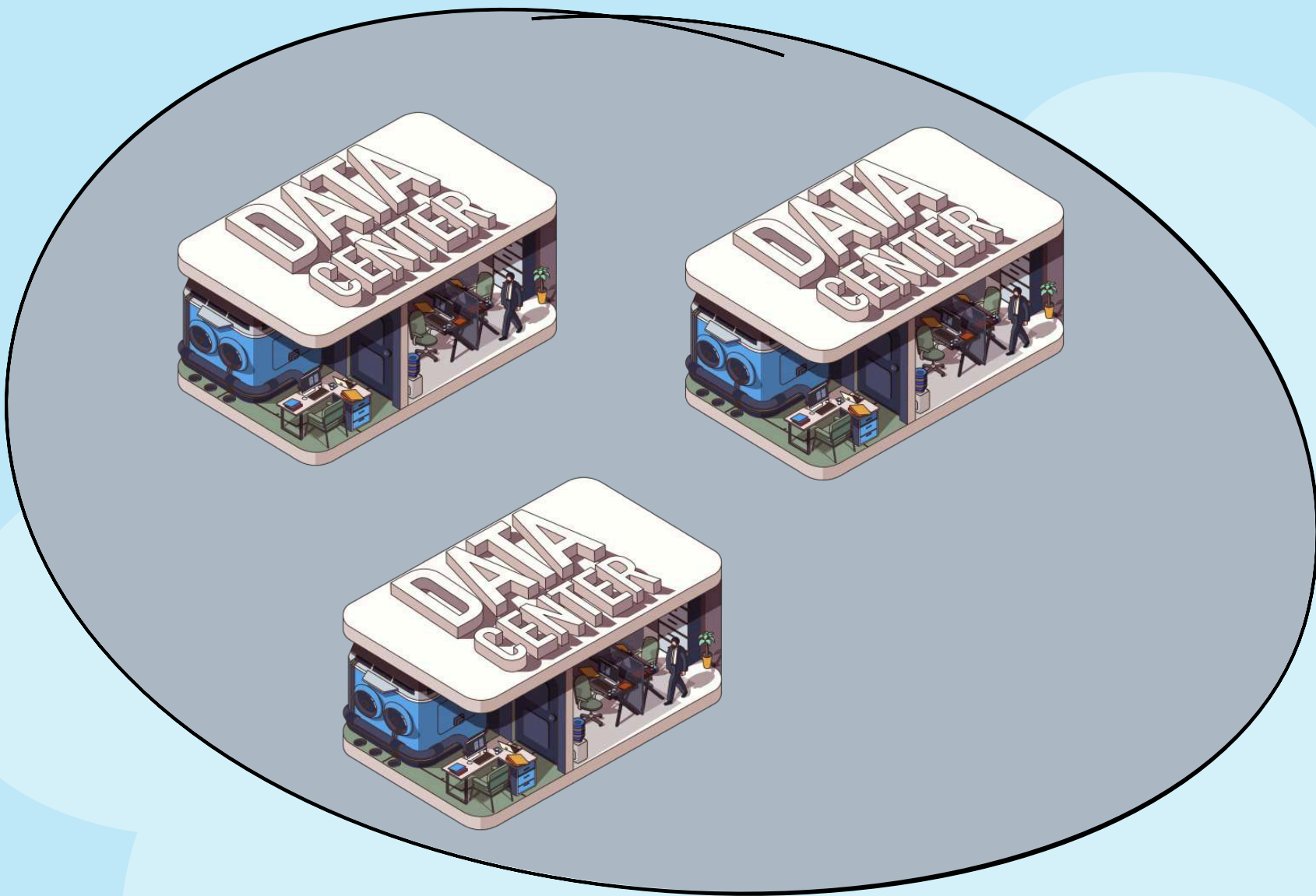
2. Second layer is application layer. This is where the business logic resides. Going with the previous example - you browsed your products on amazon.com, and now you found the product you like and then click "add to cart". The flow comes to the application layer, validates the availability, and then creates a cart. This layer is implemented with internal facing load balancer and VMs running applications.

3. The last layer is the database layer. This is where information is stored. All the product information, your shopping cart, order history etc. Application layer interacts with this layer for CRUD (Create, Read, Update, Delete) operations. This could be implemented using one or mix of databases - SQL (e.g. Amazon Aurora), and/or NoSQL (DynamoDB)

Lastly - why is this so popular in interviews? This architecture comprised of many critical patterns - microservices, load balancing, scaling, performance optimization, high availability, and more. Based on your answers, interviewer can dig deep and check your understanding of the core concepts.

How Many Data Centers in One Availability Zone?

Incorrect Answer:
One Availability Zone means one data center



1 Availability Zone

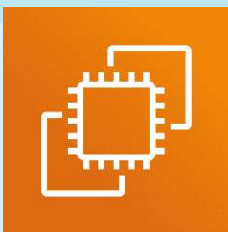
Correct Answer:
An AWS availability zone (AZ) can contain multiple data centers. Each zone is usually backed by one or more physical data centers, with the largest backed by as many as five.

Is AWS Lambda Cheaper than Amazon EC2?



Incorrect Answer:
Yes, AWS Lambda is cheaper than Amazon EC2

S



Lambda Cost Factors:
Architecture (x86 vs. Graviton)
Number of requests
Duration of each request
Amount of memory allocated (NOT used)
Amount of ephemeral storage allocated

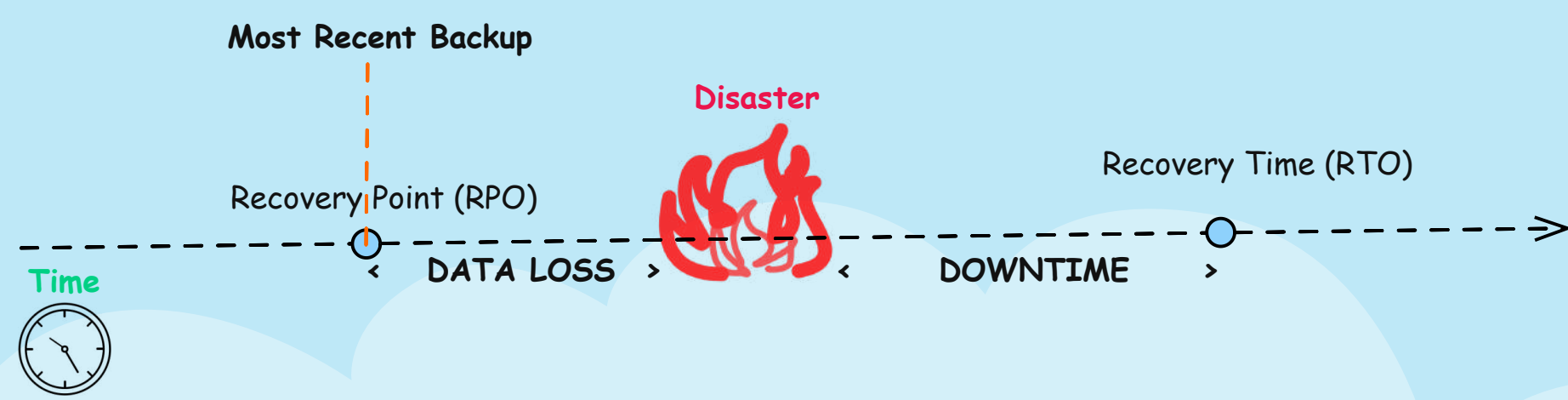
EC2 Main Cost Factors:
Instance family
Attached EBS
Duration of EC2 runtime

Correct Answer:
It depends on the application. Both Lambda and EC2 have different cost factors (see above). It is possible that, depending on the application, AWS Lambda can have a higher charge than EC2, and vice versa. it is important to consider not just the compute cost but the TCO (Total Cost of Ownership). With AWS Lambda there is no AMI to maintain, patch, and rehydrate, reducing management overhead and hence overall TCO.

RPO Vs RTO



CLOUD WITH RAJ
www.cloudwithraj.com

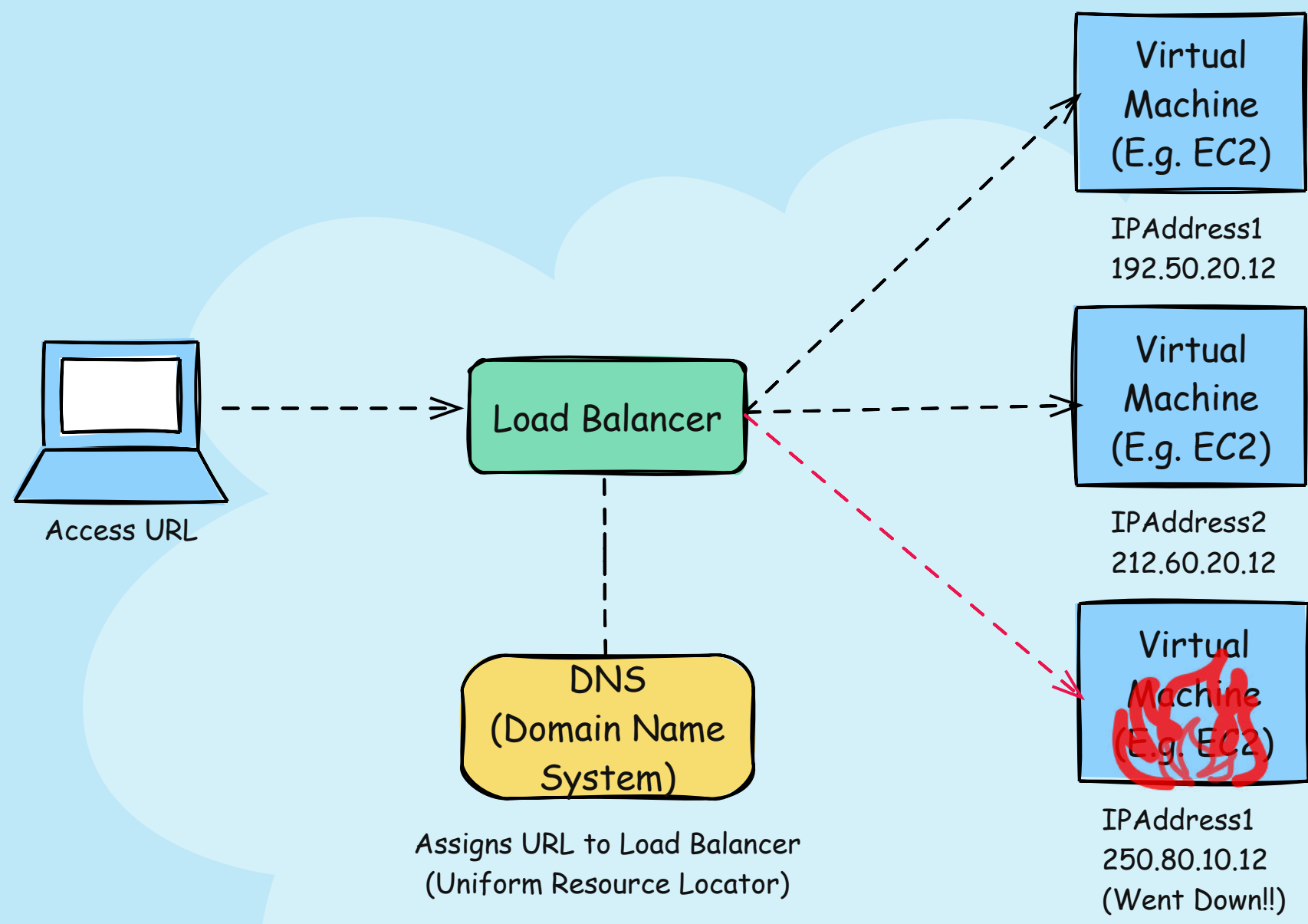


Candidates are sometimes confused by RPO thinking it's measured in unit of data, e.g. gigabyte, petabyte etc.

Correct Answer:

Both RPO and RTO are measured in time. RTO stands for Recovery Time Objective and is a measure of how quickly after an outage an application must be available again. RPO, or Recovery Point Objective, refers to how much data loss your application can tolerate. Another way to think about RPO is how old can the data be when this application is recovered i.e. the time between the last backup and the disaster. With both RTO and RPO, the targets are measured in hours, minutes, or seconds, with lower numbers representing less downtime or less data loss. RPO and RTO can be and often have different values for an application.

IP Address Vs. URL



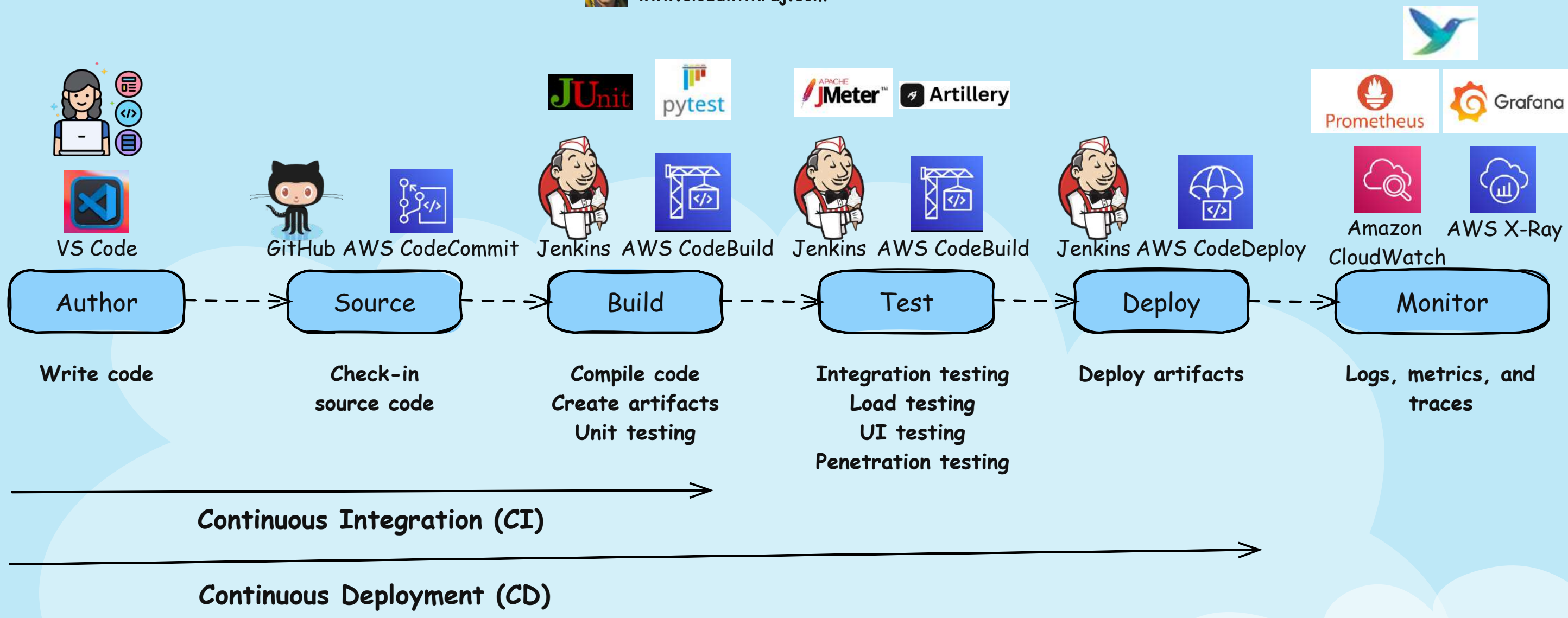
Bad Answer:
URL is a link assigned to an IP address

Correct Answer:
IP Address is a unique number that identifies a device connected to the internet, such as a Virtual Machine running your application. However, accessing a resource using this unique number is cumbersome; moreover, let's say when a VM comes down (the bottom one in the diagram), a new VM comes up to replace it with a different IP address. Hence, in reality, application running inside the VM is accessed using URL or Uniform Resource Locator.

One URL does generally NOT map to one IP address; rather, the URL (e.g., www.amazon.com) is mapped to a Load Balancer, and that Load Balancer distributes traffic to multiple VMs with different IP addresses. Even if one VM goes down and another comes up, this Load Balancer using a URL always works because the Load Balancer appropriately distributes traffic across healthy instances. This way, you (the user) do not need to worry about the underlying IP addresses.

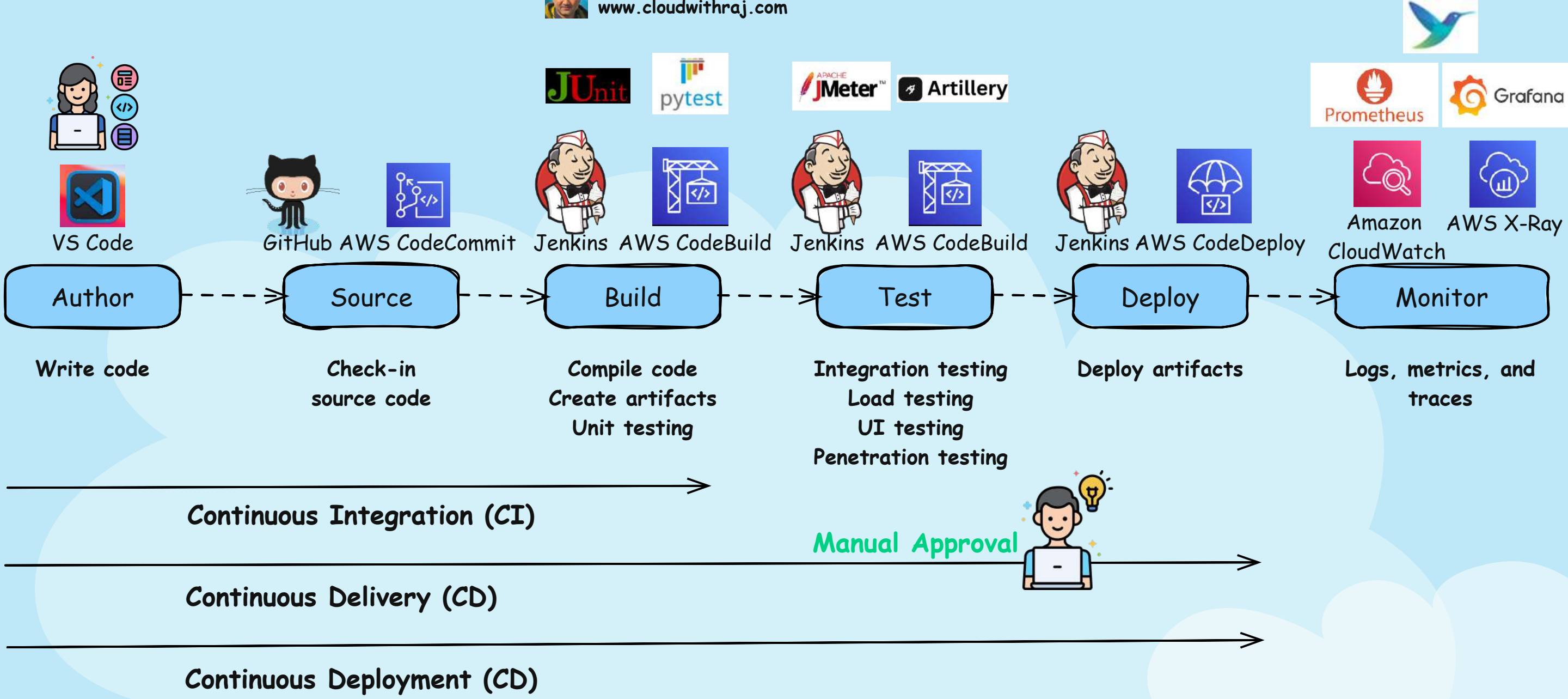
DevOps CICD Phases with Tools

CLOUD WITH RAJ
www.cloudwithraj.com



DevOps CICD Phases

CLOUD WITH RAJ
www.cloudwithraj.com



Kubernetes Tools Ecosystem with AWS

 **CLOUD WITH RAJ**
www.cloudwithraj.com



Cloud Implementation



Amazon EKS

Observability



Prometheus



Grafana



Fluentbit



Jaeger



ADOT



CloudWatch



X-Ray

Scaling



Karpenter



AutoScaling



KEDA

Delivery/Automation



Argo



HELM



Terraform



Jenkins



Github Actions



Gitlab CI/CD

Security



Gatekeeper



Trivy



ECR Scan



GuardDuty



Kube Bench



Secrets Manager



Istio

Cost Optimization



CloudWatch
Container
Insights



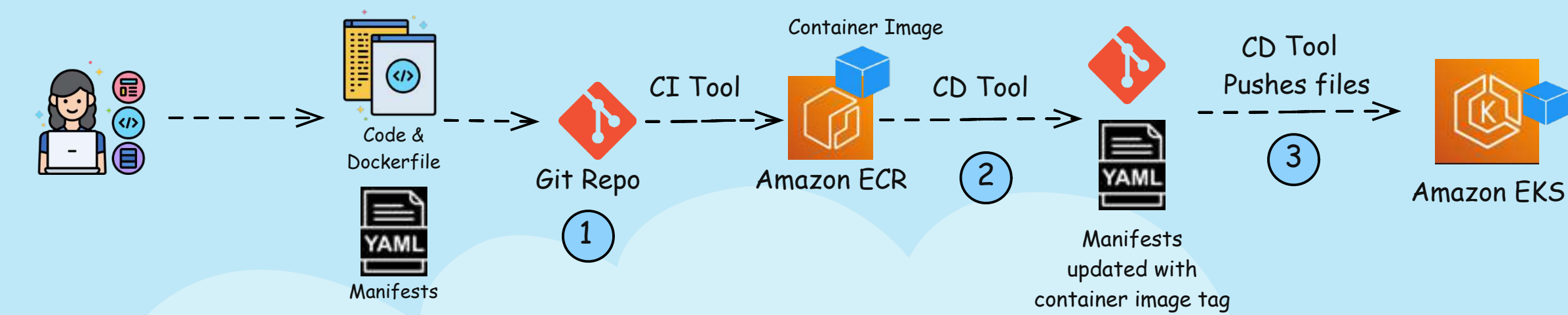
Cost and Usage
Report (New
feature - Split
Cost Allocation)



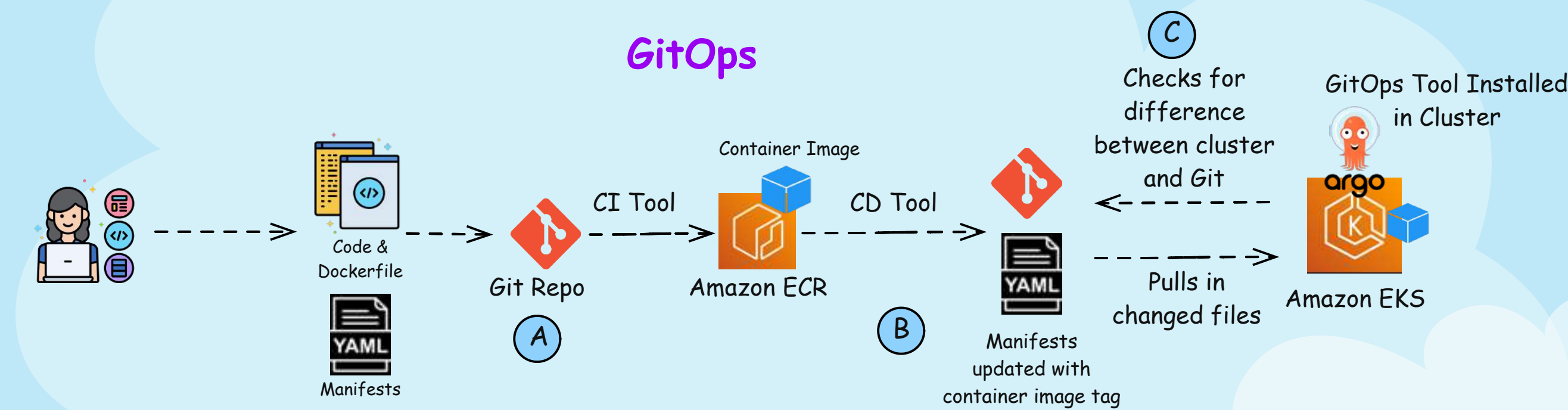
Kubecost

Traditional CICD Vs. GitOps

Traditional CICD



GitOps



Traditional DevOps

Step 1: Developers check in Code, Dockerfile, and manifest YAMLs to an application repository. CI tools (e.g., Jenkins) kick off, build the container image and save the image in a container registry such as Amazon ECR.

Step 2: CD tools (e.g. Jenkins) update the deployment manifest files with the tag of the container image.

Step 3: CD tools (e.g. Jenkins) execute the command to deploy the manifest files into the cluster, which, in terms, deploys the newly built container in the Amazon EKS cluster.

Conclusion - Traditional CICD is a push based model. If a sneaky SRE changes the YAML file directly in the cluster (e.g. changes number of replica, or even the container image itself!), the resources running in the cluster will deviate from what's defined in the YAML in the Git. Worse case, this change can break something, and DevOps team need to rerun part of the CICD process to push the intended YAMLs to the cluster

GitOps

Step A: Developers check in Code, Dockerfile, and manifest YAMLs to an application repository. CI tools (e.g., Jenkins) kick off, build the container image and save the image in a container registry such as Amazon ECR.

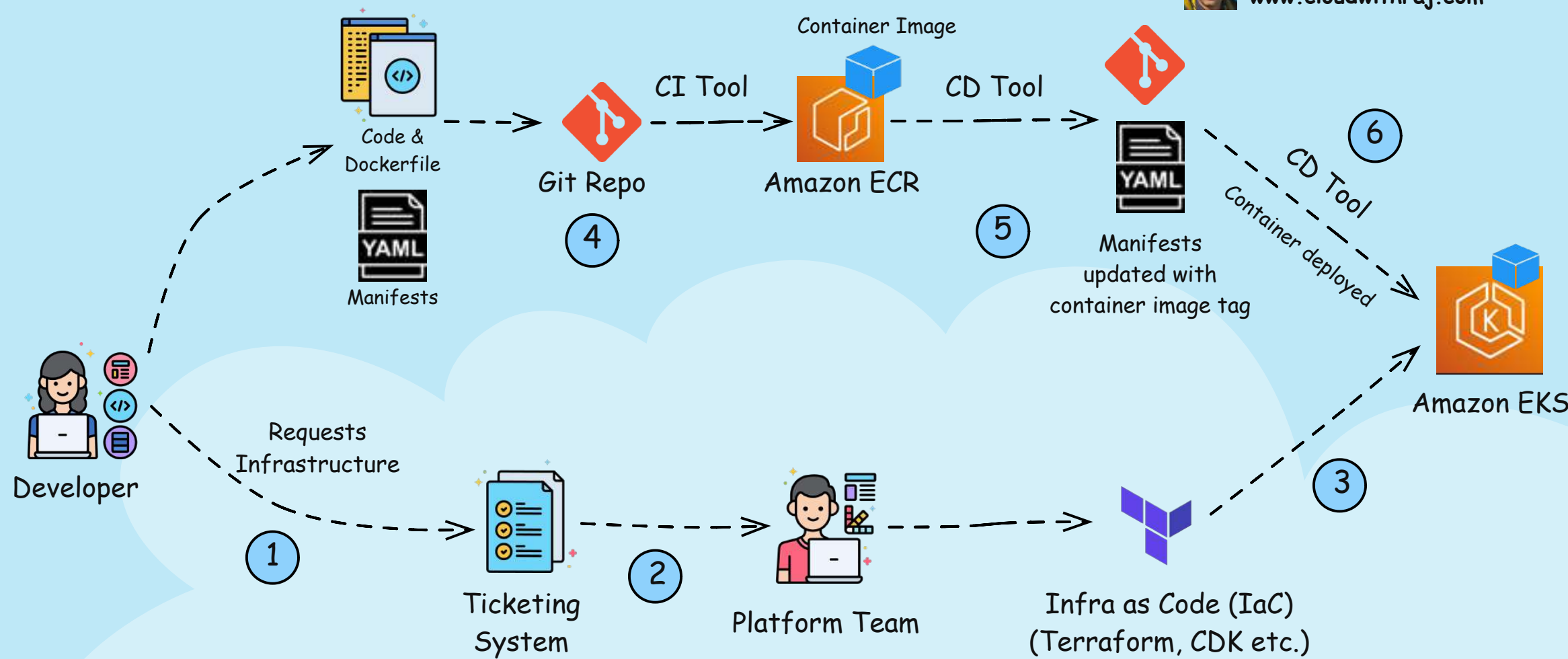
Step B: CD tools (e.g. Jenkins) update the deployment manifest files with the tag of the container image.

Step C: With GitOps, Git becomes the single source of truth. You need to install a GitOps tool like Argo inside the cluster and point to a Git repo. Git keeps checking if there is a new file, or if the files in the cluster drifts from the ones in Git. As soon as YAML is updated with new container image, there is a drift between what's running in the cluster vs what's in Git. ArgoCD pulls in this updated YAML file and deploys new container.

Conclusion - GitOps does NOT replace DevOps. As you can see GitOps only replaces part of the CD process. If we think about the previous scenario where the sneaky SRE directly changes the YAML in cluster, ArgoCD will detect the mismatch between the changed file vs the one in Git. Since there is a difference, it will pull in the file from Git and bring Kubernetes resources to it's intended state. And don't worry, Argo can also send a message to the sneaky SRE's manager ;).

Platform Team and Developer Team

 **CLOUD WITH RAJ**
www.cloudwithraj.com



Recently, the term "platform team" has been floating around plenty. But what do platform team do? How are they different from the developer team? Let's understand with the diagram below:

Step 1: The developer team requests the Platform team to provision appropriate AWS resources. In this example, we are using Amazon EKS for the application, but this concept can be extended to any other AWS service. This request for AWS resources is typically done via the ticketing system.

Step 2: The platform team receives the request.

Step 3: The platform team uses Infrastructure as Code (IaC), such as Terraform, CDK, etc., to provision the requested AWS resources, and share the credentials with the Developer team.

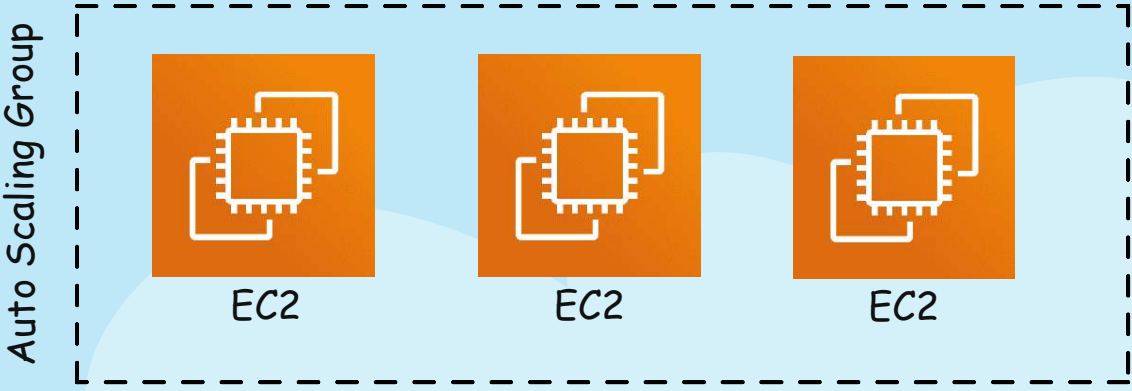
Step 4: The developer team kicks off the CI/CD process. We are using a container process to understand the flow. Developers check in Code, Dockerfile, and manifest YAMLs to an application repository. CI tools (e.g., Jenkins, GitHub actions) kick off, build the container image and save the image in a container registry such as Amazon ECR.

Step 5: CD tools (e.g. Jenkins, Spinnaker) update the deployment manifest files with the tag of the container image.

Step 6: CD tools execute the command to deploy the manifest files into the cluster, which, in terms, deploys the newly built container in the Amazon EKS cluster.

Conclusion - The platform team takes care of the infrastructure (often with the guardrails) appropriate for the organization, and the developer team uses that infrastructure to deploy their application. The platform team does the upgrade and maintenance of the infrastructure to reduce the burden on the developer team.

Scaling Difference Between Lambda, EC2, EKS



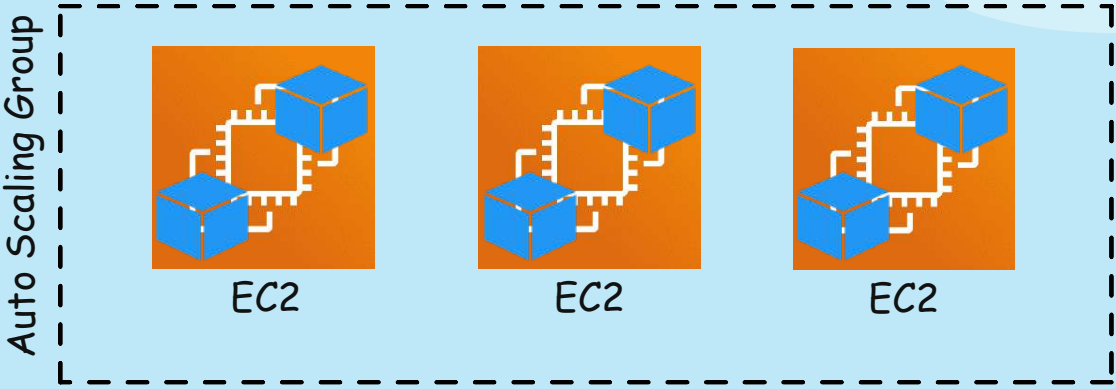
EC2 Scaling: You need to use a Auto Scaling Group (ASG) and define on what EC2 metric you want it to scale e.g. CPU utilization. You can use ASG's "minimum number of instances" to run certain number of instances on all time. Recently ASG also supports scheduled scaling, and warm pool.



Lambda Scaling: No ASG needed. For each incoming connection, Lambda automatically scales. Consider increasing the concurrency setting for Lambda as needed. Implement Provisioned Concurrency to keep certain number of Lambda pre-warmed. This can be done either with schedule or based on Provisioned Concurrency utilization.

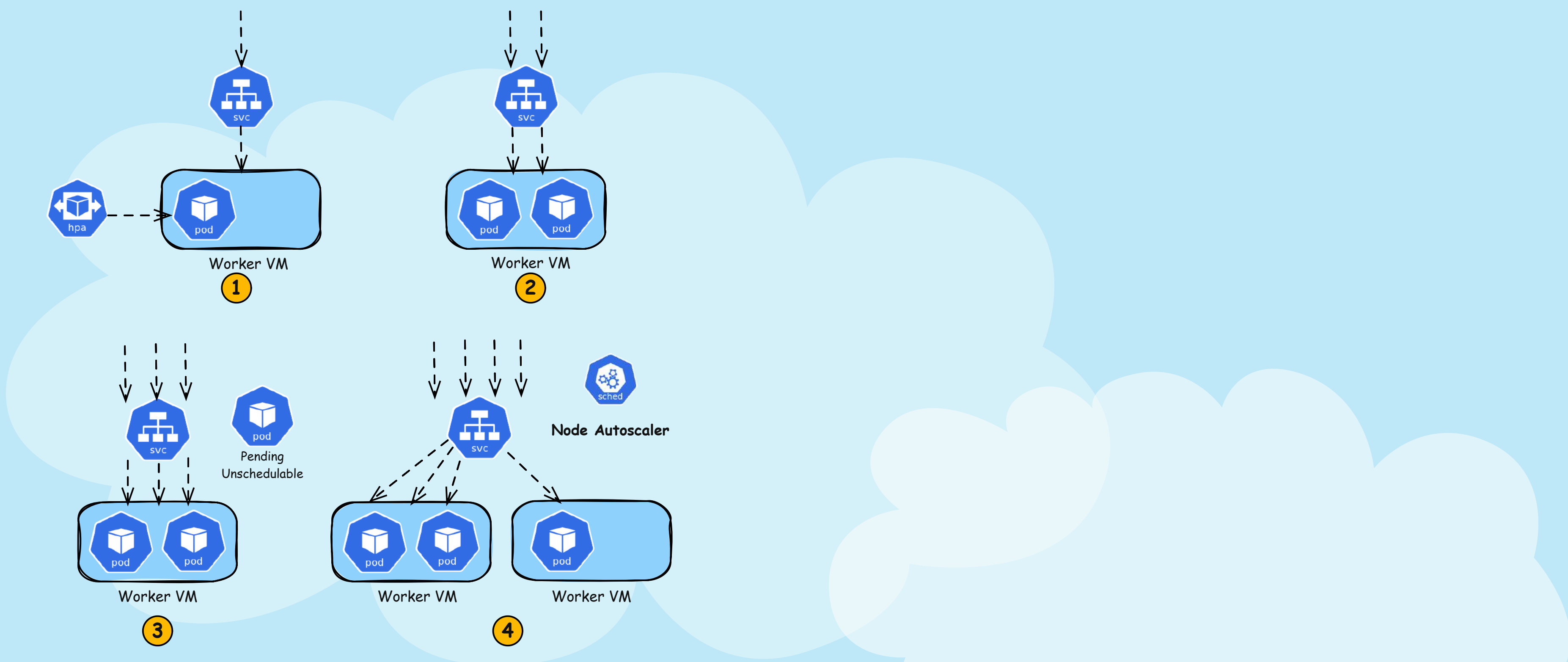


EKS



EKS Scaling: EKS scaling is the most complex scaling. You may or may NOT use a Auto Scaling Group but it does NOT work like regular EC2 scaling. Please refer to the next page to learn about EKS (Kubernetes/K8s) scaling in detail

How Does Kubernetes Worker Nodes Scale?



Incorrect Answer:
Set Auto Scaling Groups to scale at a certain VM metric utilization like scaling regular VMs.

Correct Answer:
Step 1: You configure HPA (Horizontal Pod Autoscaler) to increase the replica of your pods at a certain CPU/Memory/Custom Metrics threshold.

Step 2: As traffic increases and the pod metric utilization crosses the threshold, HPA increases the number of pods. If there is capacity in the existing worker VMs, then the Kubernetes kube-scheduler binds that pod into the running VMs.

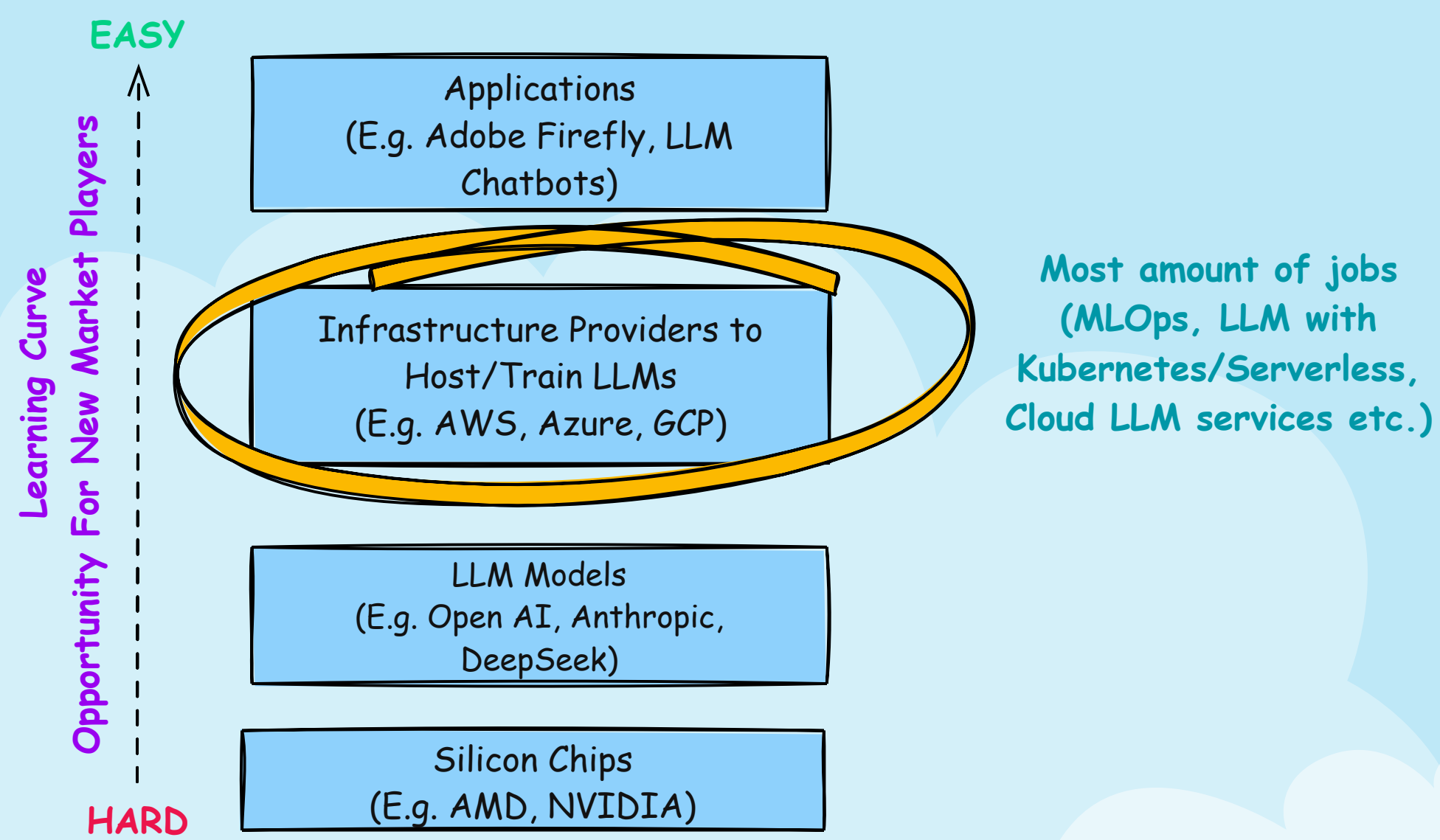
Step 3: Traffic keeps increasing, and HPA increases the number of replicas of the pod. But now, there is no capacity left in the running VMs, so the kube-scheduler can't schedule the pod(yet!). That pod goes into pending, unschedulable state

Step 4: As soon as pod(s) go to pending unschedulable state, Kubernetes node scalers (such as Cluster Autoscaler, Karpenter etc.) provisions a new node. Cluster Autoscaler requires an Auto Scaling Group where it increases the desired VM count, whereas Karpenter doesn't require an Auto Scaling Group or Node Group. Once the new VM comes up, kube-scheduler puts that pending pod into the new node.

Gen AI 4 Layers



CLOUD WITH RAJ
www.cloudwithraj.com



Gen AI hype is at an all-time high, and so is the confusion. What do you study, how do you think about it, and where are the most jobs? These are the burning questions in our minds. Gen AI can be broken down into the following four layers.

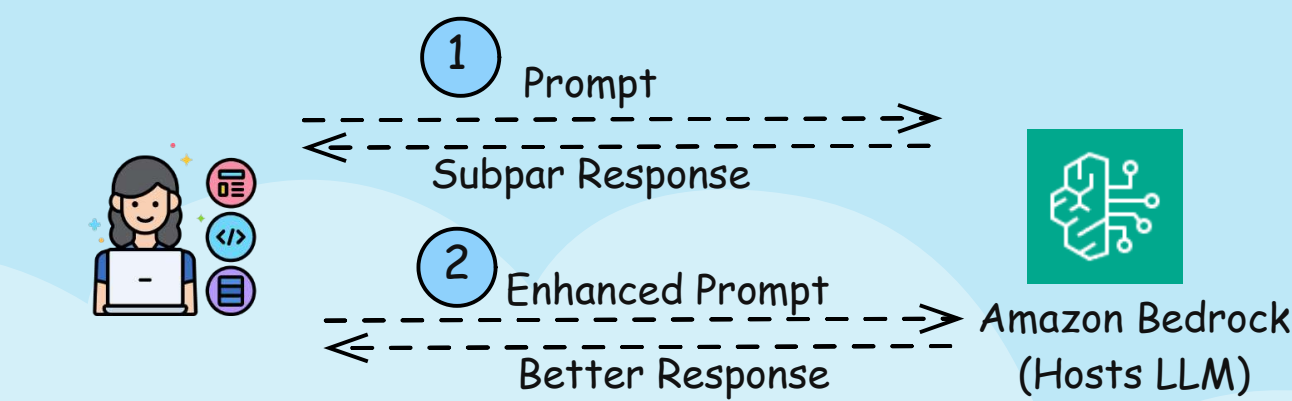
1. The bottom layer is the hardware layer, i.e., the silicon chips that can train the models. Example - AMD, NVIDIA
2. Then comes the LLM models that get trained and run on the chips. Examples are Open AI, Anthropic etc.
3. Then comes infrastructure providers who provide an easier way to consume, host, train, inference the models. Example is AWS. This layer consists of managed services such as Amazon Bedrock, which hosts pre-trained models, or provision VMs (Amazon EC2) where you can train your own LLM
4. Finally, we have the application layer which uses those LLMs. Some examples are Adobe Firefly, LLM chatbots, LLM travel agents etc.

Now, the important part - as you go from the bottom to the top, the learning curve gets easier, and so does the opportunity for new market players to enter. Building new chips requires billions of dollars of investments, and hence, it's harder for new players to enter the market. The most opportunities are in the top two layers. If you already know the cloud, then integrating Gen AI with your existing knowledge will increase your value immensely. If you are working in DevOps, learn MLOps; if you know K8s/Serverless, learn how you can integrate Gen AI with those; if you work in an application; integrate with managed LLM services to enhance functionality, you got the idea!

Prompt Engineering Vs RAG Vs Fine Tuning

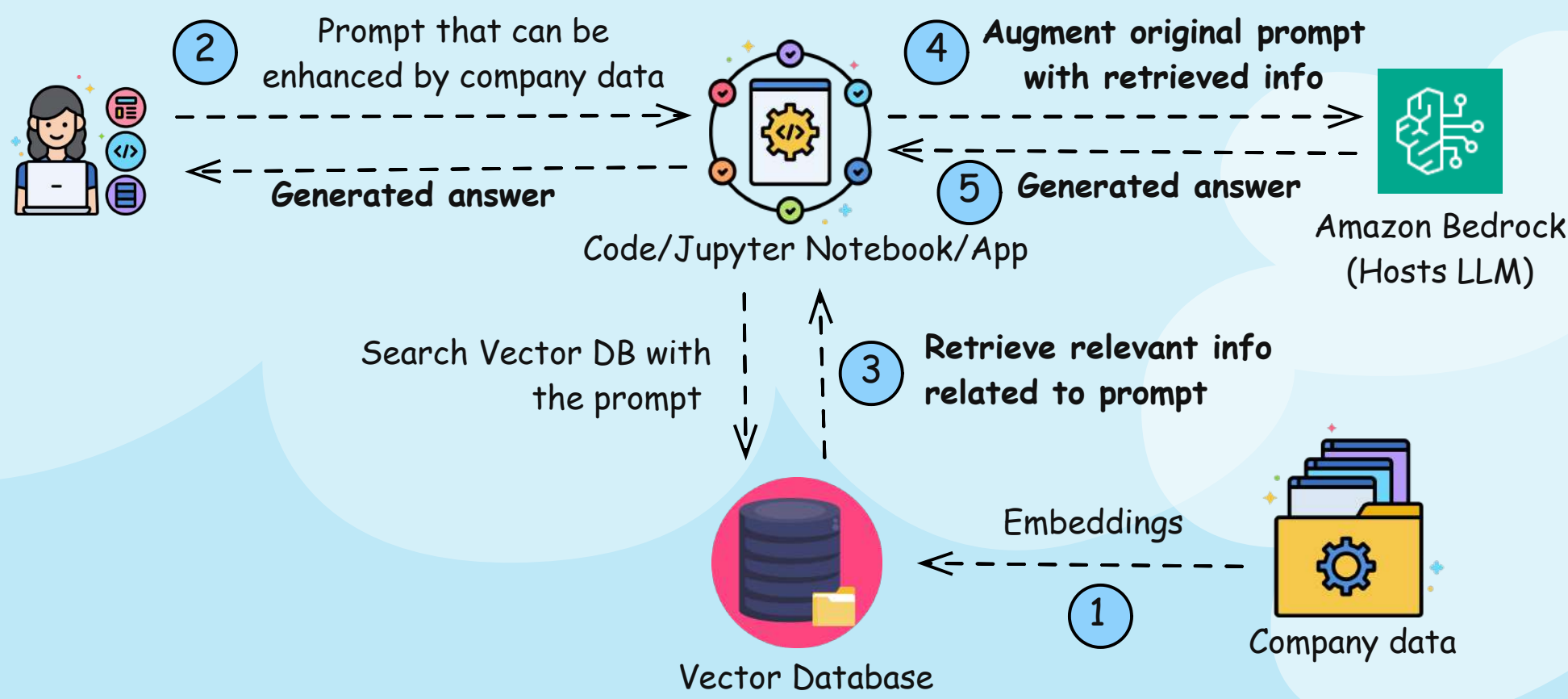


Prompt Engineering



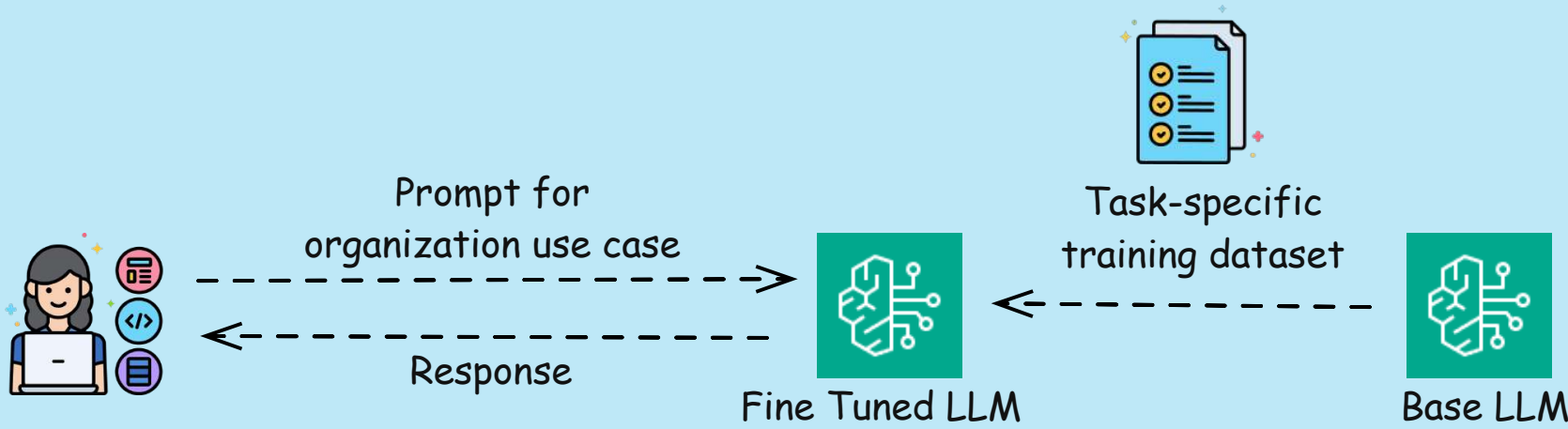
- 1. You send a prompt to the LLM (hosted in Amazon Bedrock in this case), and get a response which you are not satisfied with
- 2. You enhance the prompt, and finally come up with a prompt that gives desired better response

RAG (Retrieval Augmented Generation)



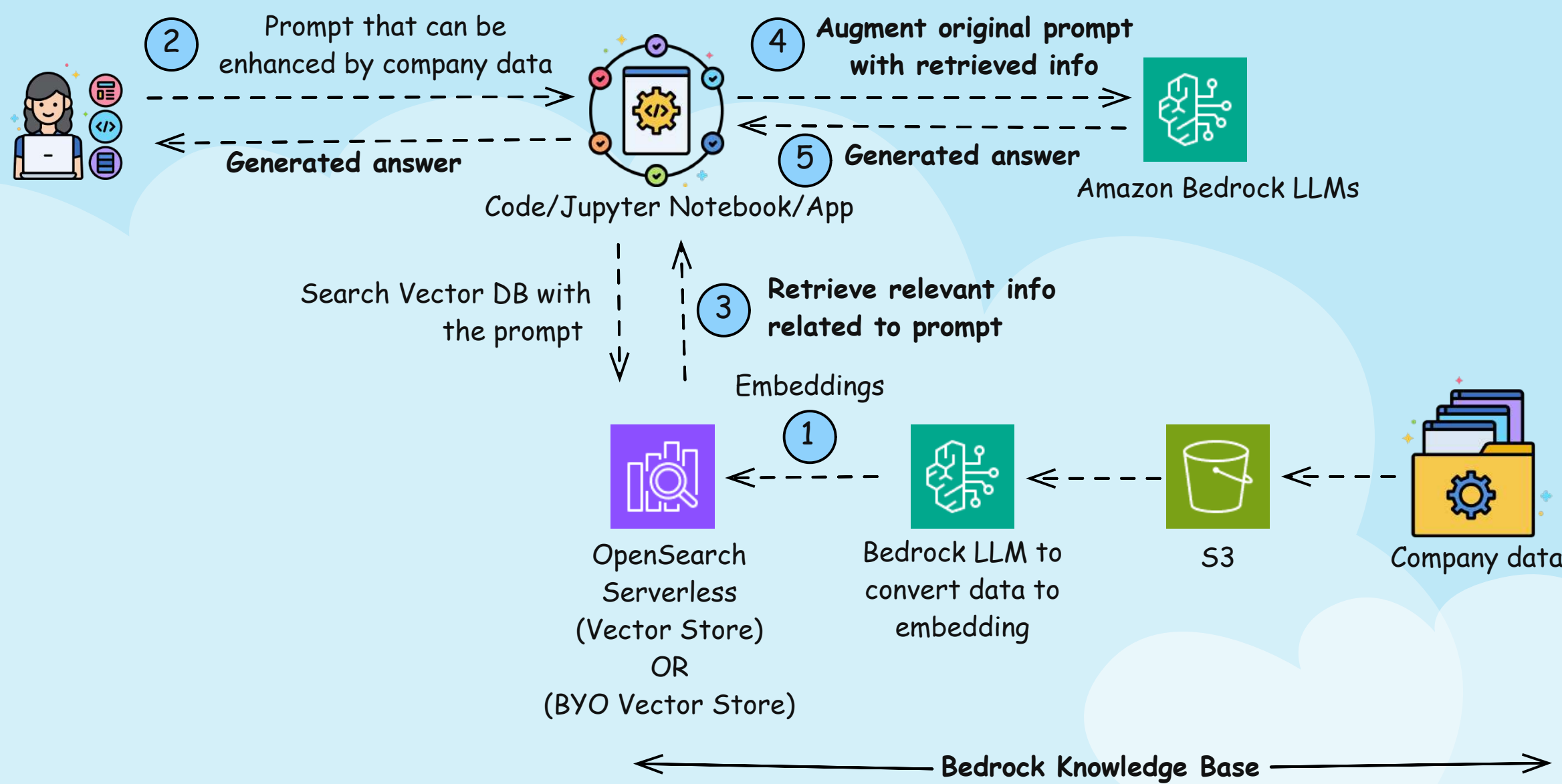
- 1. RAG (Retrieval Augmented Generation) is used where the response can be made better by using company specific data that the LLM does NOT have. You store relevant company data into a vector database. This is done by a process called embeddings where data is transformed into numeric vectors
- 2. User gives a prompt which can be made better by adding company specific info
- 3. A process (code/jupyter notebook/application) converts the prompt into vector and then search the vector database. Relevant info from the vector database is RETRIEVED (First Part of RAG) and returned
- 4. The original prompt is AUGMENTED (Second part of RAG) with this company specific info and sent to LLM
- 5. LLM GENERATES (Last part of RAG) the response and sends back to the user

Fine Tuning



- 1. If you need a LLM which is very specific to your company/organization's use case that RAG can't solve, you train the base LLM with large training dataset for the tasks. The output is a fine tuned LLM.
- 2. User asks question related to the use case and gets answer

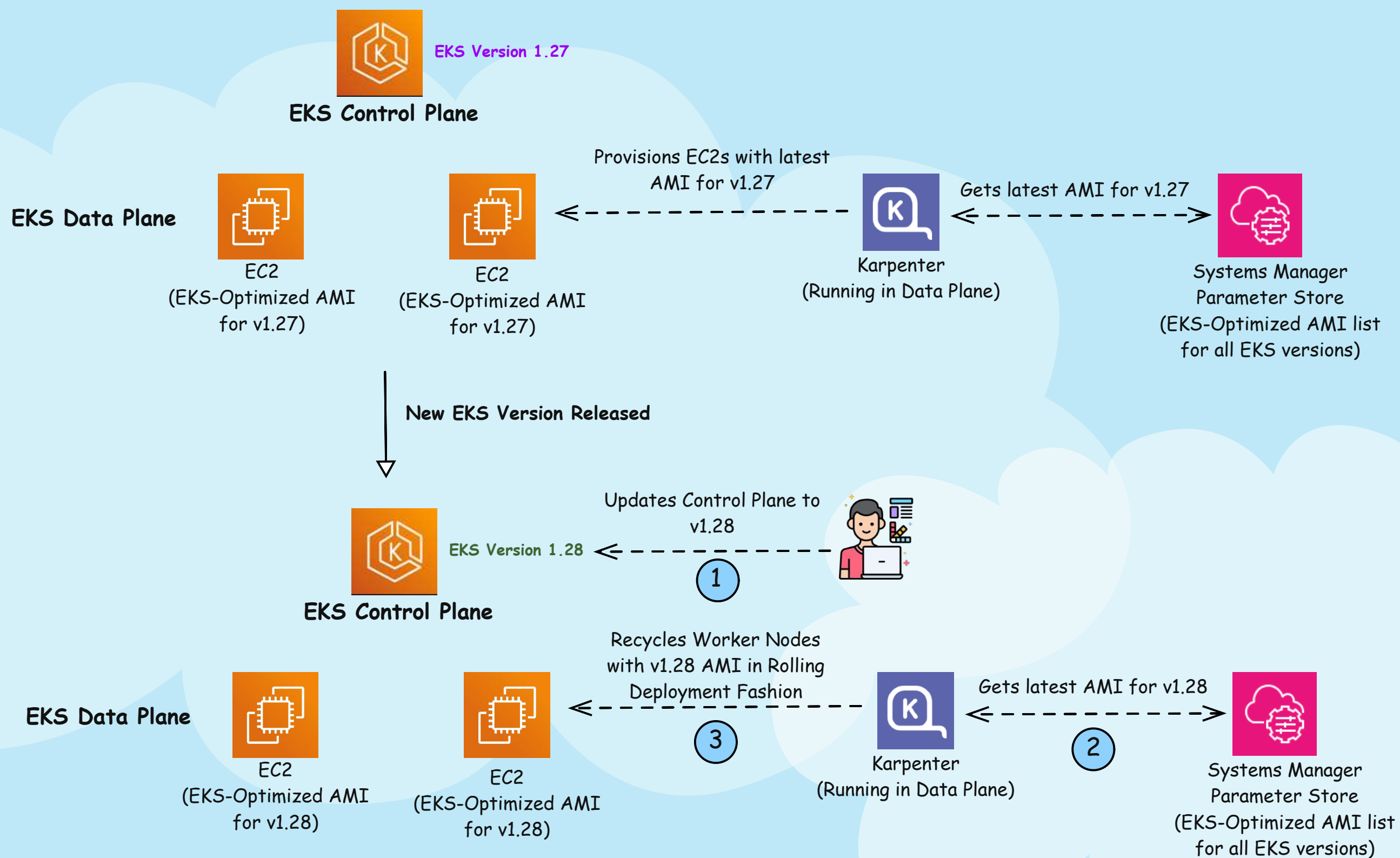
RAG (Retrieval Augmented Generation) with Amazon BedRock



RAG (Retrieval Augmented Generation) is used where the response can be made better by using company specific data that the LLM does NOT have. Amazon BedRock makes it very easy to do RAG. Below are the steps:

1. You store relevant company data into a S3 bucket. Then from BedRock Knowledge bases, you select an emedding LLM (Amazon Titan Embed or Cohere Embed) which converts the S3 data into embeddings (vector). Knowledge Base can also create a serverless vector store for you to save those embeddings. Alternatively you can also bring your own Vector Database (OpenSearch, Aurora, Pinecone, Redis)
2. User gives a prompt which can be made better by adding company specific info
3. A process (code/jupyter notebook/application) converts the prompt into vector and then search the vector database. Relevant info from the vector database is RETRIEVED (First Part of RAG) and returned
4. The original prompt is AUGMENTED (Second part of RAG) with this company specific info and sent to another Bedrock LLM
5. BedRock LLM GENERATES (Last part of RAG) the response and sends back to the user

EKS Upgrade Simplified Using Karpenter



EKS upgrade can be tedious. But Karpenter can automatically upgrade your Data Plane worker nodes reducing your burden. Here's how:

a. EKS Optimized AMI IDs are listed in AWS Systems Manager parameter store. CNCF project Karpenter, the next gen cluster autoscaler, periodically checks this, and reconciles with the running worker nodes to see if they are running with the latest EKS-Optimized AMI for the particular EKS version. In this case, let's assume EKS is running with v1.27.

b. At a certain point, EKS releases next version 1.28. And the below workflow takes place:

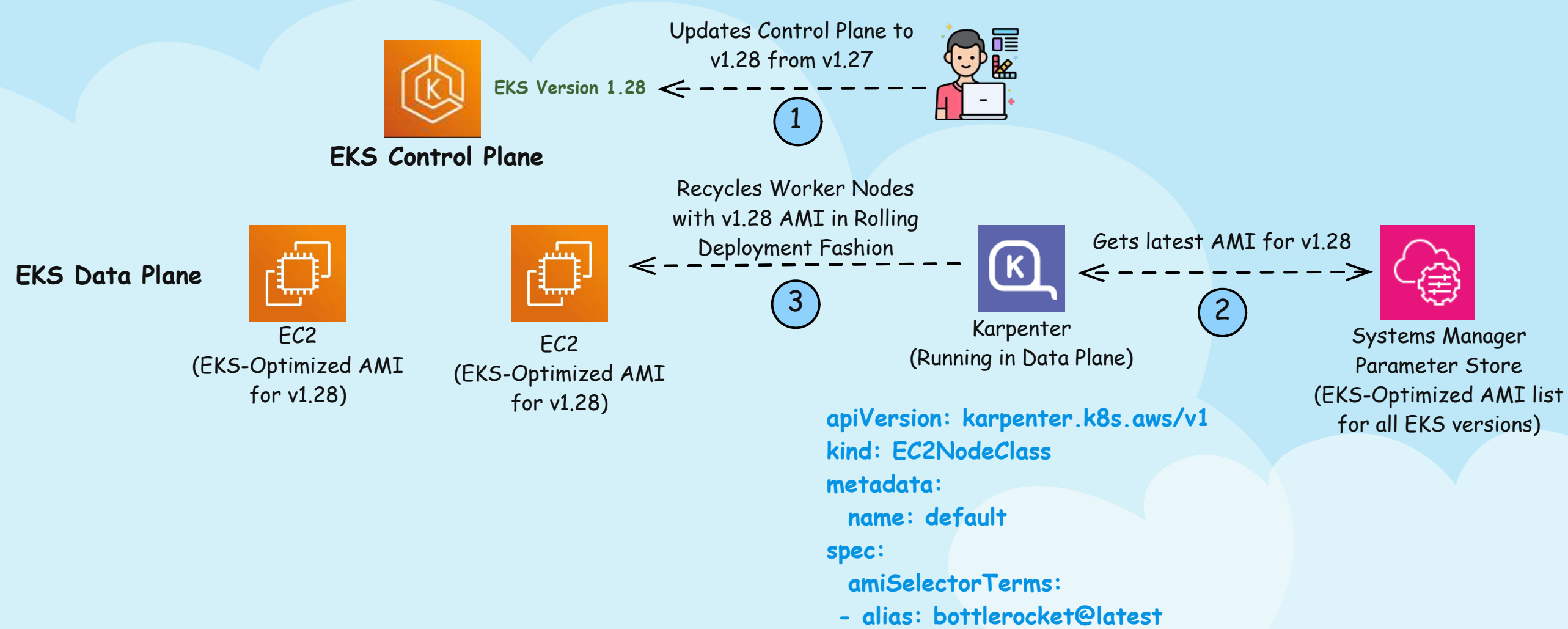
1. Admin upgrades the EKS control plane to v1.28.
2. Following the previous logic, Karpenter retrieves the latest AMI for v1.28 and check if worker nodes are running with those. They are NOT, so a Karpenter Drift is triggered.
3. To fix this Drift, Karpenter automatically updates the worker nodes to v1.28 AMIs. And it does it using rolling deployment (i.e. a new node comes up, existing node cordoned and drained, then terminated). It also respects the Kubernetes eviction API parameters, such as maintaining PDB.

If you want to know the process in detail, including with custom AMIs, check out the Karpenter drift blog - <https://aws.amazon.com/blogs/containers/how-to-upgrade-amazon-eks-worker-nodes-with-karpenter-drift/>

EKS Upgrade Using Karpenter - Automatic and Manual

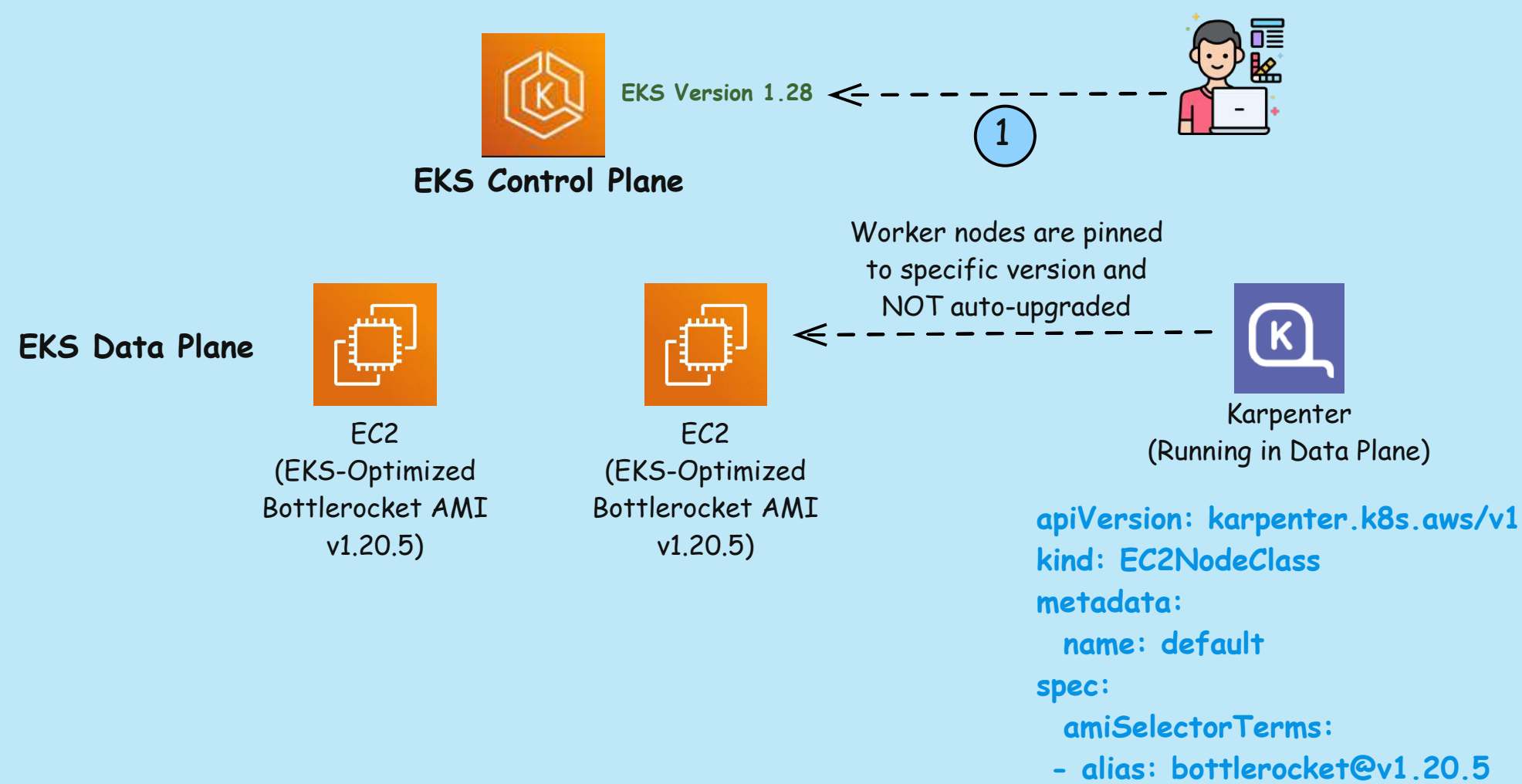


Automatic



The "alias: bottlerocket@latest" in EC2NodeClass ensures Karpenter will always automatically upgrade your worker nodes if AWS releases a new AMI OR you upgrade your EKS Control Plane

Manual (More Control)



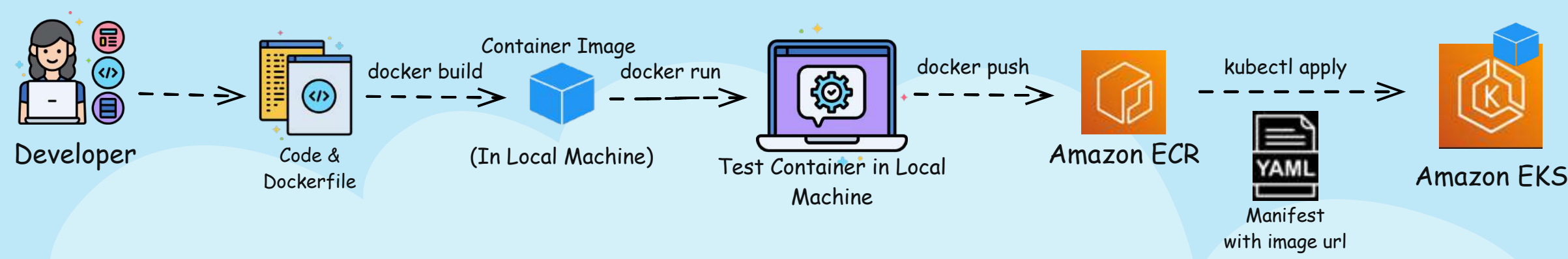
Karpenter can automatically upgrade your K8s worker nodes. What if you don't want to - that's possible too. Here's how:

You can pin your worker nodes to a specific version of the AMI using Karpenter EC2NodeClass. For example, in the below diagram, Karpenter will provision the worker nodes with Amazon EKS BR Optimized Linux AMI for version v1.20.5.

Even after AWS releases a new AMI or you upgrade your EKS Control plane, Karpenter will NOT upgrade data plane. Remember that - EKS Data Plane can run with AMIs 3 versions behind the Control Plane. Though it is recommended to run with latest version due to security patches.

The general practice I see for critical projects with my customers is - that they will let Karpenter automatically upgrade to the latest AMI version in dev/test and pin to a specific AMI in prod. Once the newest version is tested in dev/test, production EC2NodeClass will be changed to point to this new version. Once EC2NodeClass is updated, Karpenter will upgrade the worker node AMIs in a rolling deployment fashion.

Container Lifecycle - Local to Cloud



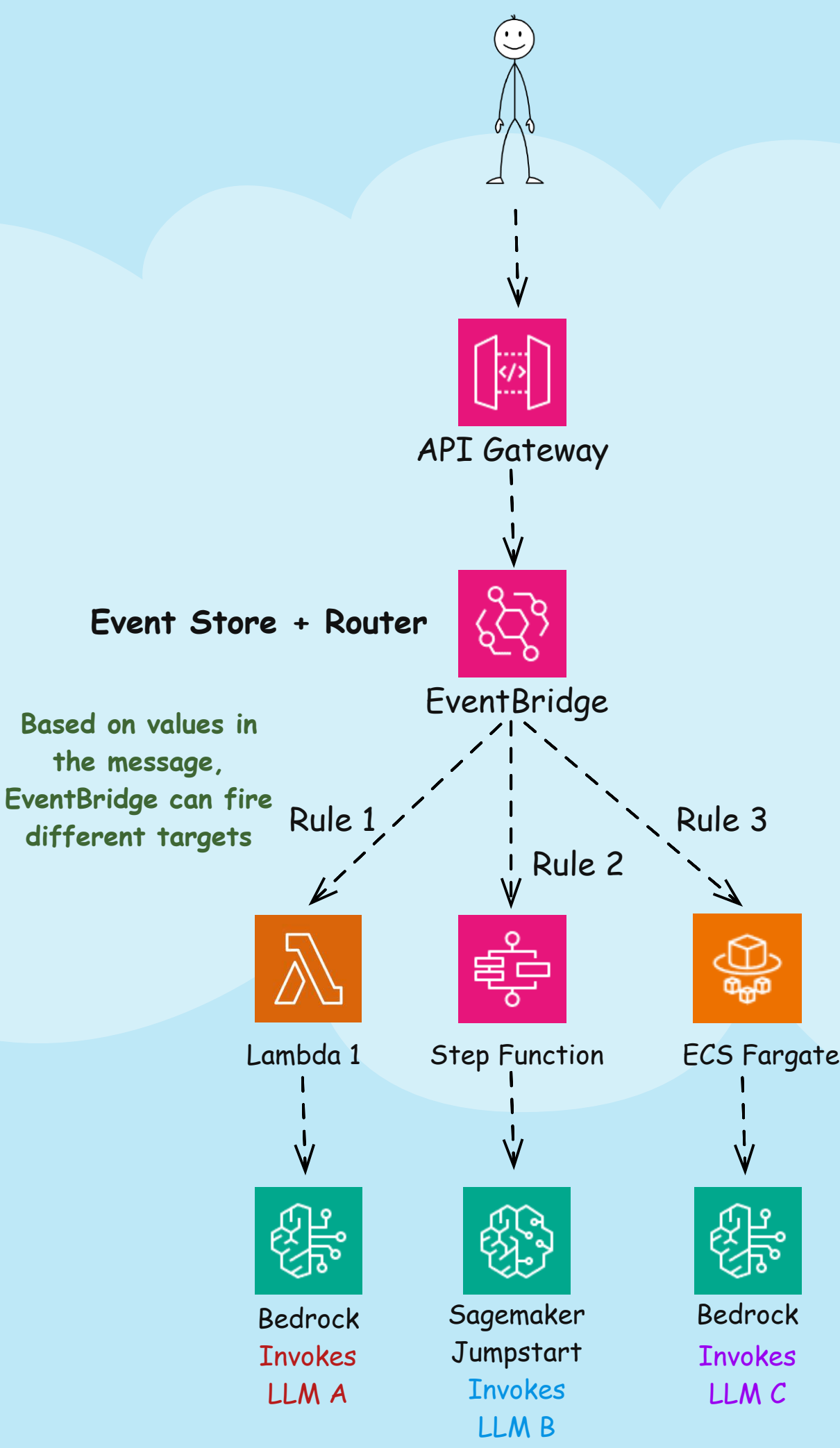
The fundamental container workflow from local machine to cloud is below:

1. Developer writes code, and associated Dockerfile to containerize the code in her local machine
2. She uses "Docker build" command to create the container image, in her local machine. At this point container image is saved in the local machine
3. Developer uses "Docker run" command to run the container image, and test out the code running from the container. Developer can repeat Steps 1-3, till the testing goes as per the requirements
4. Next, developer runs "Docker push" command to push the container image from the local machine to a container registry. Some examples are DockerHub, or Amazon ECR.
5. Finally, using "KubectI apply" command, an YAML manifest which has the URL of the container image from the Amazon ECR, is deployed into the running Kubernetes cluster.

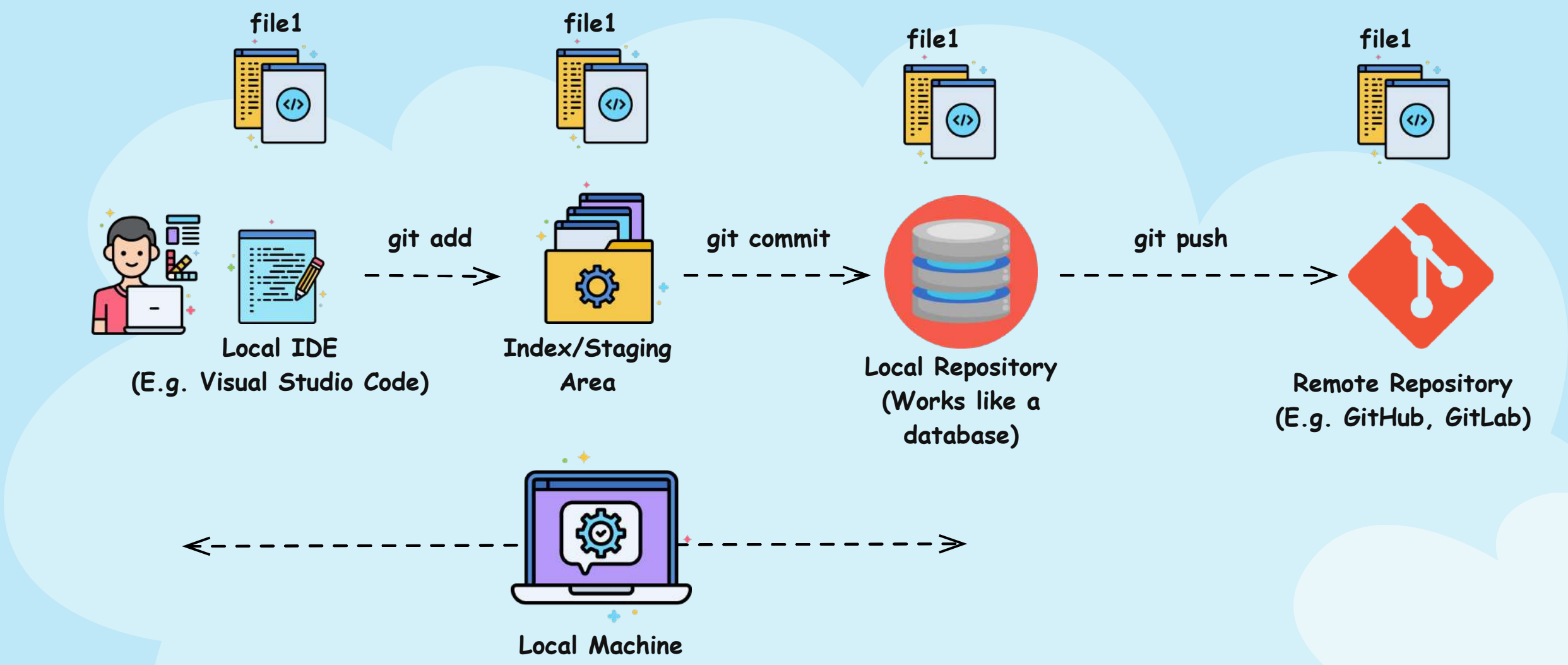
Note that, this is to understand the lifecycle of the container. In real-world, after testing is done in local machine, the following steps are automated. Refer to the "Traditional CI/CD vs GitOps" page for that workflow

Gen AI Multi Model Invocation

CLOUD WITH RAJ
www.cloudwithraj.com



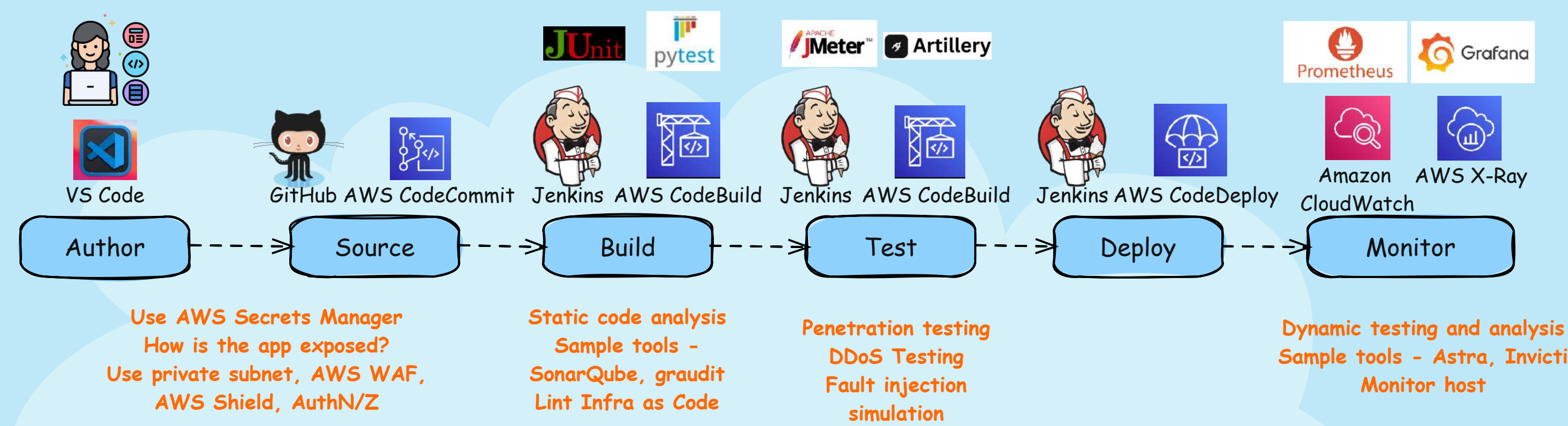
Git Workflow



Git and GitHub for Beginners Crash Course (Click on the YouTube icon):



DevSecOps Workflow

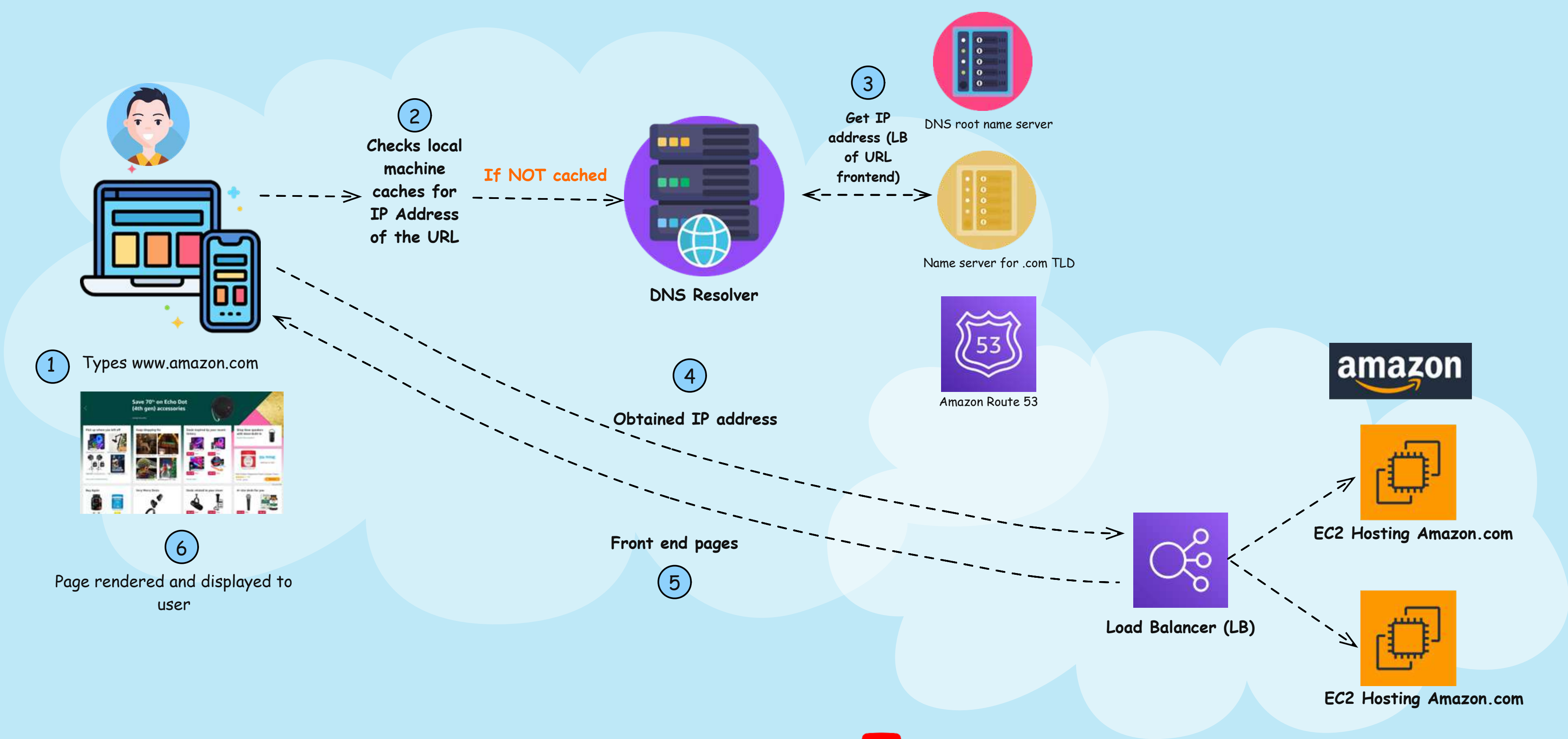


Security Embedded Throughout The Pipeline



What Happens When You Type An URL

CLOUD WITH RAJ
www.cloudwithraj.com



Free video explaining steps in detail: [Click](#) 

Well Architected Framework



CLOUD WITH RAJ
www.cloudwithraj.com

The AWS Well-Architected Framework describes key concepts, design principles, and architectural best practices for designing and running workloads in the cloud. By answering a few foundational questions, learn how well your architecture aligns with cloud best practices and gain guidance for making improvements. AWS Well-Architected Framework assessment questions can be answered inside your AWS account, and then a scorecard can be obtained that evaluates your application in respect to the below six pillars

Operational Excellence Pillar

The operational excellence pillar focuses on running and monitoring systems, and continually improving processes and procedures. Key topics include automating changes, responding to events, and defining standards to manage daily operations.

Security Pillar

The security pillar focuses on protecting information and systems. Key topics include confidentiality and integrity of data, managing user permissions, and establishing controls to detect security events.

Reliability Pillar

The reliability pillar focuses on workloads performing their intended functions and how to recover quickly from failure to meet demands. Key topics include distributed system design, recovery planning, and adapting to changing requirements.

Performance Efficiency Pillar

The performance efficiency pillar focuses on structured and streamlined allocation of IT and computing resources. Key topics include selecting resource types and sizes optimized for workload requirements, monitoring performance, and maintaining efficiency as business needs evolve.

Cost Optimization Pillar

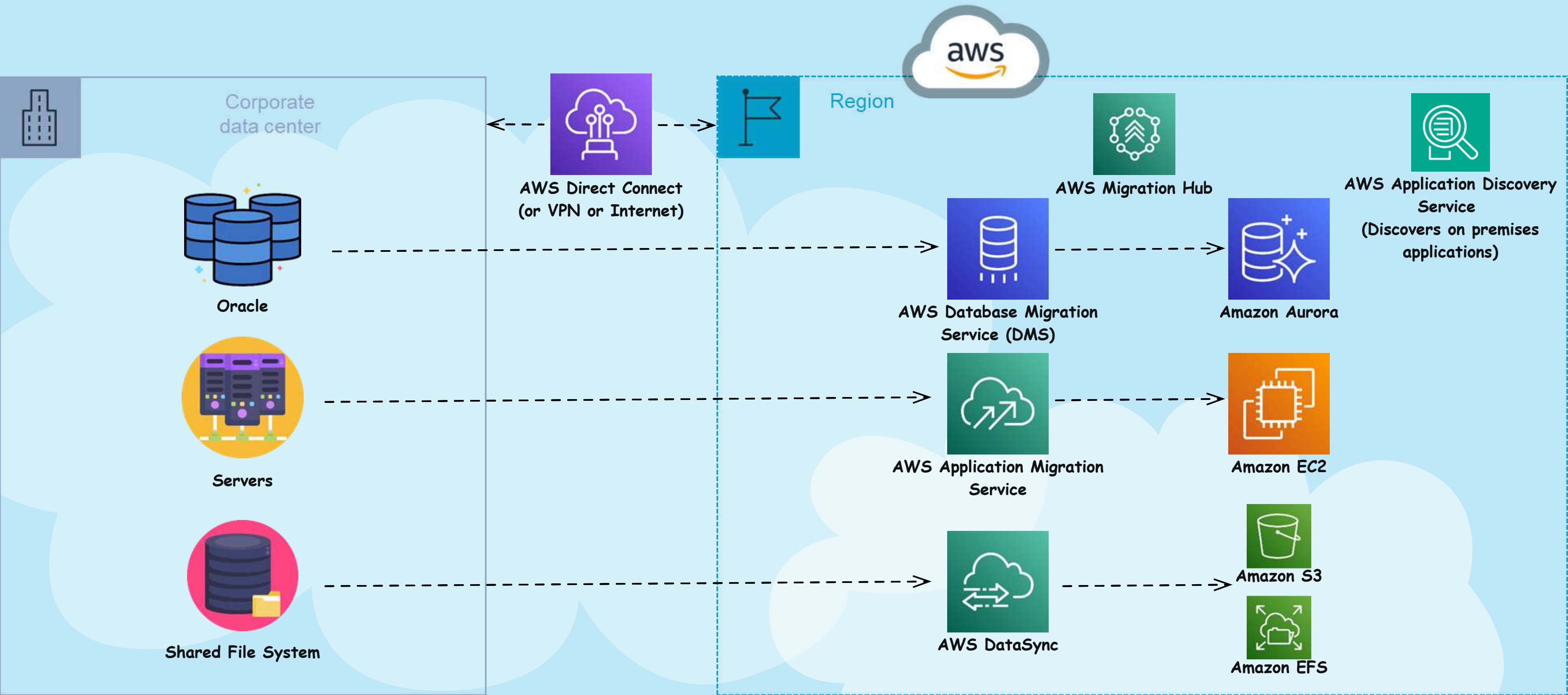
The cost optimization pillar focuses on avoiding unnecessary costs. Key topics include understanding spending over time and controlling fund allocation, selecting resources of the right type and quantity, and scaling to meet business needs without overspending.

Sustainability Pillar

The sustainability pillar focuses on minimizing the environmental impacts of running cloud workloads. Key topics include a shared responsibility model for sustainability, understanding impact, and maximizing utilization to minimize required resources and reduce downstream impacts.

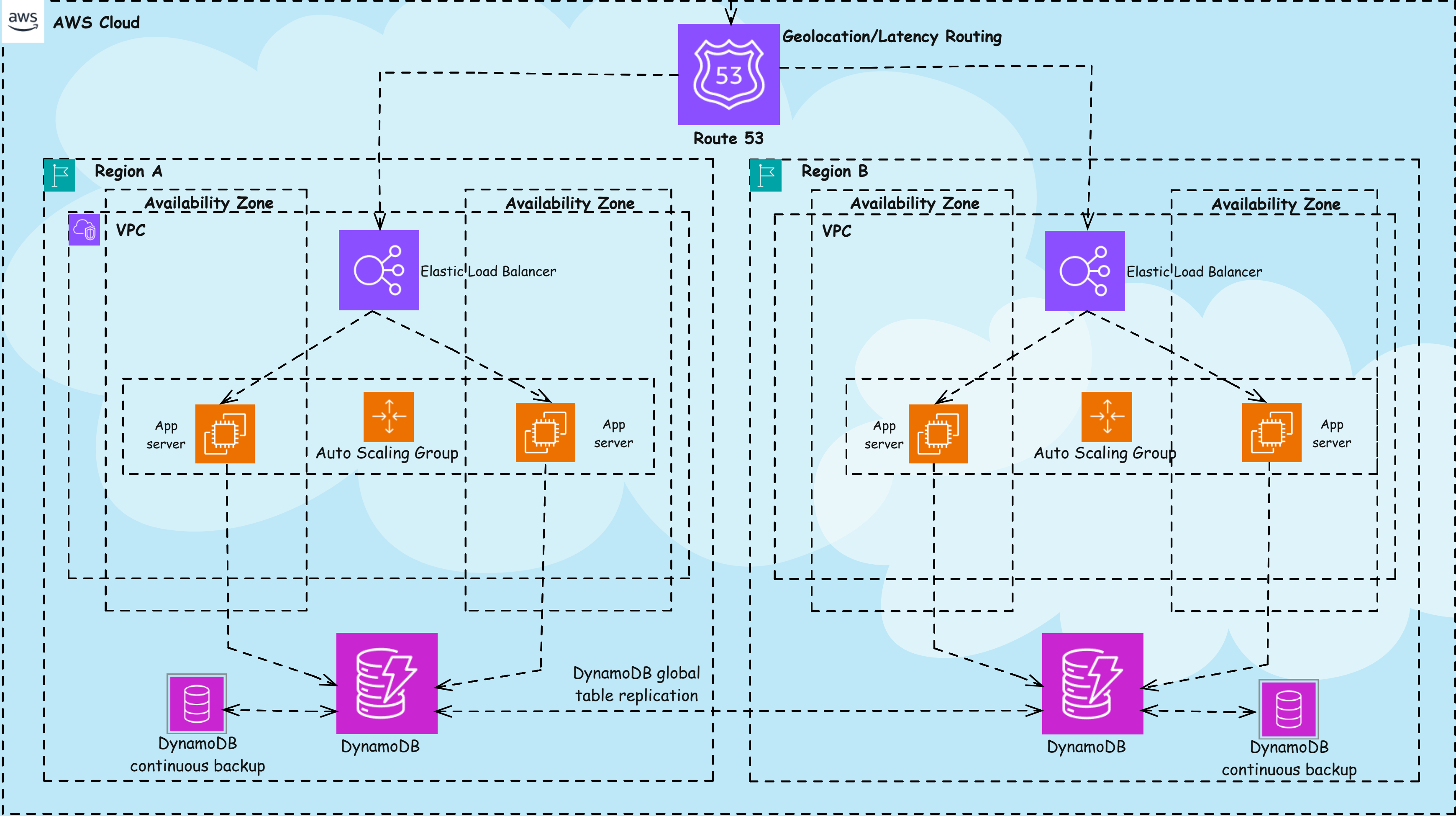
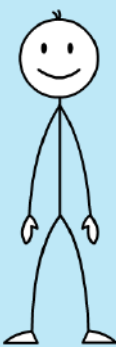
AWS Migration Tools

CLOUD WITH RAJ
www.cloudwithraj.com



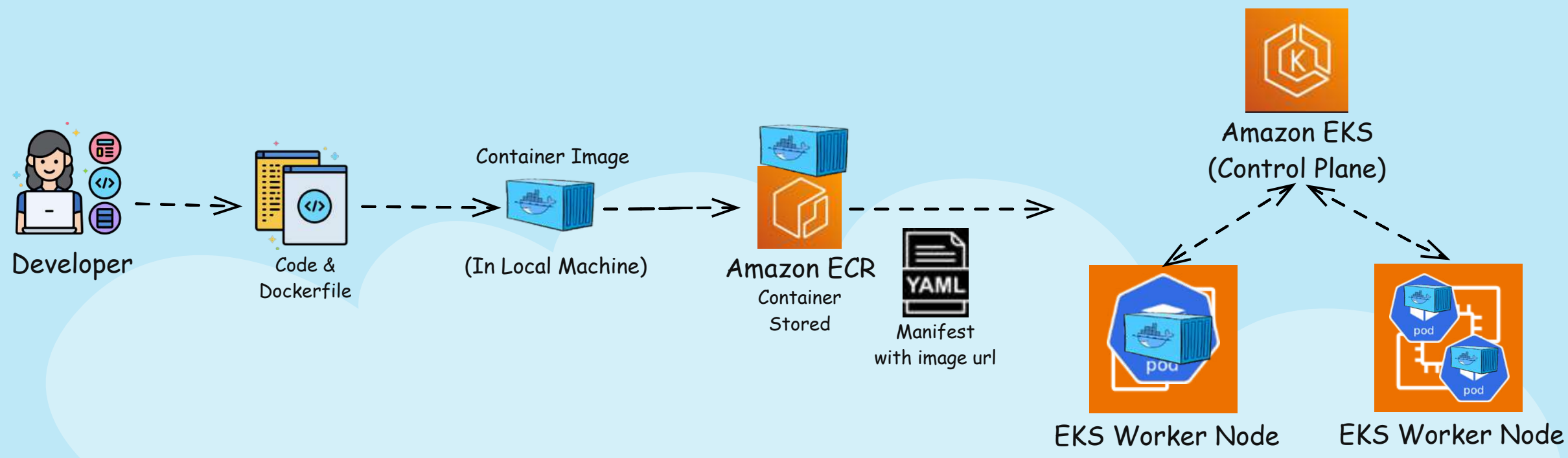
Multi-site Active Active

CLOUD WITH RAJ
www.cloudwithraj.com



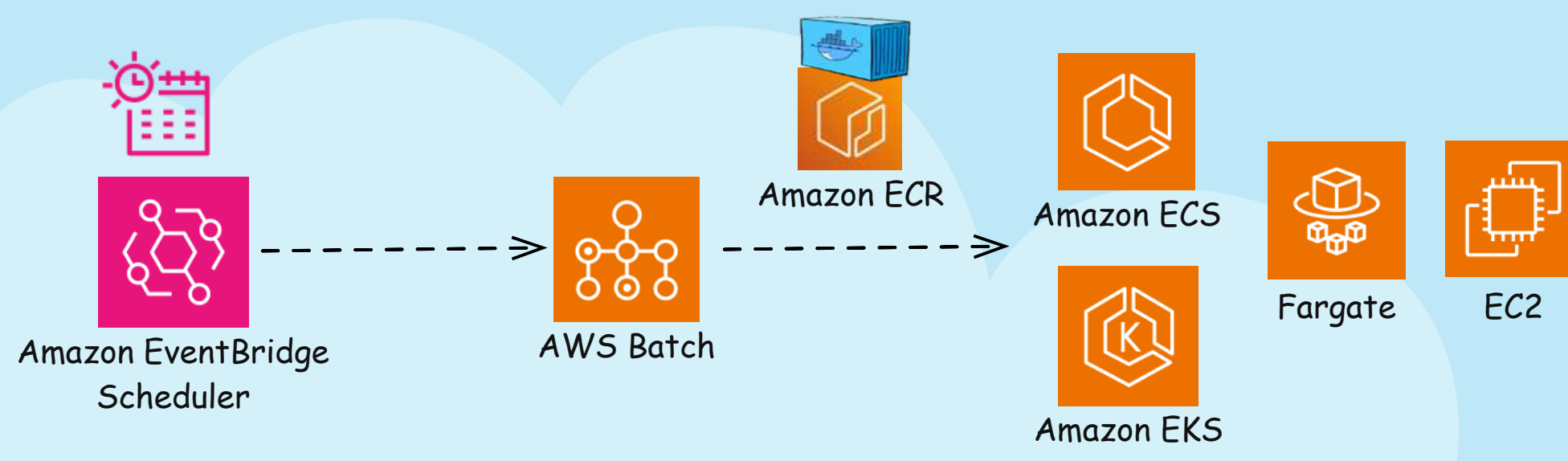
Kubernetes Node Pod Container Relationship

 **CLOUD WITH RAJ**
www.cloudwithraj.com



1. Developer creates the container image which gets deployed on Amazon EKS.
2. The container image runs inside Kubernetes Pod. Pod is the minimum deployable unit in Kubernetes. A container can NOT run without being inside a pod
3. Pod runs inside EC2 worker nodes. There can be multiple pods running inside one EC2.
4. Easy way to remember this is "NPC" (like in video games) = Node - Pod - Container in that order
i.e. Node hosts one or many pods. One pod hosts one or many containers

Batch Workloads on AWS



1. Use EventBridge scheduler to schedule jobs. You can also trigger jobs based on EventBridge rules which give you great flexibility and power!

2. Then the batch job is submitted via AWS Batch. Obviously you are thinking, I see container image and then ECS/EKS, why can't I submit a container job directly from EventBridge without AWS Batch. it's because AWS Batch maintains job queues, restart, resource management etc. that is not available if you skip it.

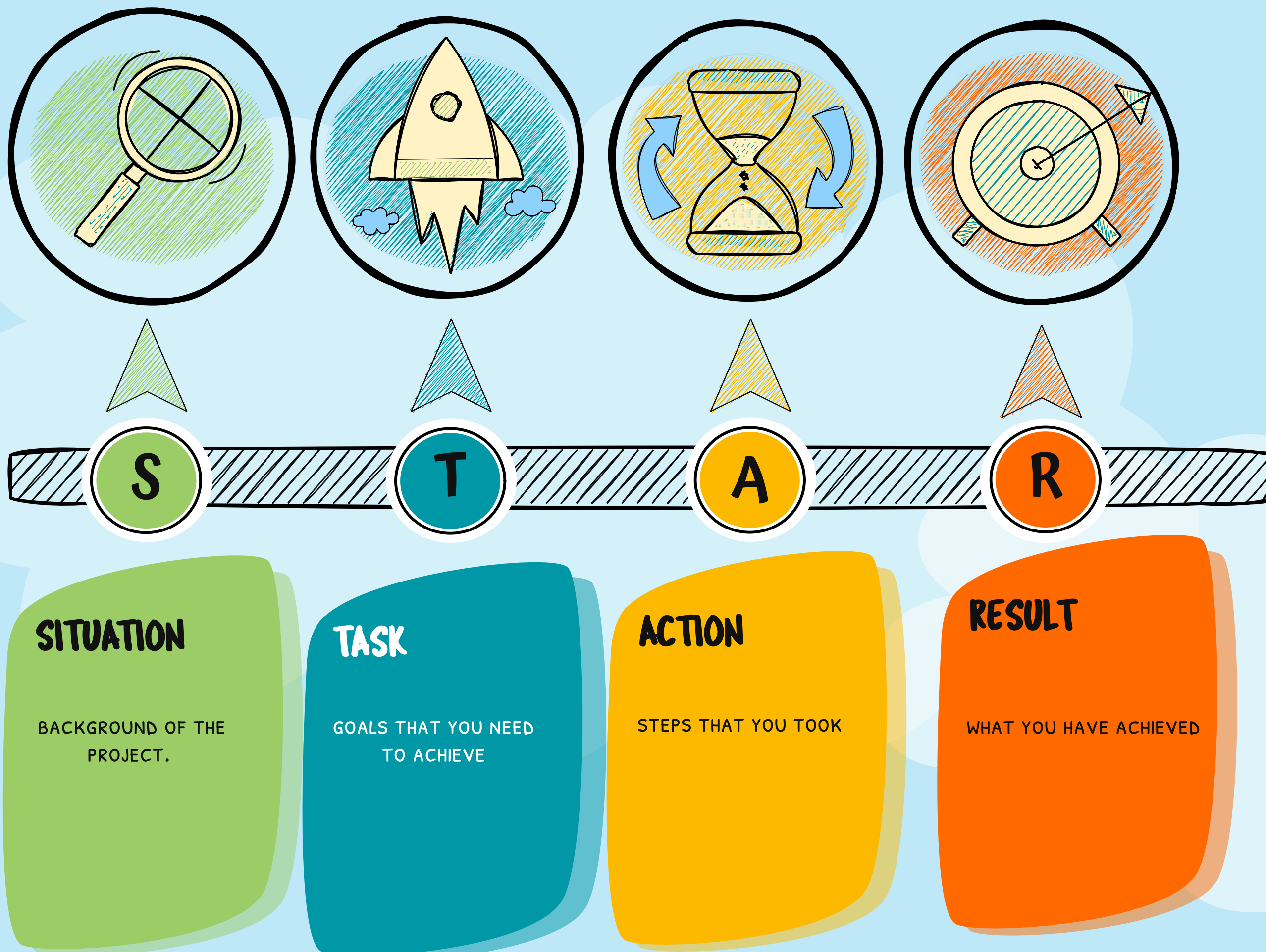
3. The actual steps of the job needs to be in a container

4. The job running in a container gets submitted in either ECS or EKS. AWS Batch supports both EC2 and Fargate

STAR Interview



CLOUD WITH RAJ
www.cloudwithraj.com



One of the biggest mistake people make in behavioral interviews is, they keep on saying "we" - "we did this task", "we came up with a plan", "we completed step X". This may sound weird - in Amazon we expect you to be a team player, but in interview we want to know precisely what YOU did. Think of this as a player selection in your superbowl team. Yes, we want the receiver to play well with the team, but while selecting, we only care about his stats and his abilities. So make sure to clarify what part YOU played in your answers.

Next biggest mistake people do is they talk in hypotheticals. When a question starts with "Tell me a time when you..", you must give example from your past projects. If you just answer in hypotheticals such as "I will do this, that..", you will fail the interview.

Okay, now let's look at a sample question and answer.

Q: Tell me about one difficult project that you delivered. What was the difficulty, and how did you determine the course of action? What was the result?

A: I migrated our project to AWS Serverless after considering K8s and EC2. We coded the lambda, tested it, implemented DevOps then deployed into prod and it was a huge success.

Is the above answer good or bad? It's quite bad, why?

- Situation and Task not described
- "We" - what actions did YOU perform?
- "Huge Success" - No Data, Very Subjective

A good answer may look like this:

Situation - We have 20 Microservices running on-prem on PCF. PCF license needed to be renewed in 6 months, leadership wanted the project to migrate to AWS before that to save cost and increase agility.

Task - As a lead architect/developer/techlead, I was tasked to find out suitable AWS solution, within the timeframe given.

Action - I researched possible ways to run Microservices on AWS. I narrowed it down to three options - run each microservice on vanilla EC2, or run on K8s using EKS, or Serverless. I took one of the microservices and did POC on Vanilla EC2, EKS and Lambda-API Gateway. While they all did the job, I found that with EC2 I have to take care of making it HA by spinning multiple EC2 in multiple AZs, and there is overhead of AMI rehydration. EKS seems to be a valid solution. However, given the traffic patterns, we have to pay more than necessary. There is also an overhead of training the team on K8s. Lambda-API Gateway is inherently HA, scalable, and pay what we use and no server to manage at all. This simplifies our day 2 operational overhead and let us focus on delivering business value.

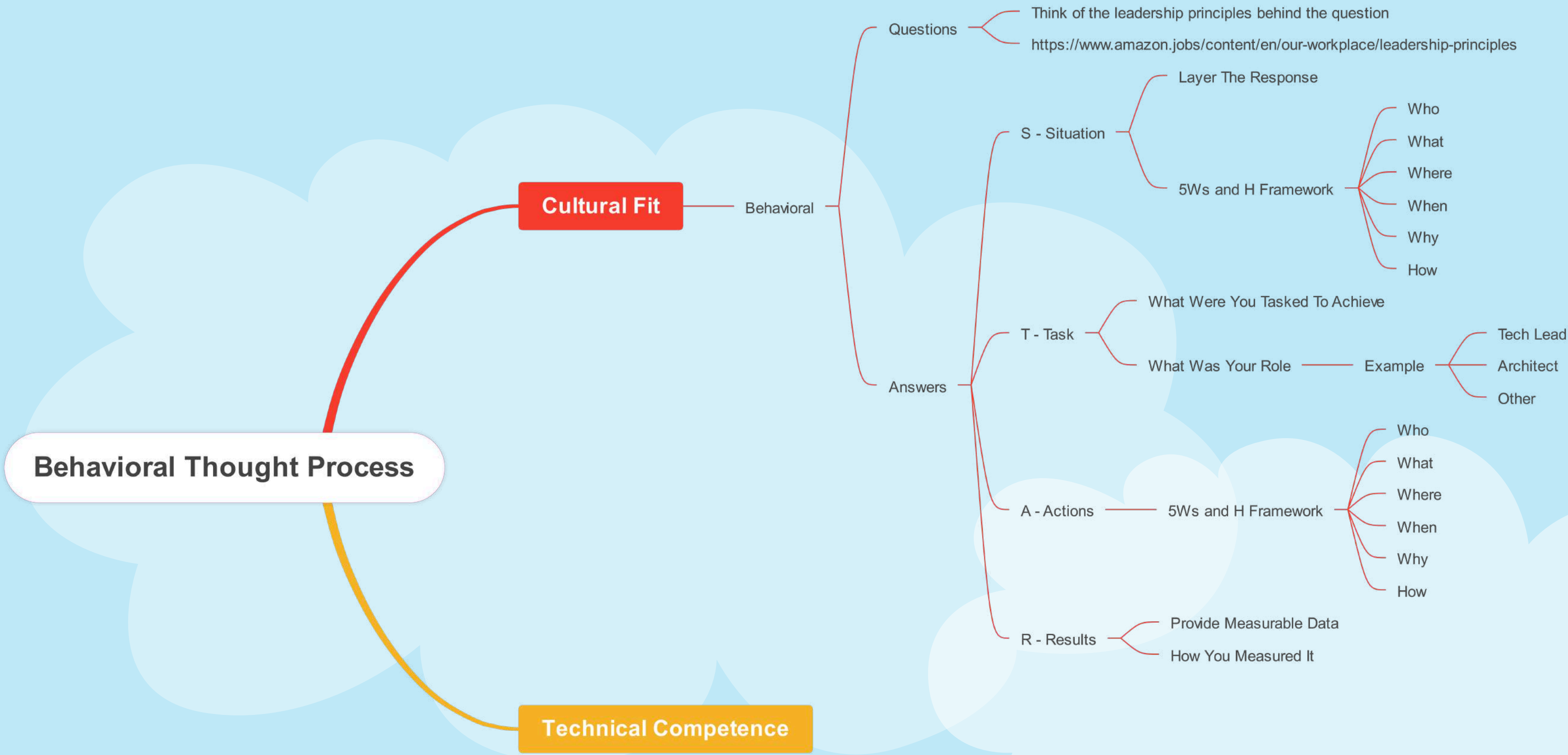
Result - Based on all the POC data of performance, cost and time to deploy, I selected Serverless solution. We converted rest of the microservices to Lambda and implemented in production within 3 months. It resulted in over 90% cost savings over EC2 and K8s. I shared my project learnings with other teams and showed them how to code Lambda so they can utilize it as well. I got recognized by CIO for this effort.

Why is this answer good?

- Situation, Task, Action, Results are clearly defined
- Gives details on what I did
- Result has data, and not just "huge success"

Note that you will get follow-up questions on the answer to understand your depth and to make sure you are not just copying and pasting a generic answer from the Internet 😊.

Behavioral Thought Process



Most candidates focus on WHAT they did. When thinking about behavioral answers, use the 5W and H framework to talk about (S)ituations and (A)ctions. Layer these on the answer. If you think like this - you will be able to answer any followups.

- 1. Who did you work with?
- 2. What did you do?
- 3. Where was this project (i.e. which company)?
- 4. When did this project happen? (Ideally in last 5 years)
- 5. IMPORTANT - Why did you take these actions? (E.g. why did you use Lambda instead of EC2? Why NoSQL?)
- 6. How did you achieve the task?

For T(Tasks), clearly mention what were you tasked to achieve, and what was your role (e.g. Developer? Tech lead? Architect? etc.)

For R(results), include data

Hybrid Cloud Architecture

 **CLOUD WITH RAJ**
www.cloudwithraj.com

Hybrid means AWS and Data Center working together for the application



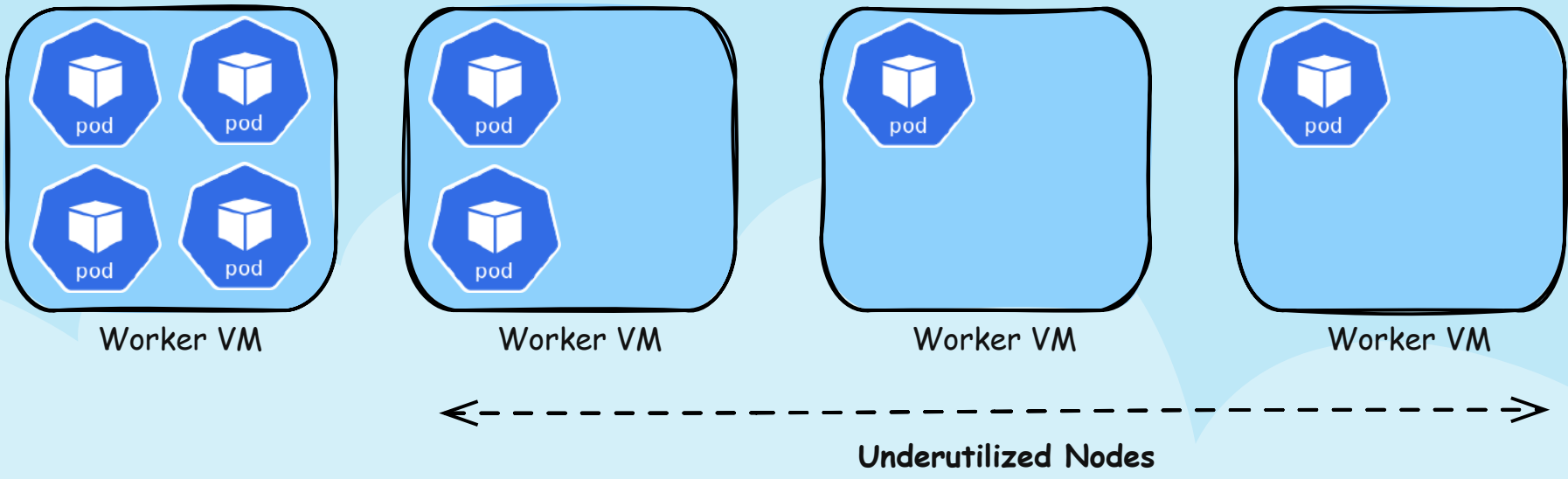


Karpenter Consolidation Saving Money



CLOUD WITH RAJ
www.cloudwithraj.com

1

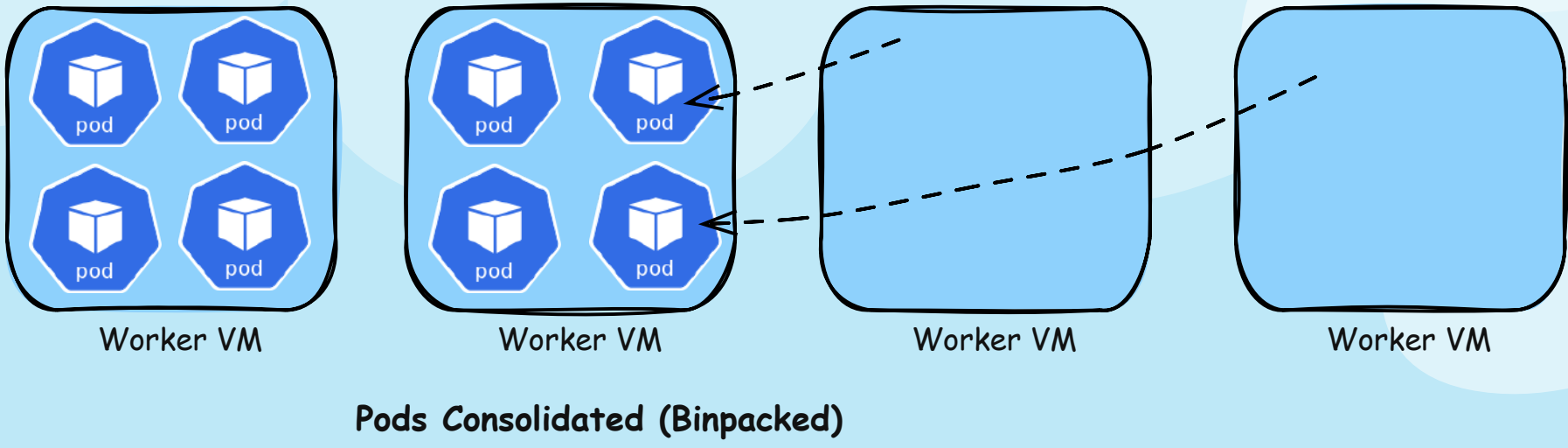


2

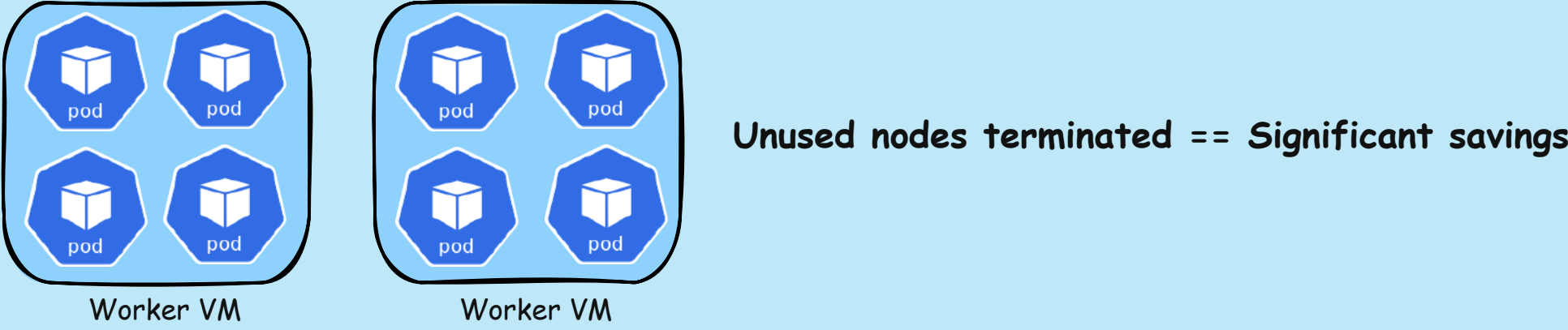
```
kind: NodePool
spec:
  disruption:
    consolidationPolicy: WhenEmptyOrUnderutilized
```

Enable Consolidation at Karpenter NodePool YAML

3



4



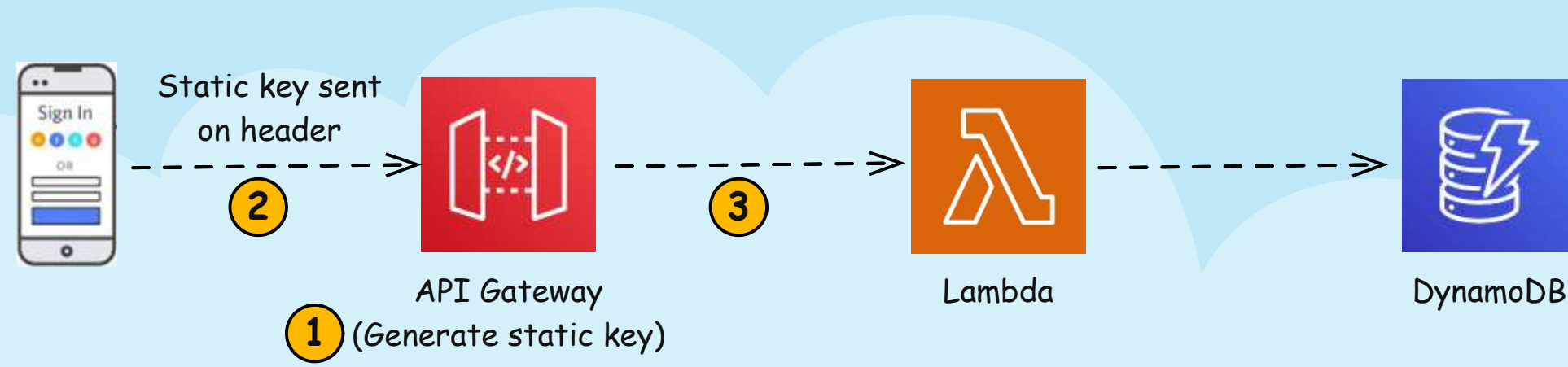
To Know More: [Get Highest Rated Karpenter Masterclass Course in Udemy \(Click the image\)](#)

API Gateway Auth

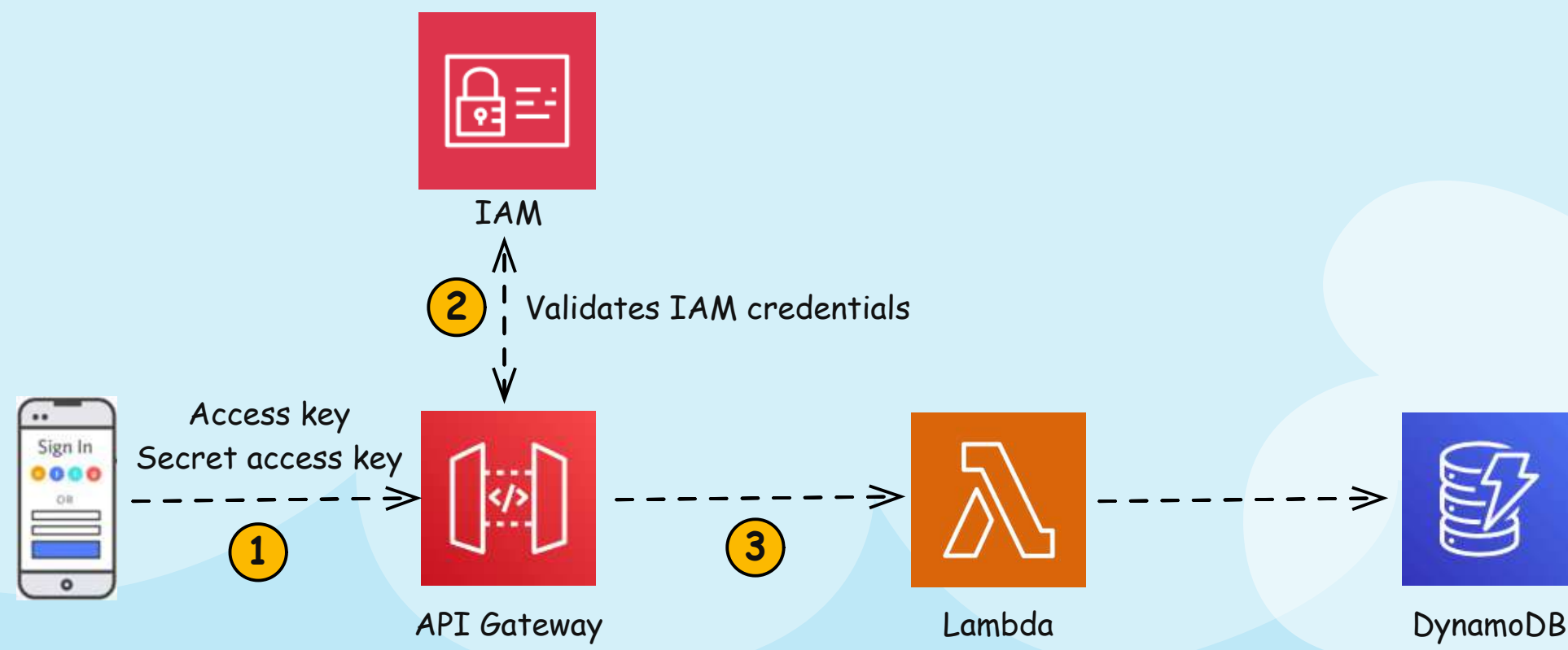


Total of 5 Auth Mechanism

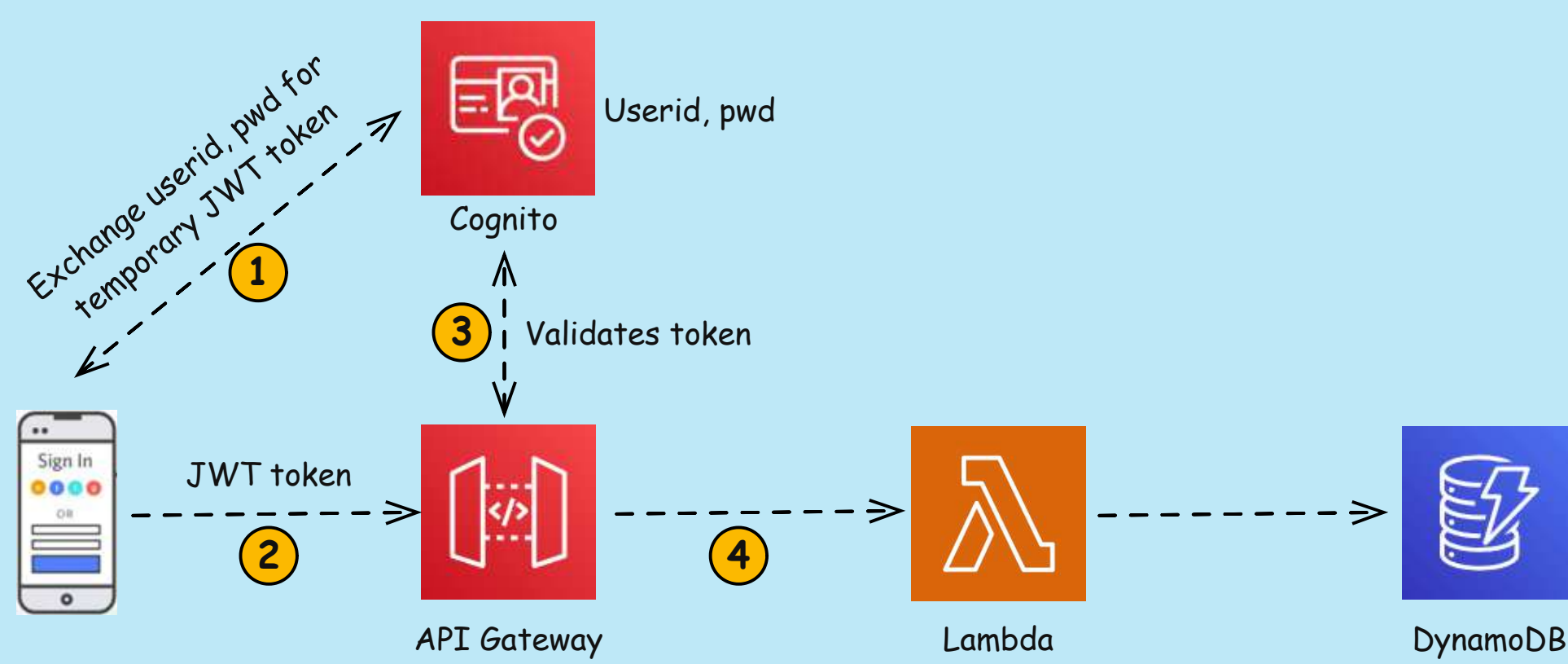
API Key



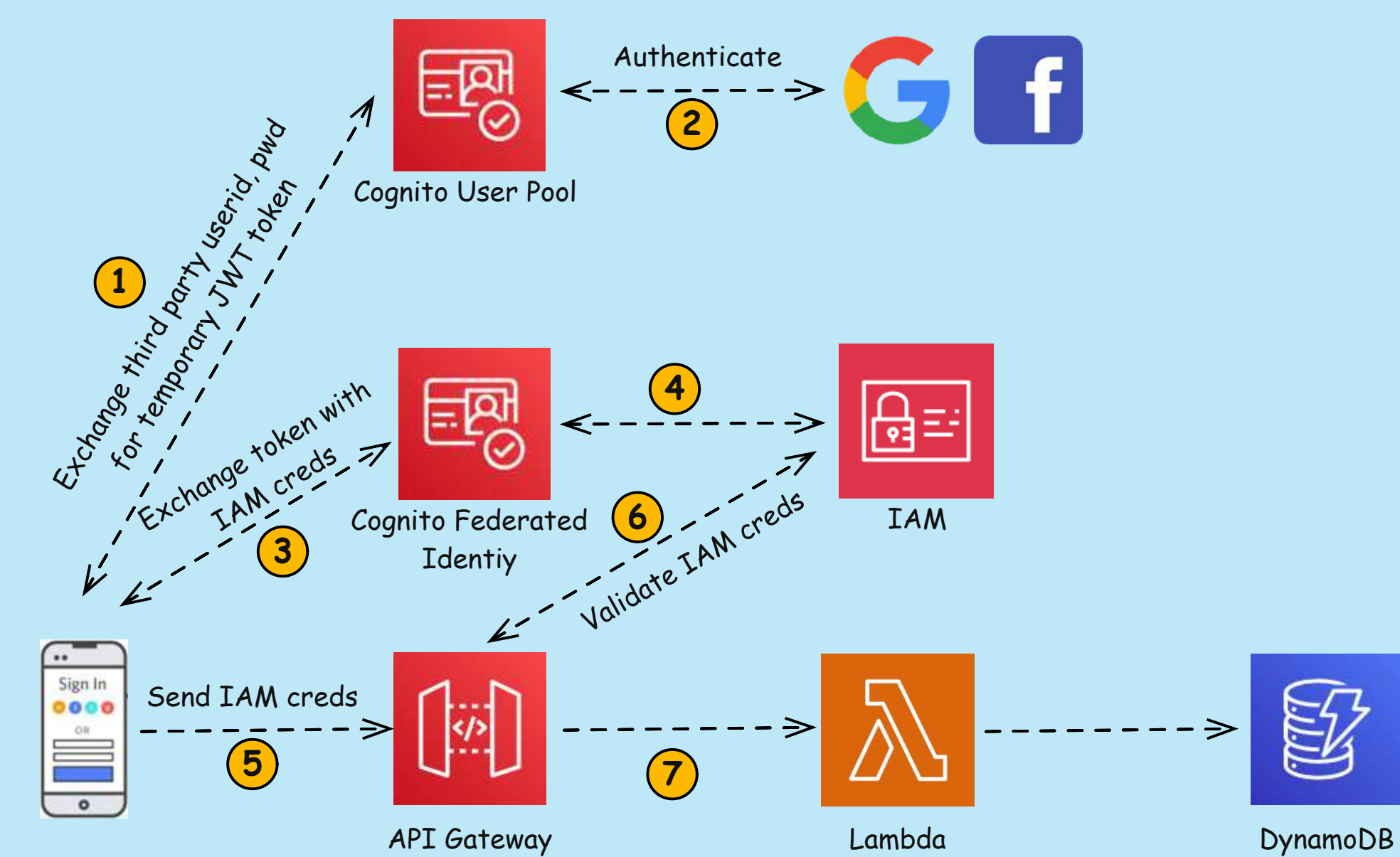
IAM



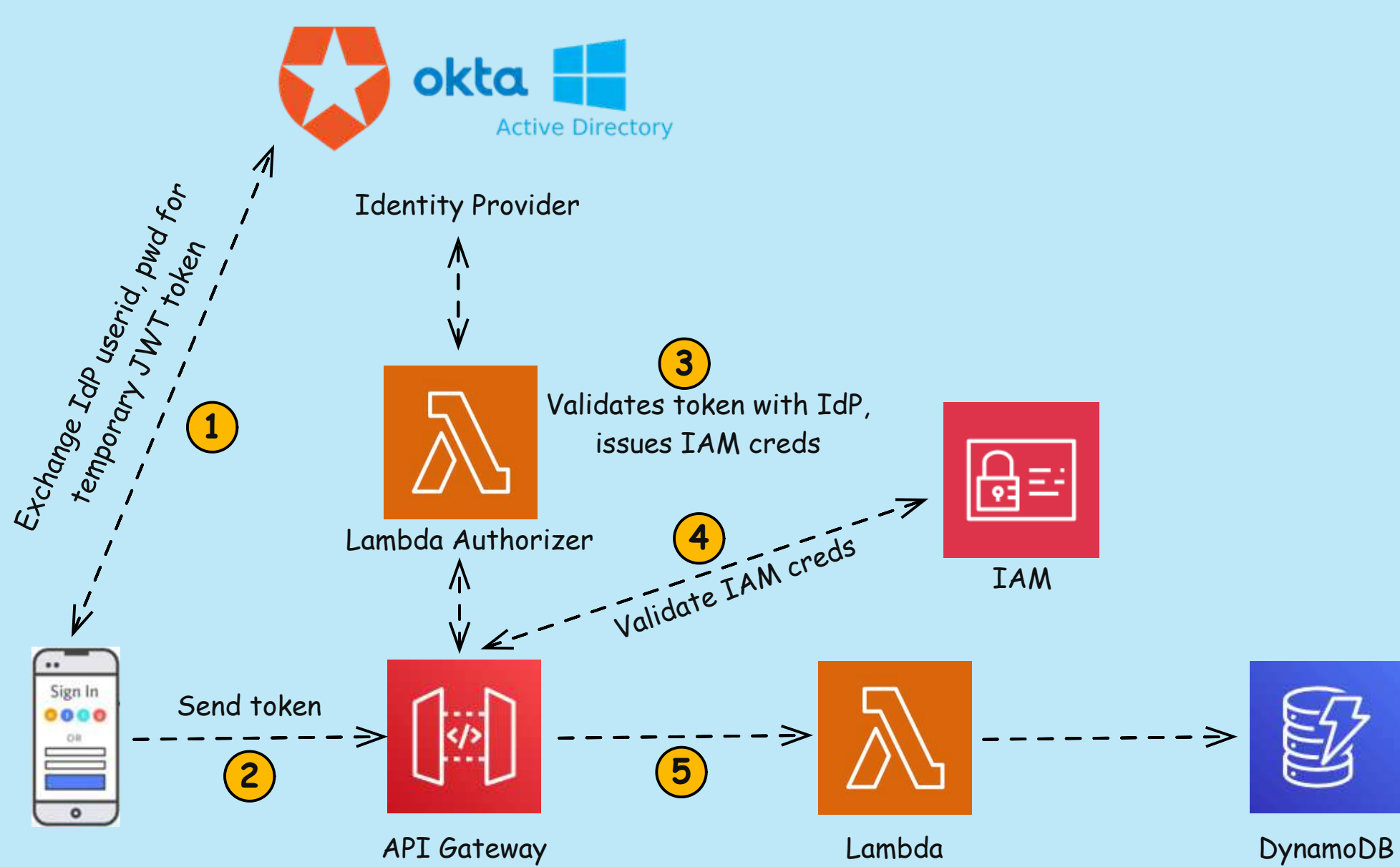
Cognito User Pool



Cognito User Pool with Federated Identities

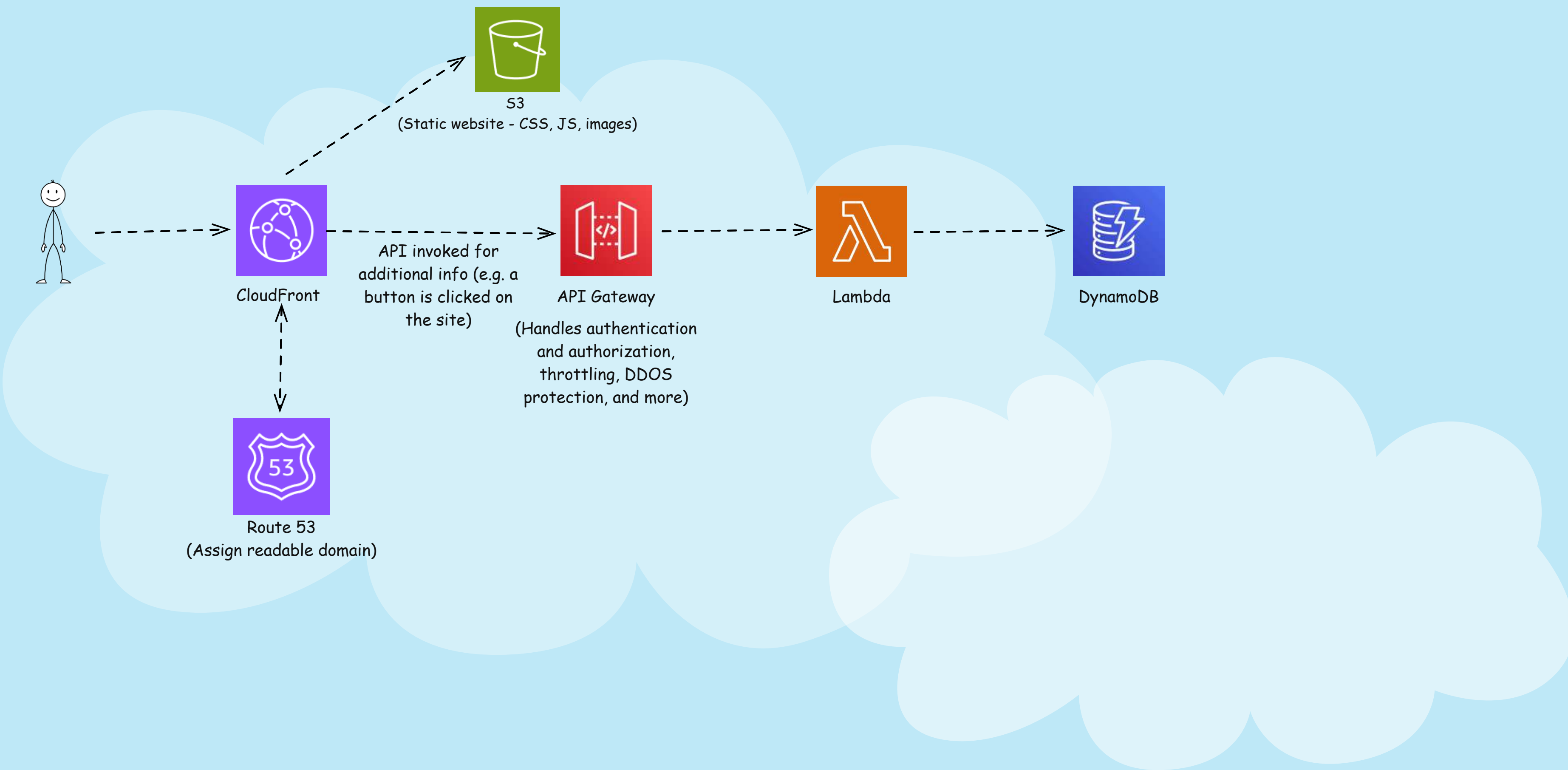


Third Party Identity Provider (IdP)

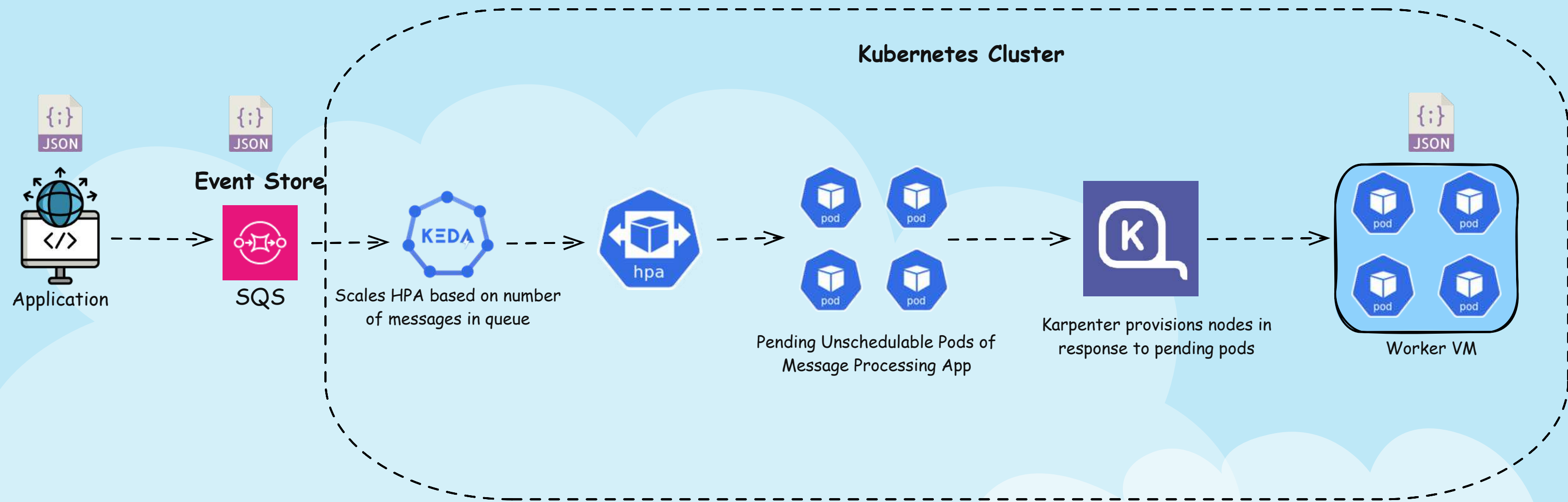


Serverless Web Application

CLOUD WITH RAJ
www.cloudwithraj.com



EDA (Event Driven Architecture) with Kubernetes

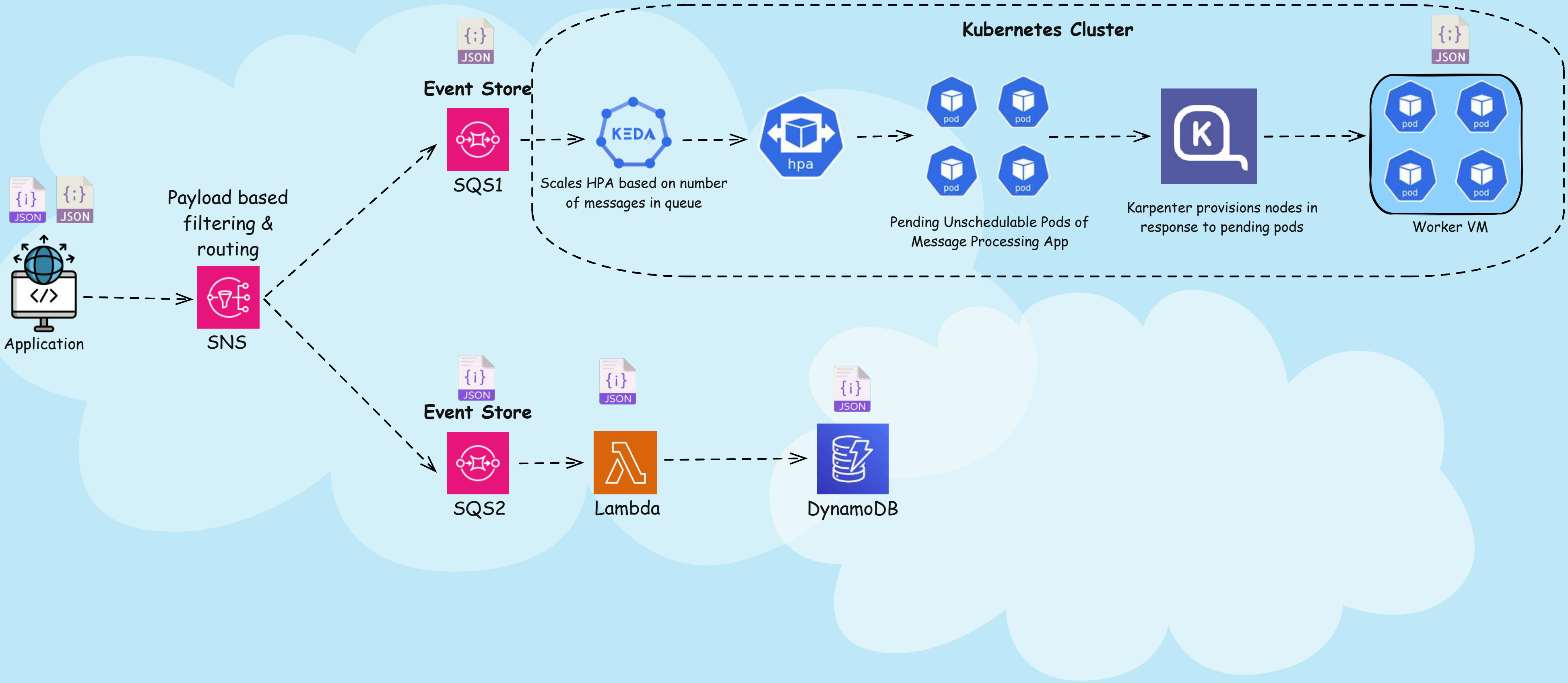


Karpenter can be used with other CNCF projects to deliver powerful solutions for common use cases. One prominent example of this is using Kubernetes Event Driven Autoscaling (KEDA) with Karpenter to implement event driven workloads.

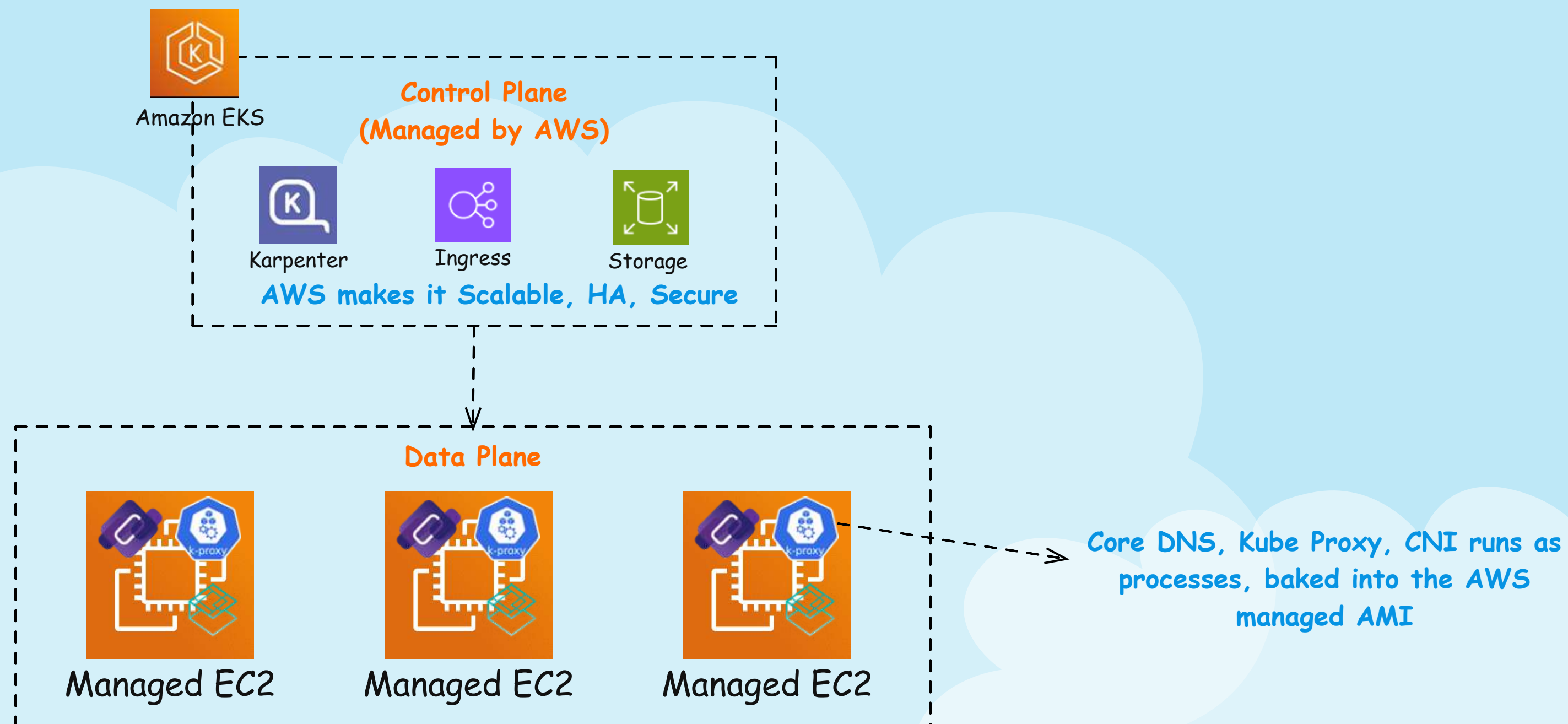
With KEDA, you can drive the scaling of any container in Kubernetes based on the number of events needing to be processed. One popular implementation is to scale up worker nodes to accommodate pods that process messages coming into a queue:

1. Application inserts messages into the queue
2. KEDA monitors queue depth, and scales HPA for the application for processing the messages
3. HPA increases number of pods. Assuming, no capacity available to schedule those pods, Karpenter provisions new nodes
4. Kube-scheduler places those pods on the VMs. The pods process the messages from queue
5. Once processing is done, number of pods go to zero. Karpenter can scale down VMs to zero for maximum cost efficiency

EDA (Event Driven Architecture) with SNS SQS Kubernetes



EKS Autonomous Mode



- AWS Manages core addons like Karpenter, Ingress, and EBS
 - AWS makes them secure, scalable, HA just like EKS control plane components
 - AWS manages the version
- AWS Manage AMIs
 - AMI is based on Bottlerocket with Core DNS, Kube-Proxy, VPC CNI components baked in

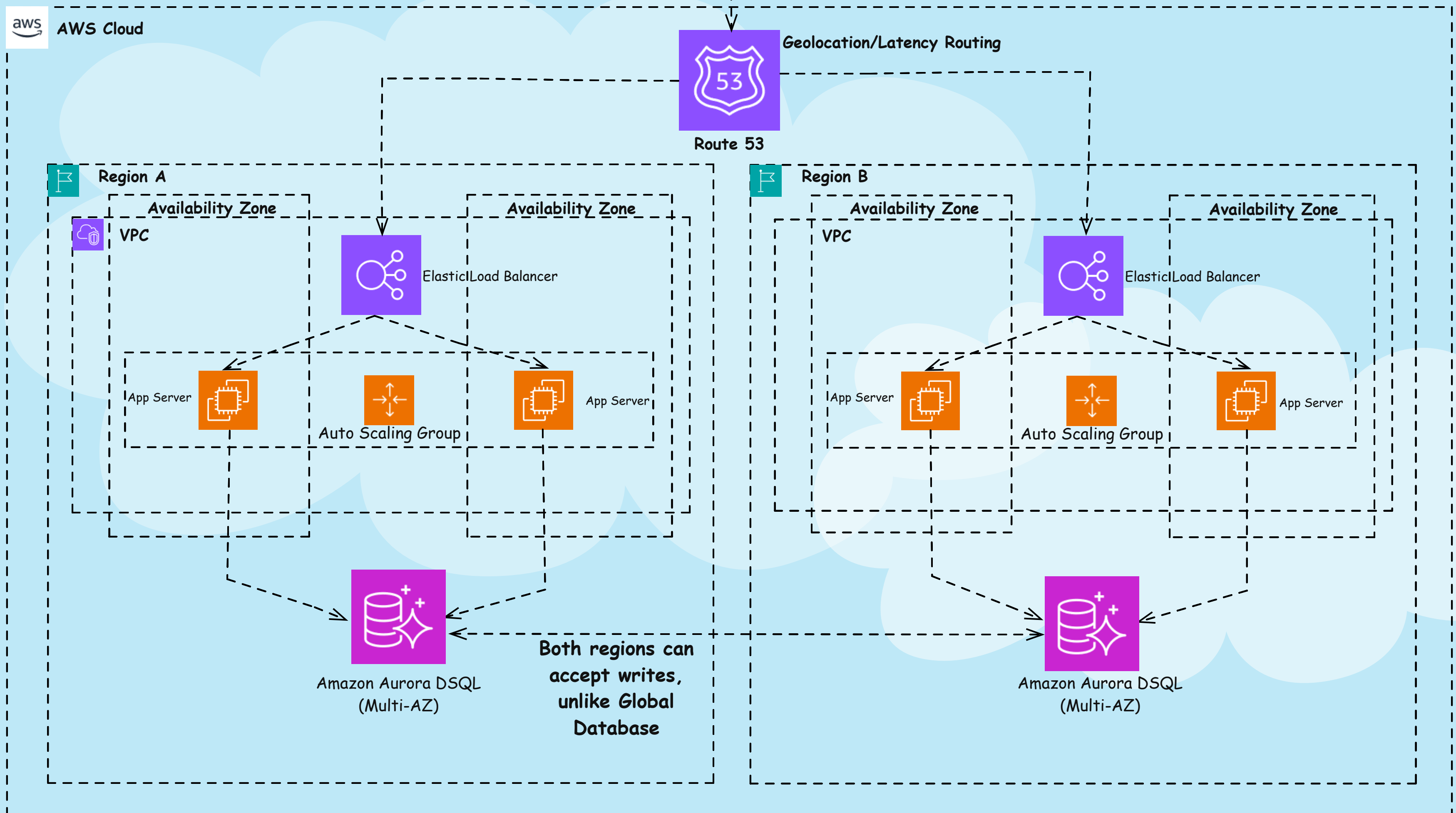
Managed worker nodes are EC2s, that means you can run pretty much anything that you can run on regular Kubernetes

This is a big one - EKS Auto supports daemonsets, so you can run your favorite tools and agents

Use Reserved Instances, Savings Plan, Spot, Graviton with EKS Auto

Amazon Aurora Global Database Vs. DSQL

CLOUD WITH RAJ
www.cloudwithraj.com



Amazon Aurora released DSQL, which may seem similar to Aurora Global Database. What are the similarities and differences?

1. Amazon Aurora, regardless of using Global Database, or DSQL, stores data in multi-AZ within a single region. In the unlikely event of a region failure, your data is highly available

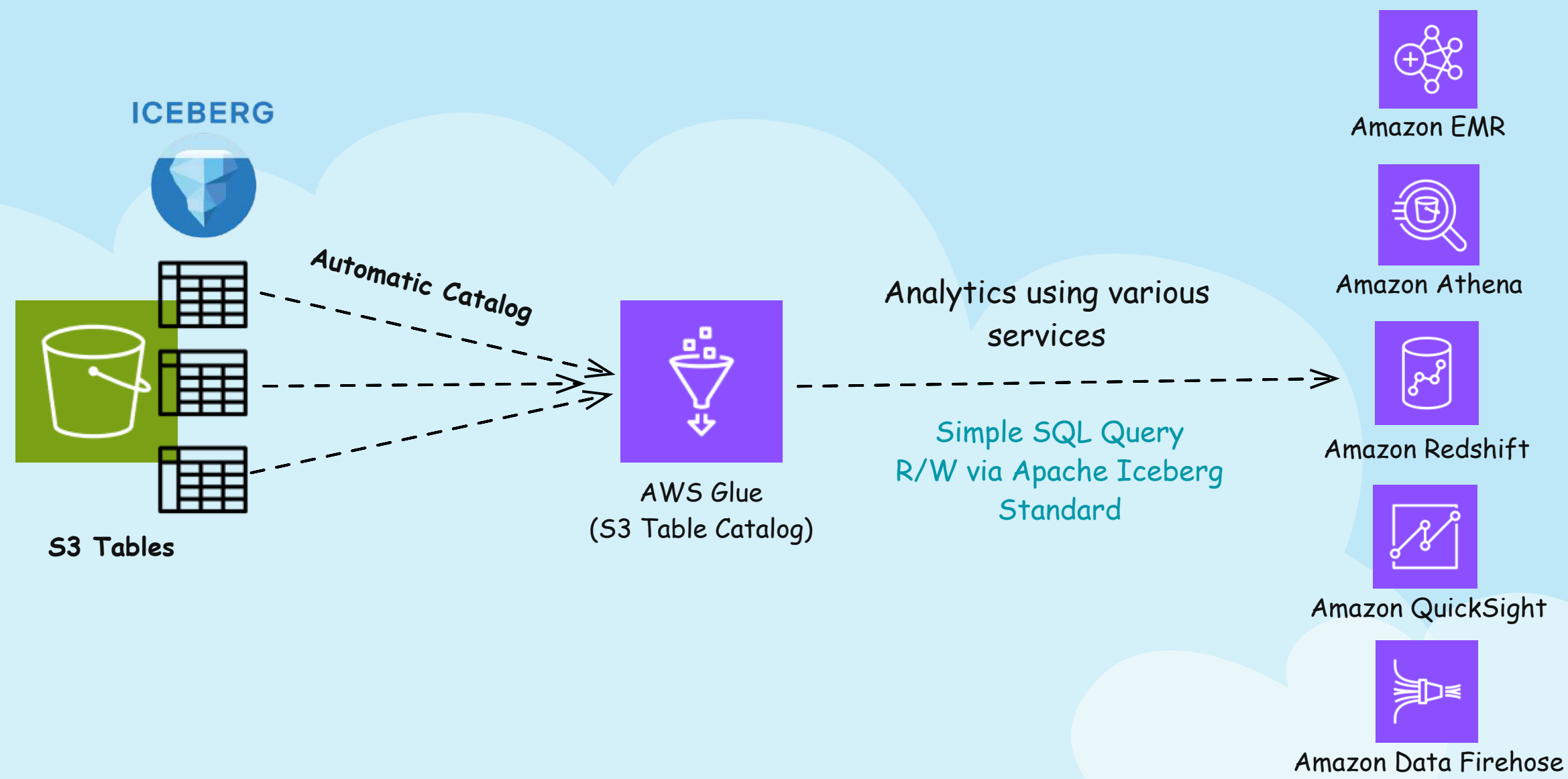
2. The main difference is - Global Database replicates data from one region to another, and only one region can write to the table. The secondary region acts as a reader. Only when the primary region fails, the secondary region table get promoted to the primary (writer) instance.

Whereas in DSQL it is Active-Active i.e. both regions can accept writes and cross-replicate. DSQL also stores the transaction logs that can be used to recover lost transactions during a disaster.

3. Aurora Global Database is supported for MySQL and PostgreSQL, and DSQL public preview is available for PostgreSQL

4. Till now, Aurora can autoscale Aurora Replicas for reading only. But DSQL can horizontally scale both reads and writes (Compute and Storage) which will be a big leap and a game changer for SQL databases.

Amazon S3 Tables with Apache Iceberg



Iceberg is a high-performance format for huge analytic tables. Iceberg brings the reliability and simplicity of SQL tables to big data, while making it possible for engines like Spark, Trino, Flink, Presto, Hive and Impala to safely work with the same tables, at the same time.

- Prior to this announcement, you could save data as Iceberg tables in General S3 bucket which has overhead (use Data Catalog in Lake Formation)

- With this announcement - Define tables easily and much more performant
 - 10X TPS, 3X faster query performance compare to Iceberg data stored in general buckets

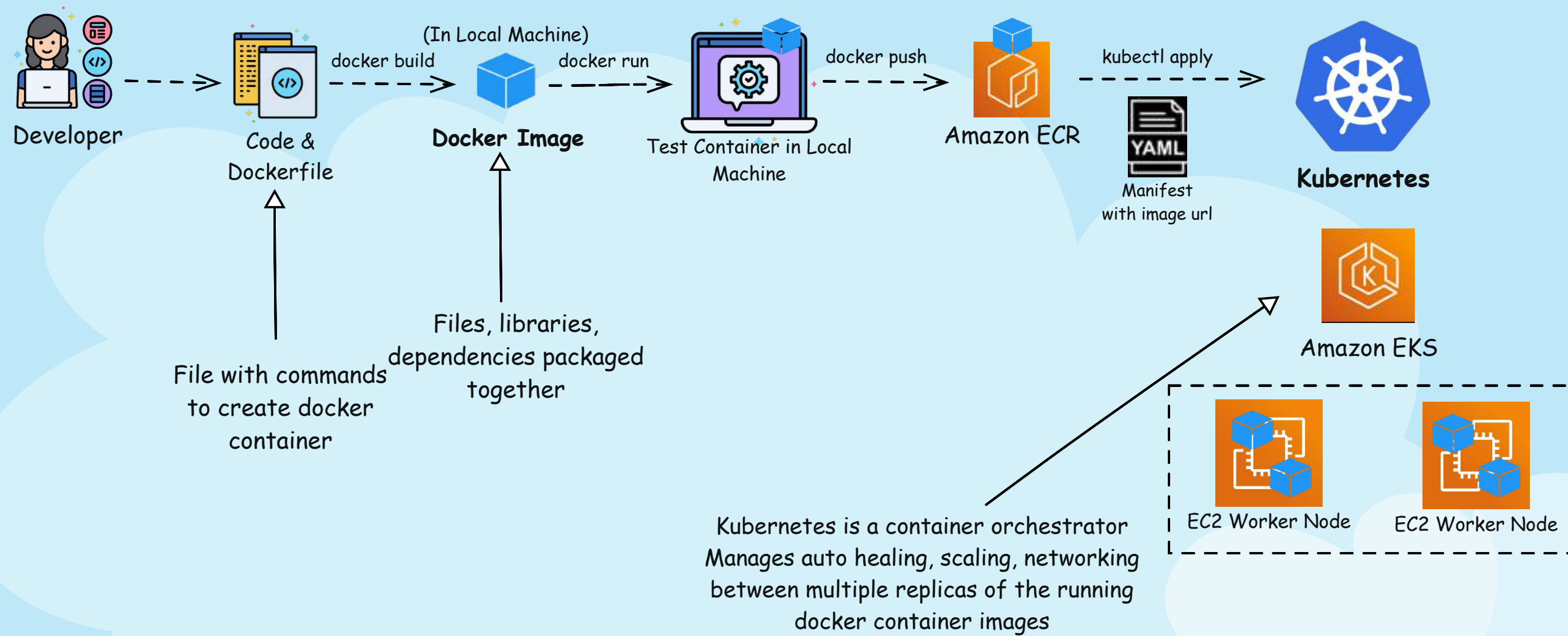
- Automatically registered to Glue data catalog
 - Catalog can be used by Amazon EMR (Spark), Athena, Redshift, Quicksight, Data Firehose

- Fully managed Apache Iceberg Tables in S3 - each table gets an ARN (Amazon Resource Name)
 - Optimized performance, security controls, cost optimization
 - Automatic compaction

- Run simple SQL query

- Applications read and write data via Apache Iceberg Standard

Docker Vs Kubernetes



Docker Vs Kubernetes

1. The term docker is used in lot of places in container lifecycle. Hence let's understand with the flow above
2. The two primary areas the term "Docker" is associated are - Dockerfile, and mostly Docker Image (also known as Docker container image, or just container image)
3. Dockerfile is a file with commands that packages up the code, libraries, and dependencies in to the Docker container image
4. This docker image is just a single copy of your application, that is often tested in your local machine (note the single docker container image on the laptop!). This image is stored in a registry like Amazon ECR
5. Finally, using "KubectI apply" command, an YAML manifest which has the URL of the container image from the Amazon ECR, is deployed into the running Kubernetes cluster.
6. Running a single docker container is easy, but when you need to run many copies (known as replicas in container lingo) of your container image, there needs to be a control plane managing different things like - when one running container dies, provision another in it's place, scale the replicas, networking between them and more. This is exactly what Kubernetes does.

System Design Trade-Offs

Increased Reliability **VS** Price

Management Overhead **VS** Control

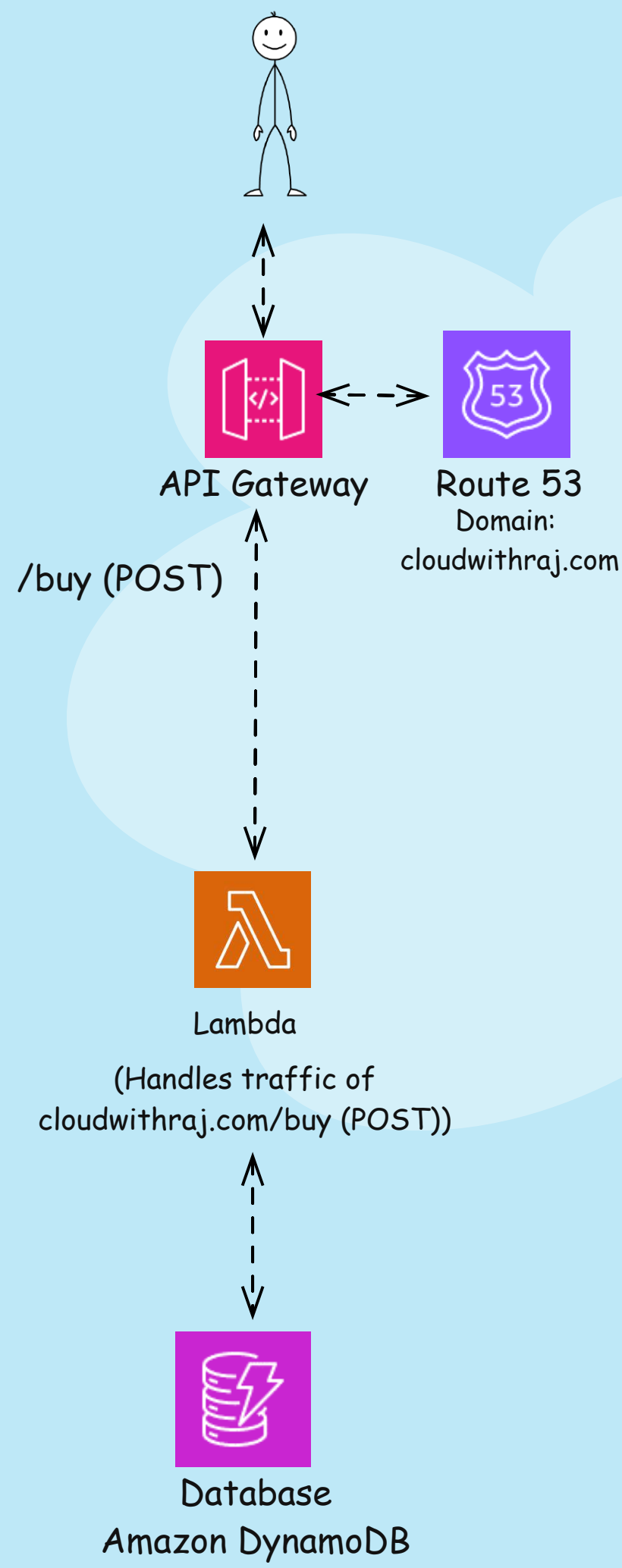
Performance **VS** Consistency



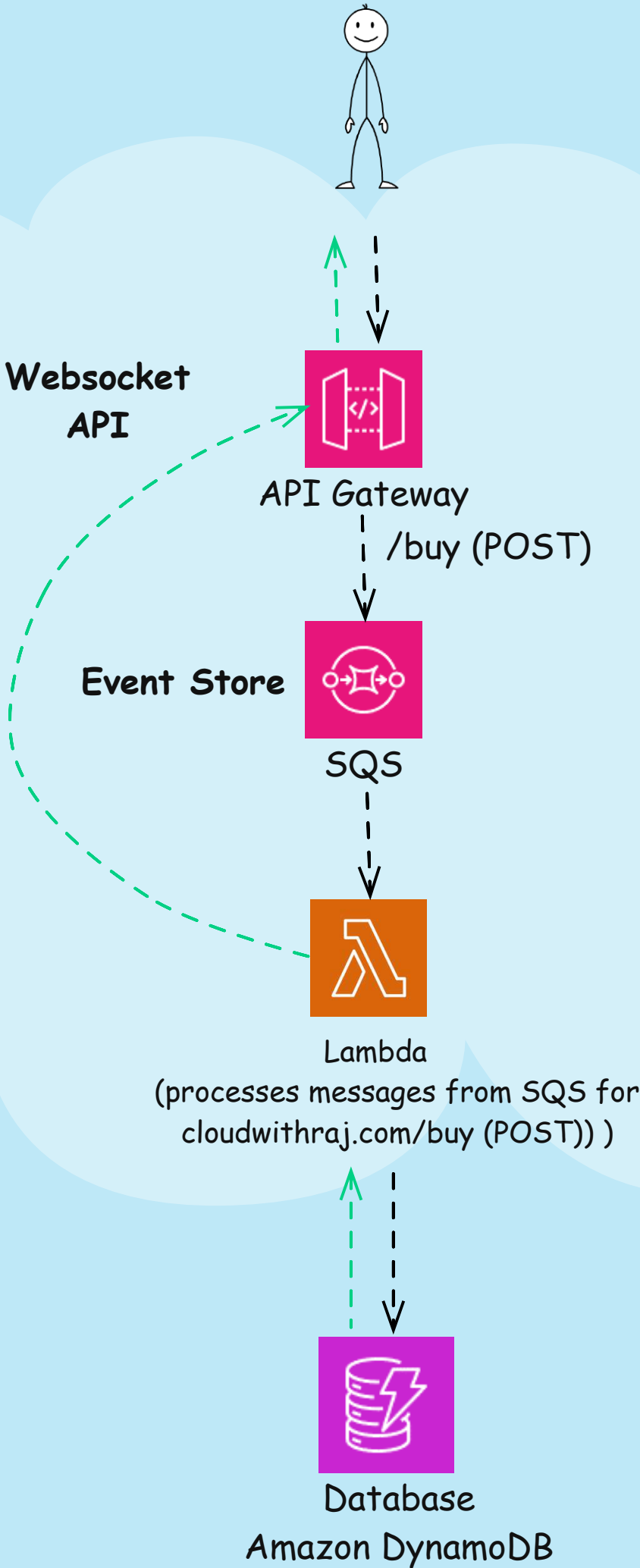
Top 3 Popular Designs



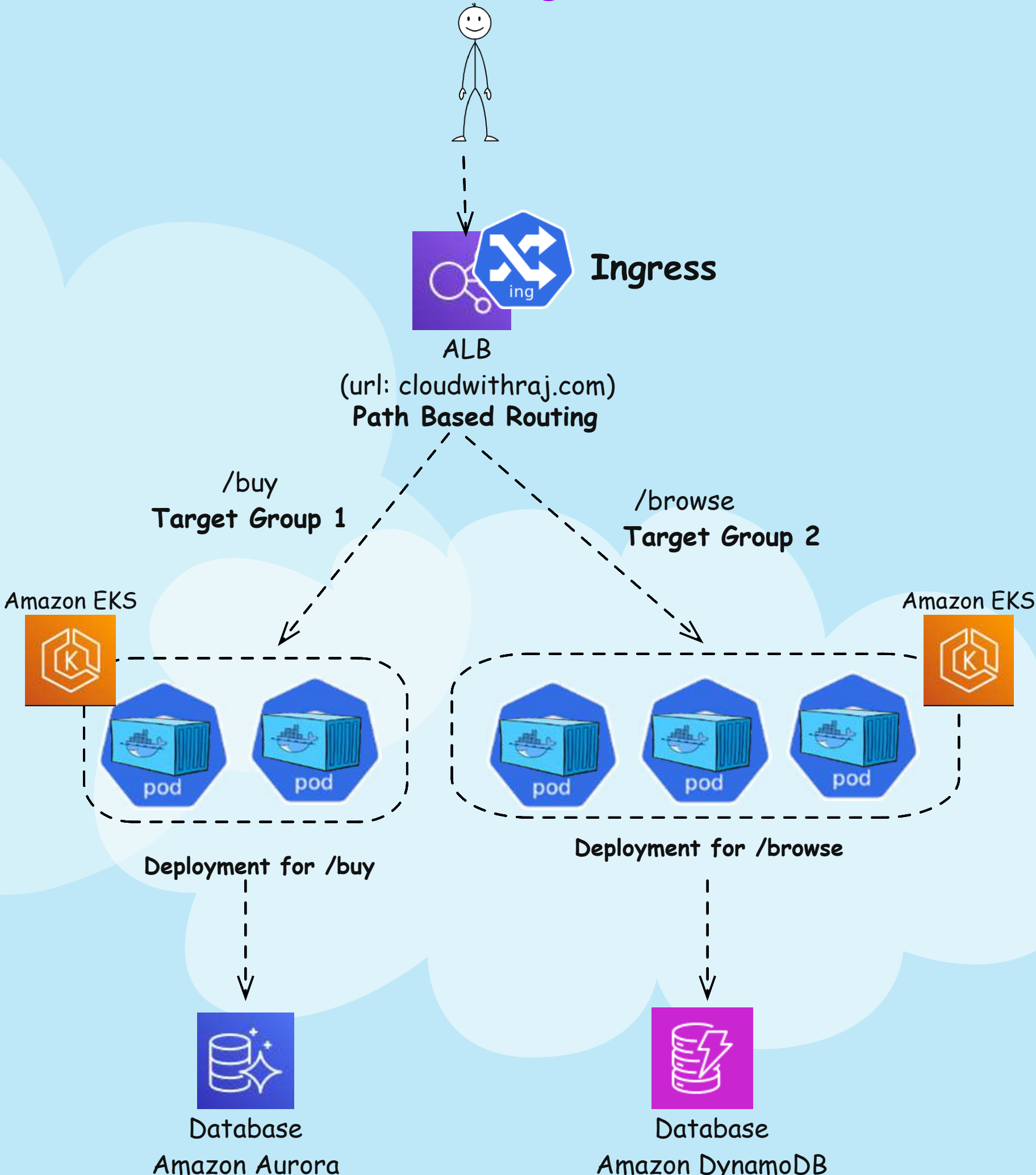
Synchronous Microservice



Event Driven Architecture



Kubernetes Ingress



3 Tier Architecture with Microservice

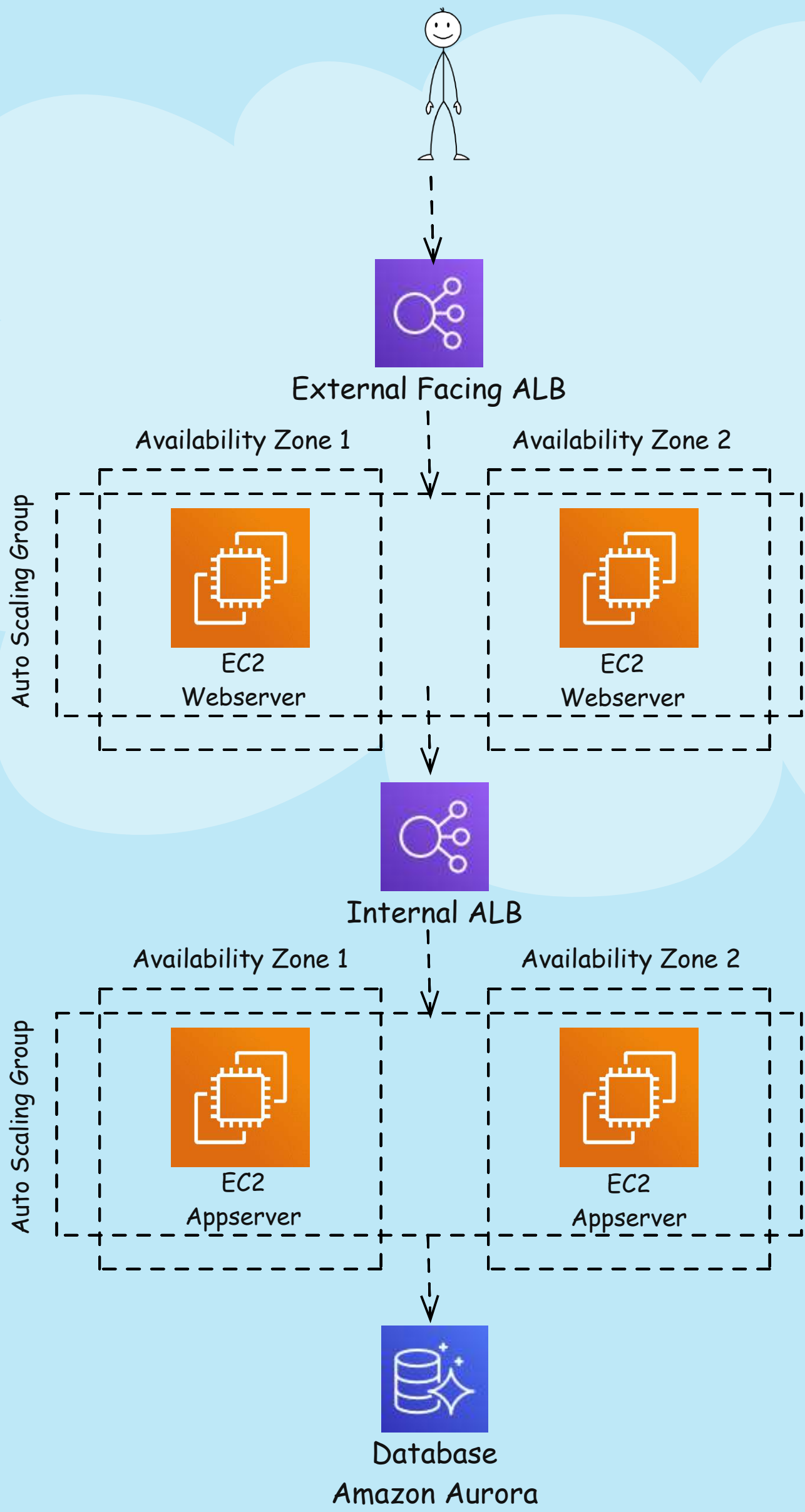


Three-Tier Design

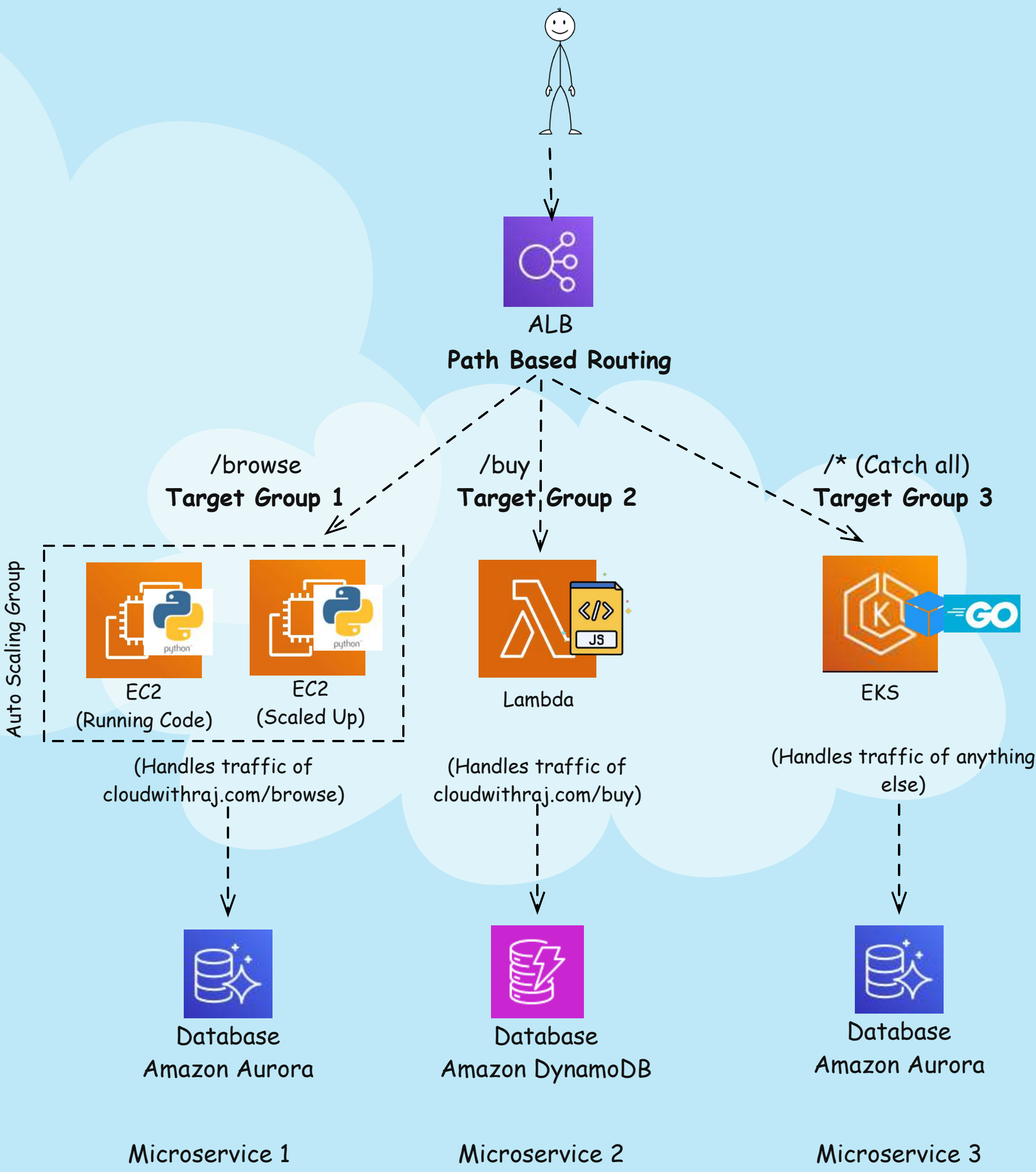
PRESENTATION LAYER

APPLICATION LAYER

DATABASE

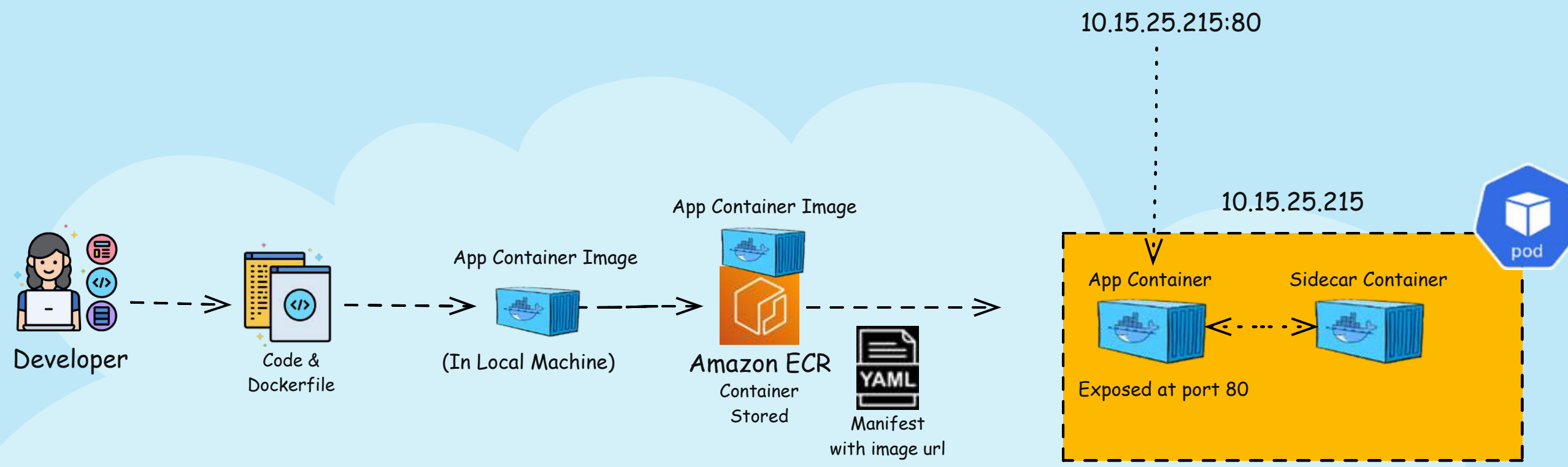


Detailed Microservice Design



Pod Container Sidecar

 **CLOUD WITH RAJ**
www.cloudwithraj.com



1. In Kubernetes container(s) run inside pod

2. Generally, inside one pod, one application container runs

3. But sometimes, another container runs inside the same pod along with application container called sidecar containers. Sidecar containers have their own independent lifecycles. They can be started, stopped, and restarted independently of app containers. This means you can update, scale, or maintain sidecar containers without affecting the primary application.

4. Each pod has an IP address, all containers inside the pod share the same IP address. For that reason, application container is exposed using a port. In this example the pod IP is 10.15.25.215, and the app container is exposed at port 80.

5. One popular example of sidecar container is Istio service mesh which runs inside each pod alongside app container

Control Karpenter Consolidation



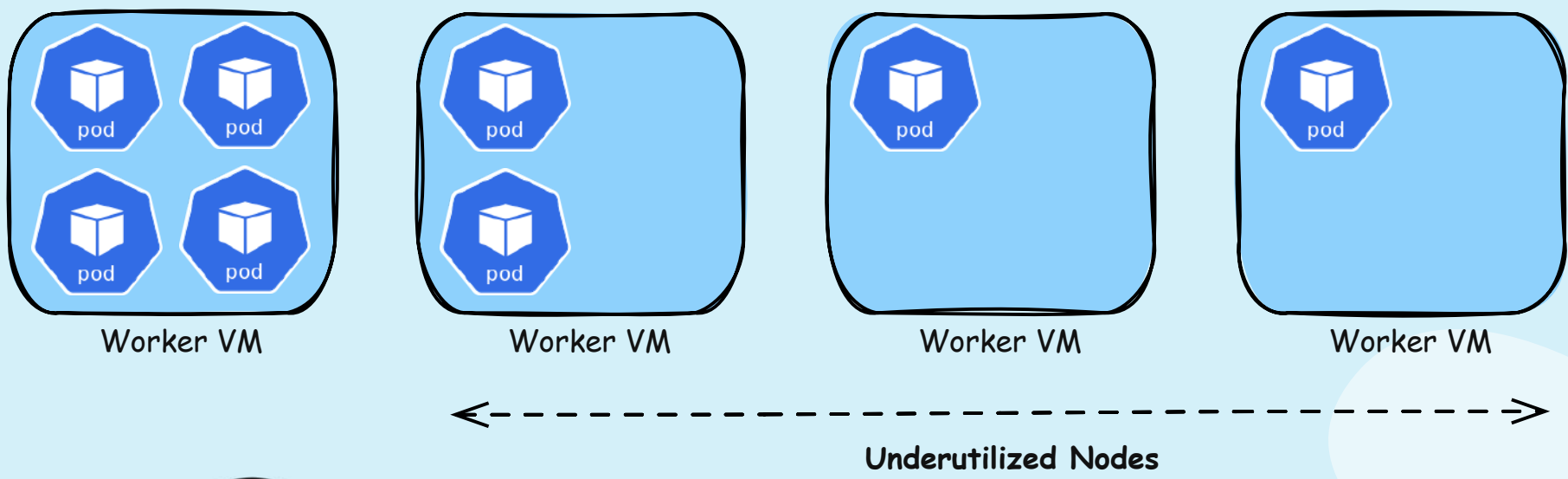
1

```
kind: NodePool
spec:
  disruption:
    consolidationPolicy: WhenEmptyOrUnderutilized
    consolidateAfter: 30m
```

Put value 'Never' to stop consolidation

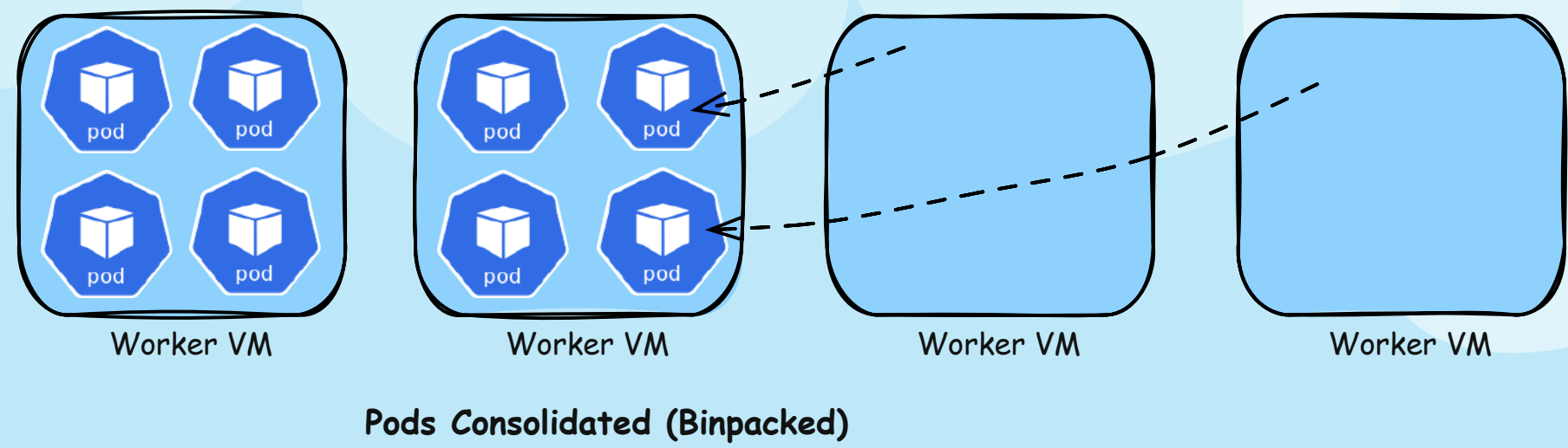
Enable Consolidation at Karpenter NodePool YAML with consolidateAfter

2



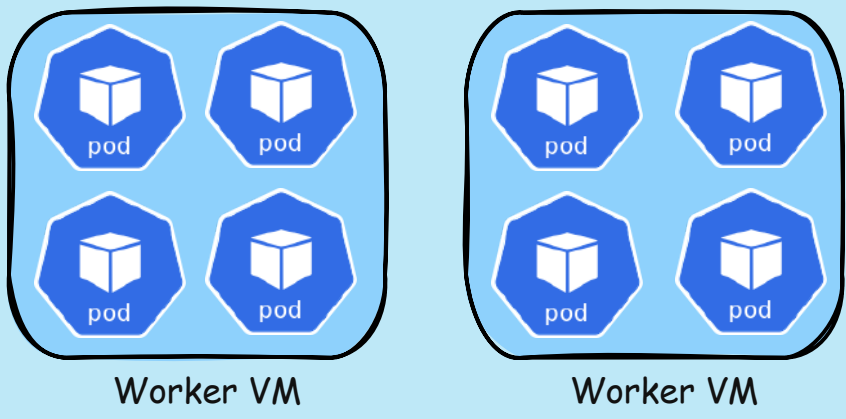
WAIT 30 mins from last pod added or deleted to consolidate

3



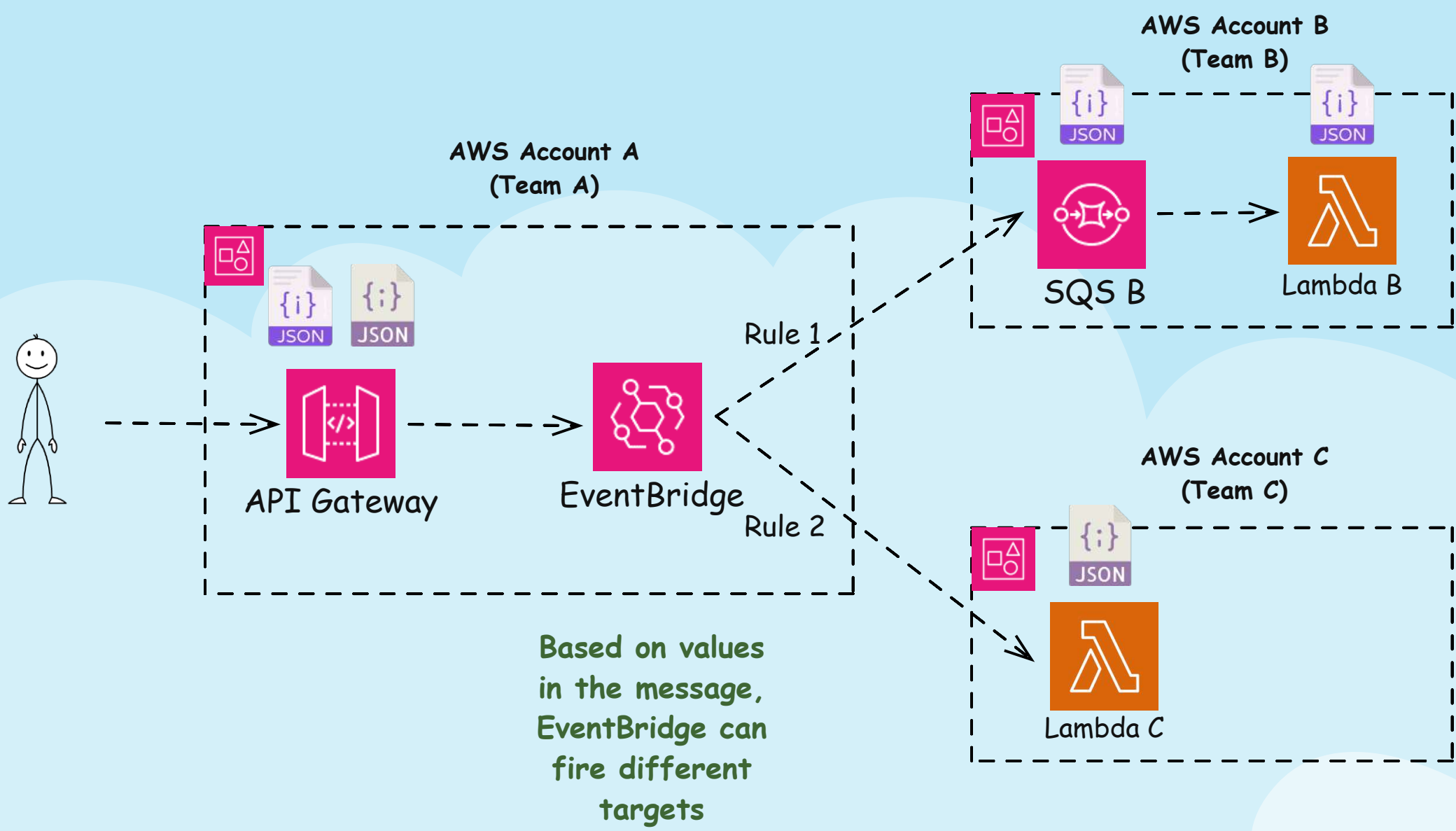
Pods Consolidated (Binpacked)

4



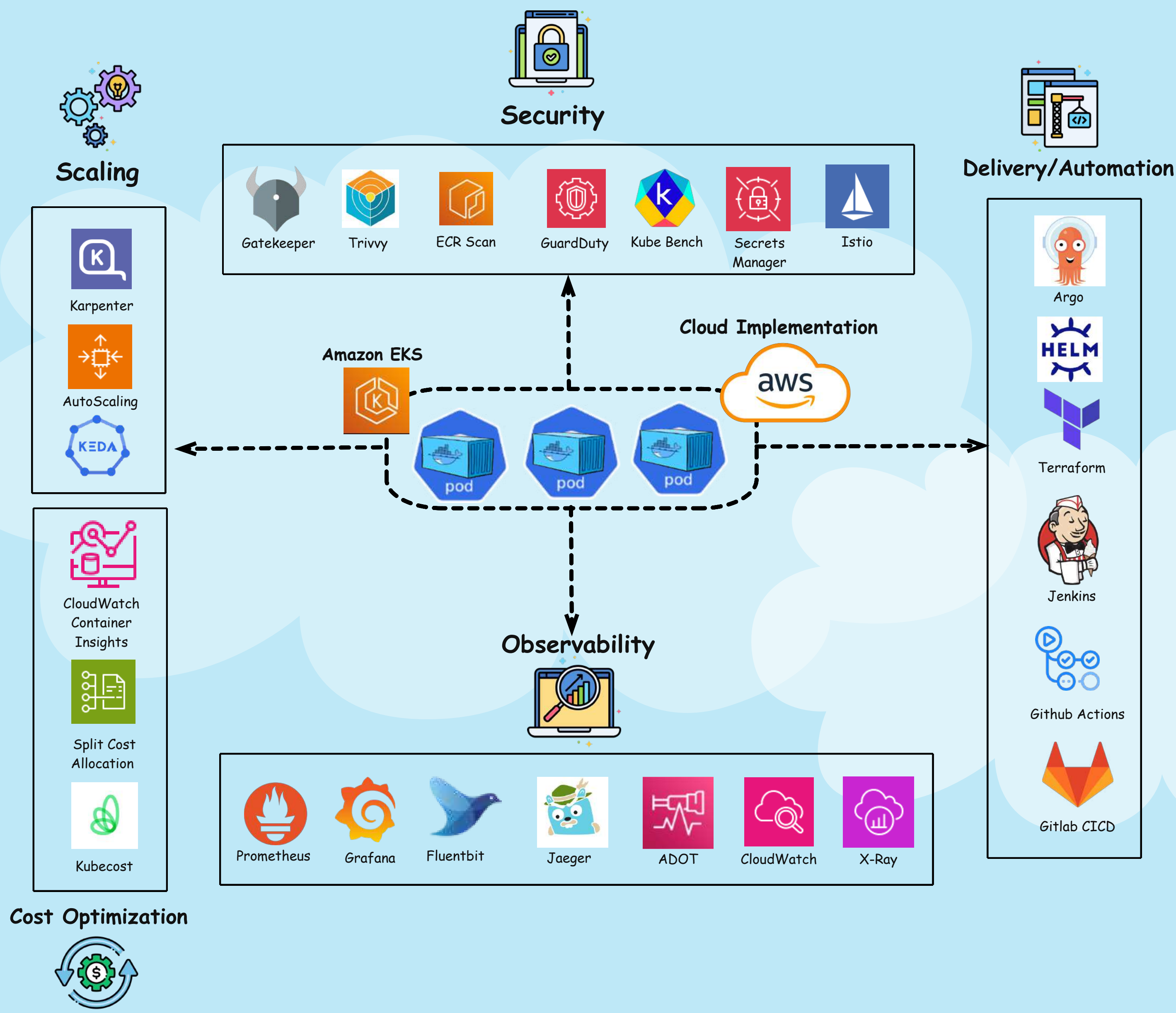
Unused nodes terminated == Significant savings

Cross-Account EventBridge Target



Kubernetes Tech Stack

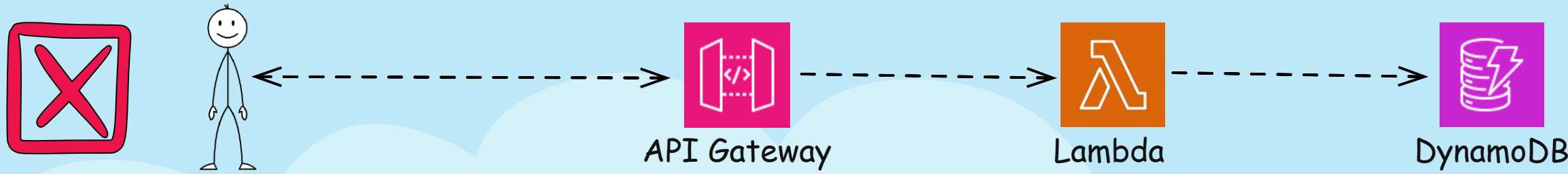
 **CLOUD WITH RAJ**
www.cloudwithraj.com



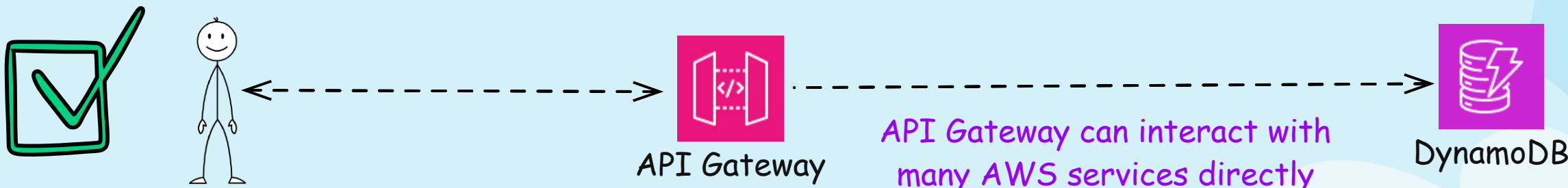
API Gateway, Lambda, AWS Service



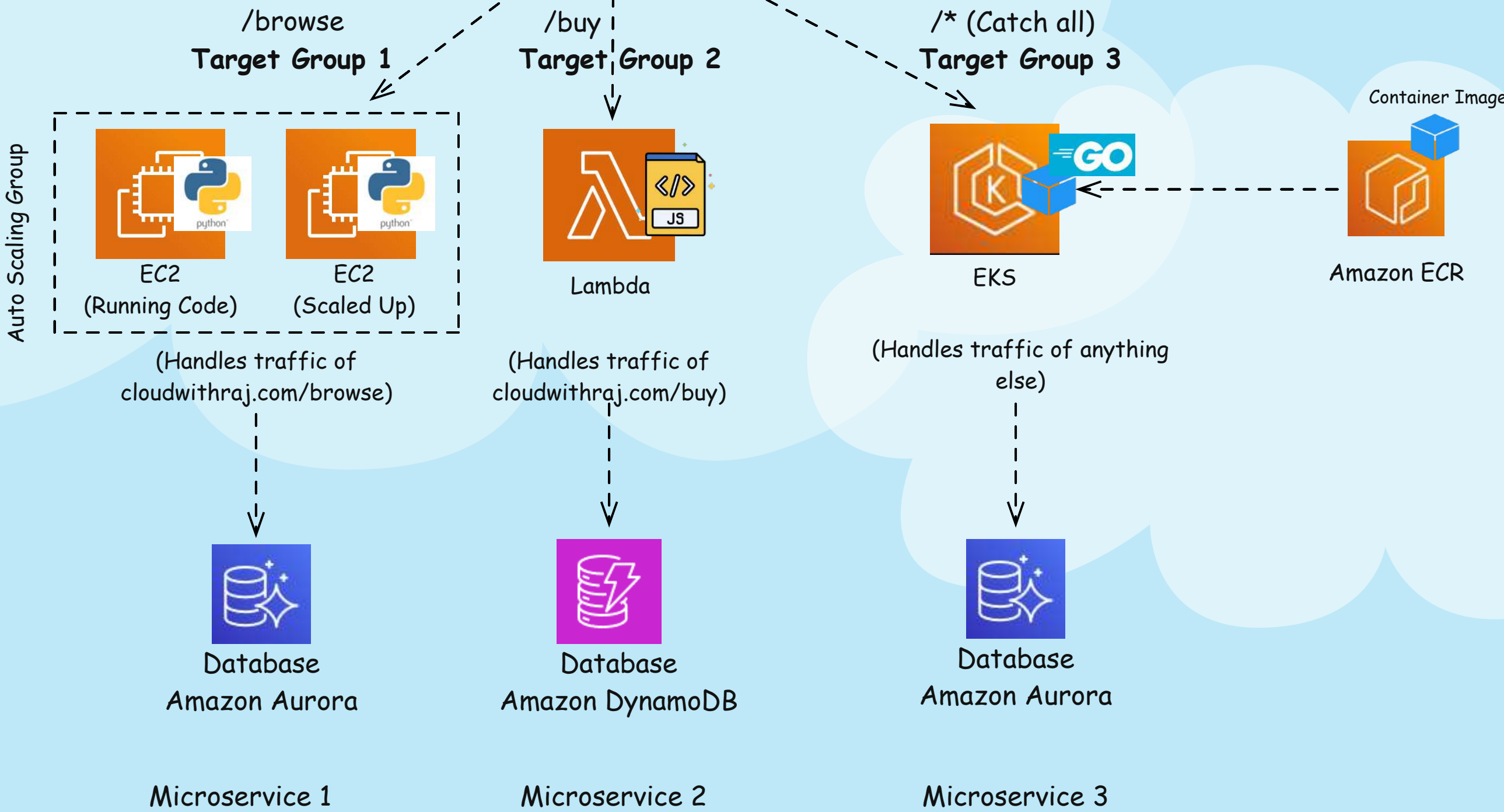
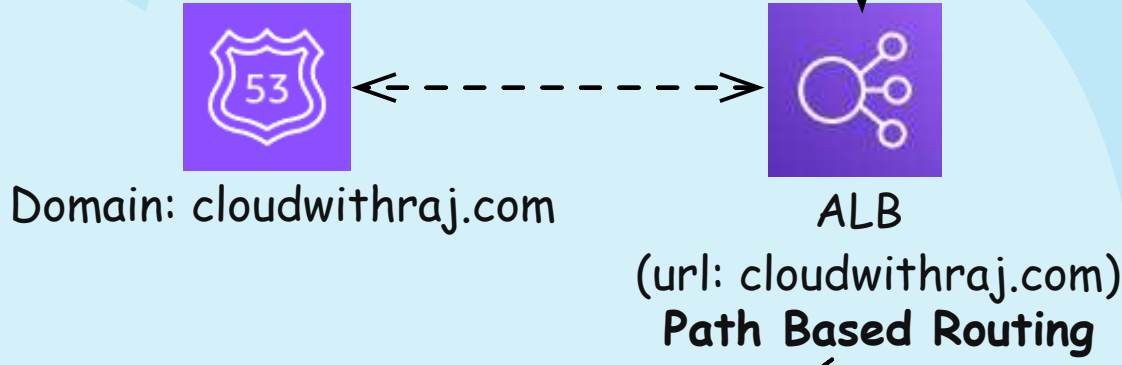
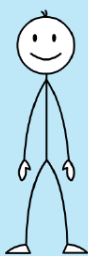
Use Lambda to Transform NOT Transport



Is Lambda inserting/reading data without any business logic or data manipulation?

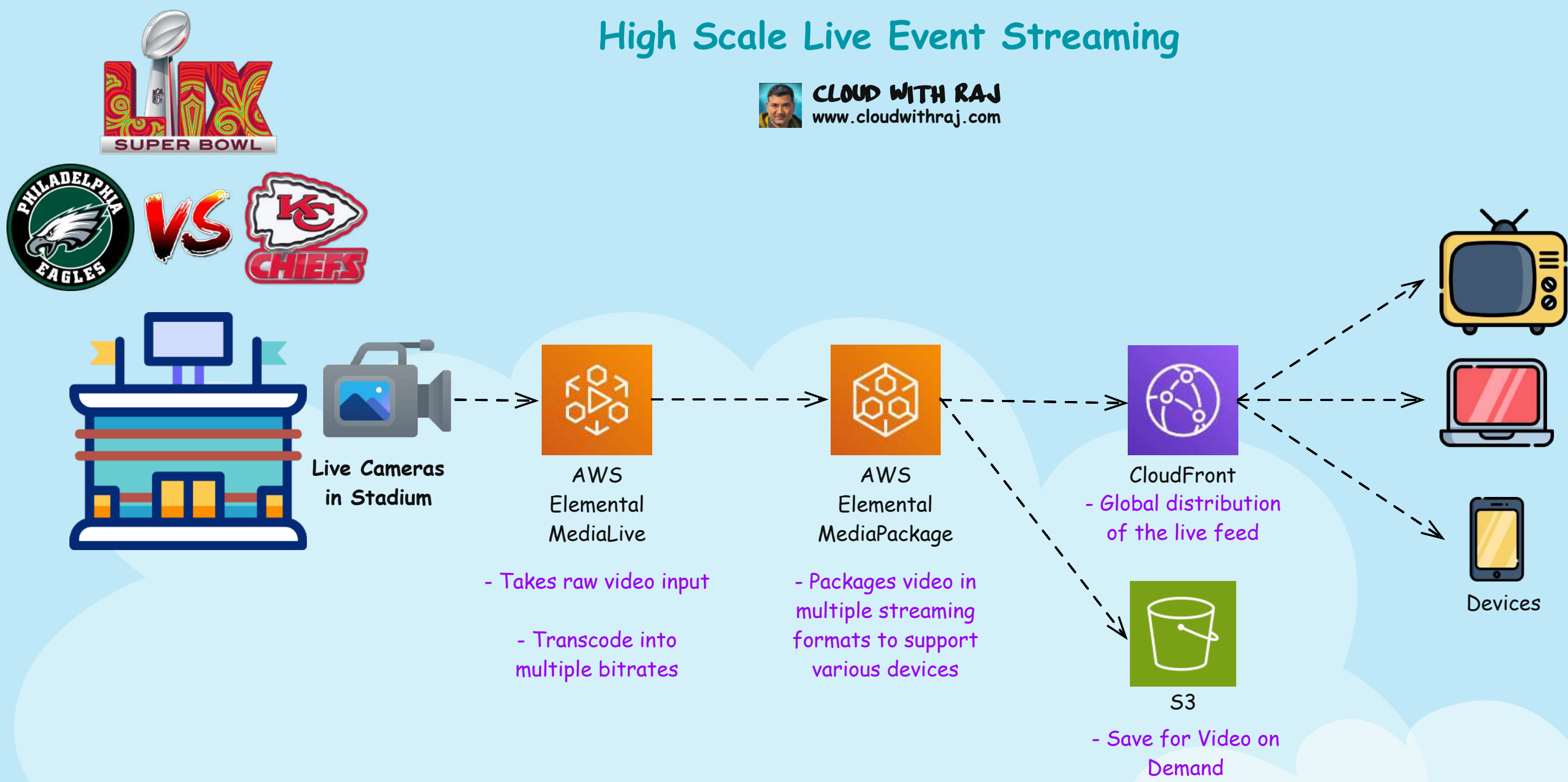


Microservices Tech Stack



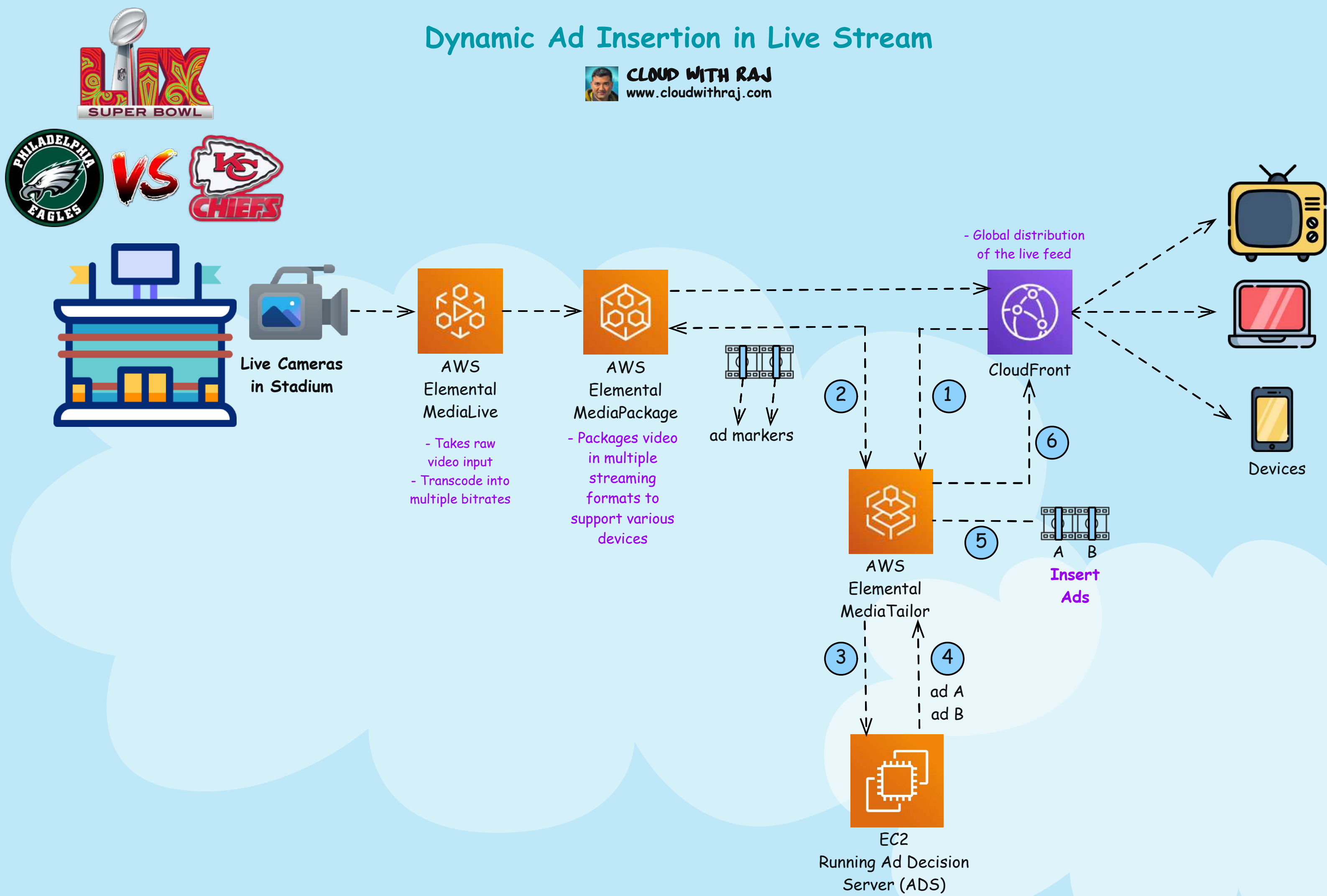
High Scale Live Event Streaming

CLOUD WITH RAJ
www.cloudwithraj.com



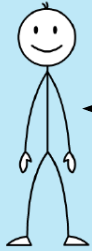
Dynamic Ad Insertion in Live Stream

CLOUD WITH RAJ
www.cloudwithraj.com



1. CloudFront requests manifest (video clip with ads) with viewer information which is used for ad personalization
2. MediaTailor gets manifest i.e. video clip with ad markers (which time frame to insert the ads). At this point NO ads have been inserted
3. MediaTailor requests personalized ads based on viewer information from ADS (Ad Decision Server). ADS is a third party software, that can run on EC2
4. ADS running on EC2 return ad(s) to the MediaTailor
5. Mediatailor inserts these ad(s) into the manifest at the ad markers
6. MediaTailor returns this video clip with ads or manifest with ads to the CloudFront. The clip with ads are shown to the viewers consuming from the CloudFront

API Gateway, Lambda, AWS Service



API Gateway



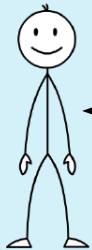
Lambda



DynamoDB

Validate fields not empty
If empty, then reject
Else, apply business logic

API Gateway can validate fields



API Gateway



Lambda



DynamoDB

Validate fields not empty
If empty, then don't call backend, save money
Else, call Backend

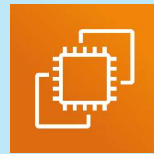
Apply business logic

10 AWS Services for 90% interviews



CLOUD WITH RAJ
www.cloudwithraj.com

Compute



EC2



Lambda

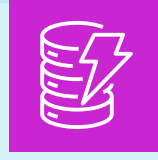
Storage



S3



RDS

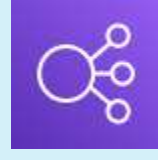


DynamoDB

Network



VPC



Load Balancer



API Gateway

Security



IAM

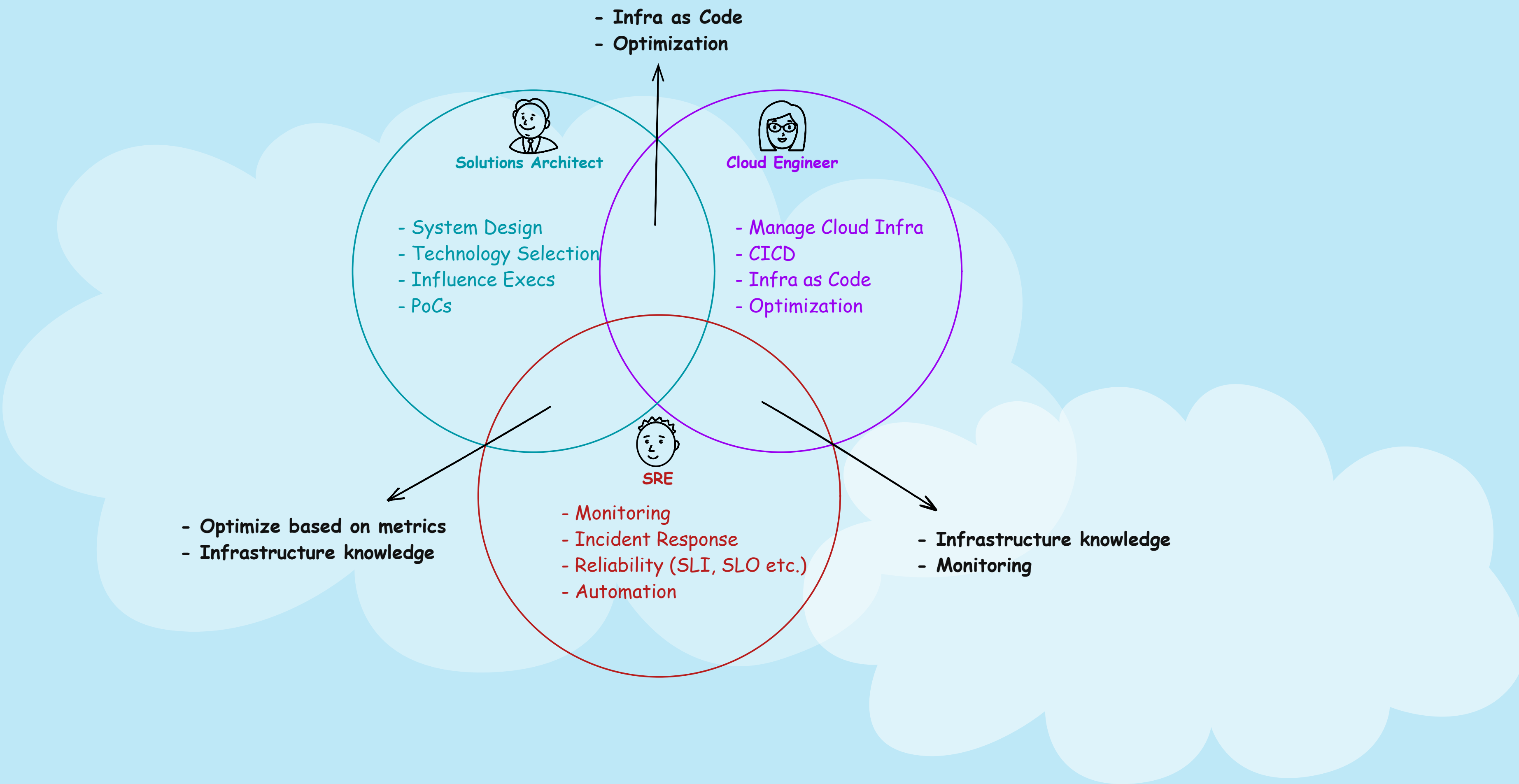
Observability



CloudWatch

- EC2 : Virtual servers in the cloud for running applications
- Lambda : Serverless compute service for running code without managing servers
- S3 : Scalable object storage for data backup, archiving, and analytics
- RDS : Managed database service supporting multiple database engines
- DynamoDB : Fully managed NoSQL database service for fast and predictable performance
- VPC : Isolated cloud resources for enhanced security and network control
- Load Balancer : Automatically distributes incoming application traffic across multiple targets
- API Gateway : fully managed service that enables developers to create, publish, maintain, monitor, and secure APIs at any scale
- IAM : Manage user access and encryption keys for AWS services
- CloudWatch : Monitoring and observability service for AWS resources and applications

Solutions Architect Vs Cloud Engineer Vs SRE



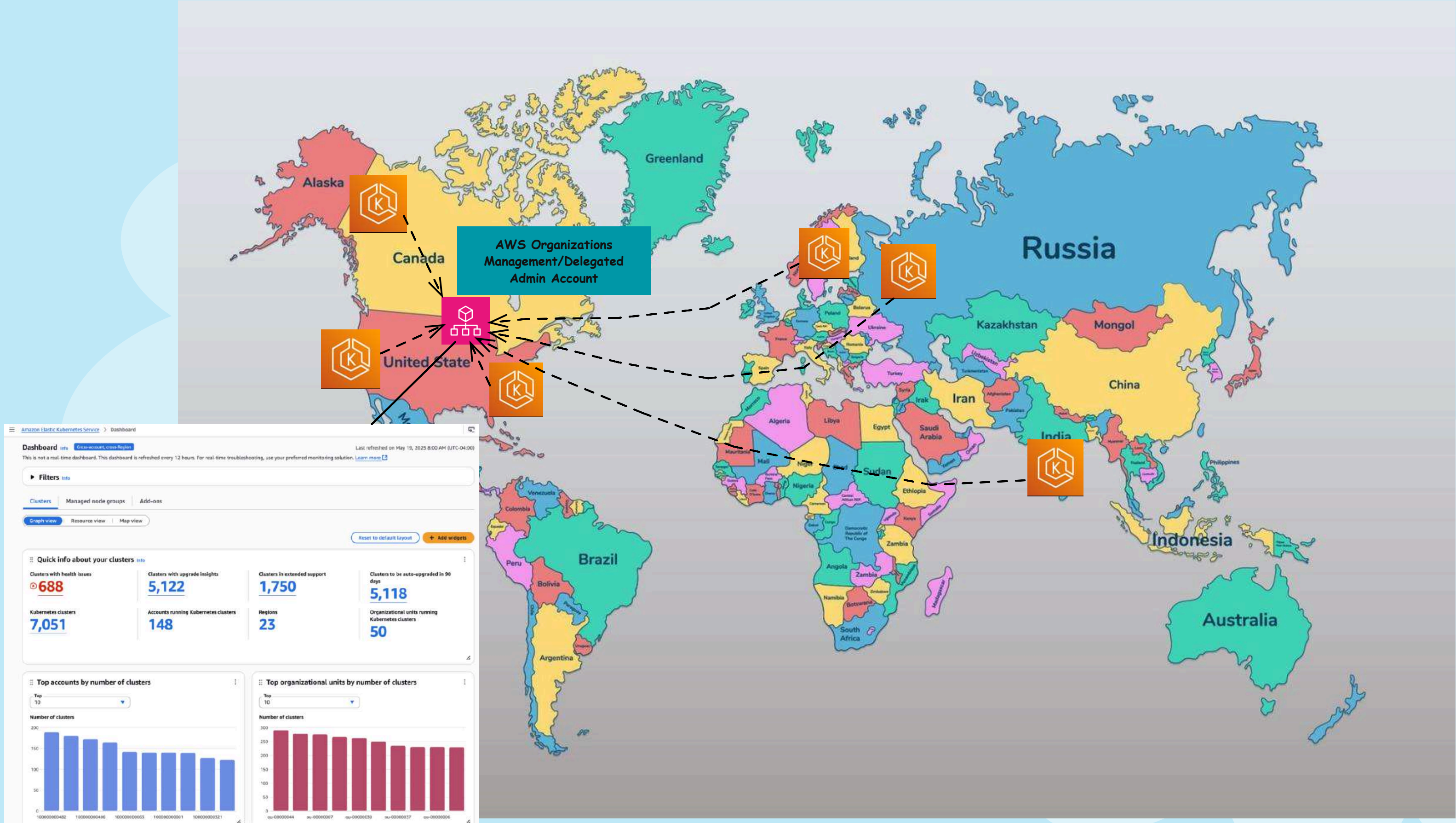
Start LinkedIn post like :
For those 9,999,999,999 boys and girls who are still confused about DevOps vs. SRE vs. Platform Engineering role boundaries...

Here, I've simplified it for you.

I understand that not many organizations have distinct roles defined.

As long as it is well managed and does not impact individuals and teams, so be it.

EKS Multi Cluster Dashboard



Today, AWS is introducing the EKS Dashboard—a powerful, native solution that gives cluster admins a unified, organization-wide view of all their Kubernetes clusters. Here's why this matters:

Centralized Visibility: Monitor clusters across different AWS Regions and accounts from one dashboard—no more switching between tools. It works with clusters running in on-prem or other clouds!

Actionable Insights: Instantly track cluster inventory, compliance, version status, and health metrics to streamline operations and plan upgrades.

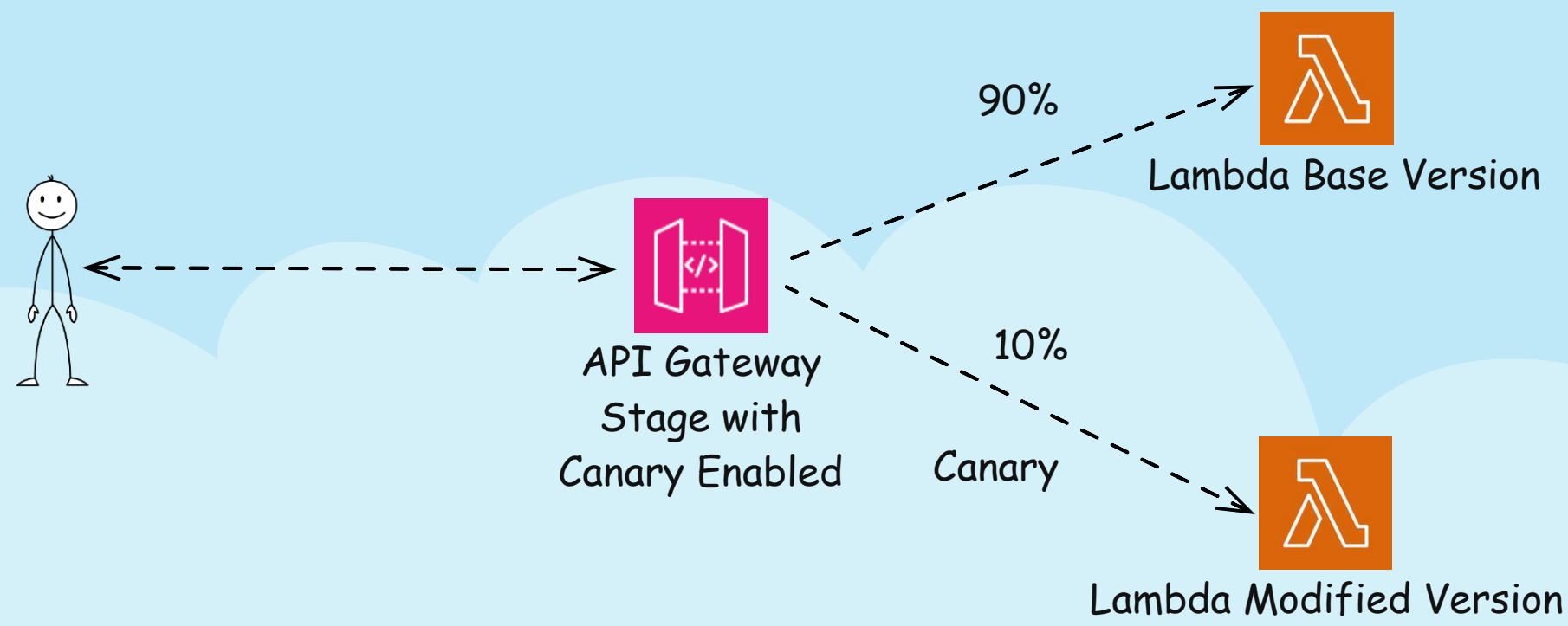
Native Integration: Say goodbye to third-party complexity, extra licensing, and identity headaches—the EKS Dashboard is built right into the AWS Console.

Deep Drill-Down: Automatic filtering lets you zero in on clusters that need attention, so you can act fast and keep your environment healthy.

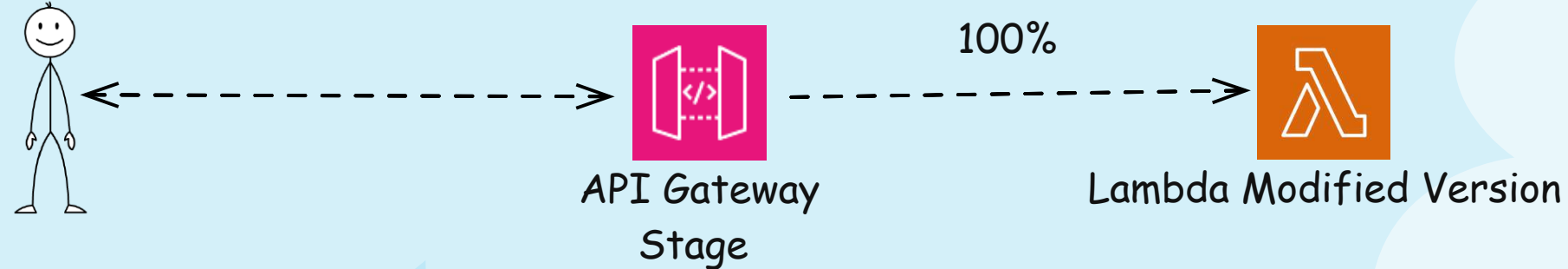
Easy Setup: Enable trusted access with just a few clicks via AWS Organizations and start gaining insights immediately.

API Gateway Canary Deployment

 **CLOUD WITH RAJ**
www.cloudwithraj.com



PROMOTE CANARY



Did you know - Amazon *API Gateway* provides canary deployment feature out of the box?

You can enable *Canary* within an *API Gateway* stage, and assign a % of the traffic to go to a different backend, such as modified *Lambda*

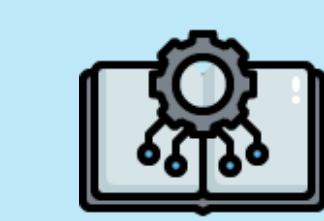
If things go bad with the % traffic testing, you can delete *Canary* and 100% of the traffic will fall back to original base *Lambda*

However, if testing with the % (10% in the diagram) traffic goes fine, then you can "promote" canary, then full traffic will be sent to the *Lambda* modified version and old *Lambda* integration will be deleted.

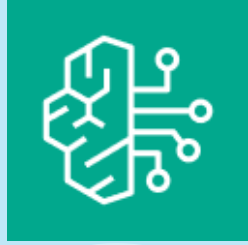
This is very handy because, you don't need to put canary logic in your code. This can also be controlled via *AWS CLI* commands or *Infra as Code*

AWS Gen AI Landscape

 **CLOUD WITH RAJ**
www.cloudwithraj.com



Embedding and RAG



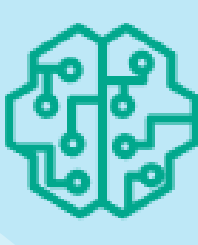
Bedrock
(Knowledge Base)



OpenSearch
Vector
Database



Neptune
for graphRAG
Vector



Embedding Models
(Amazon Titan , Cohere
Embed)



Agentic AI



Bedrock
(Using Action
Groups)



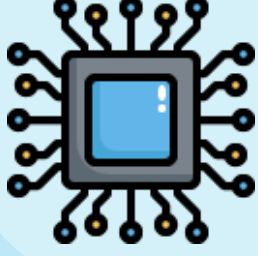
Lambda
(Invoke from
Agents)



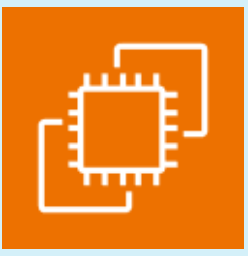
SageMaker AI



Strands Agent



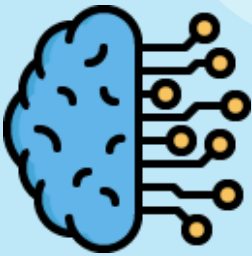
Infrastructure
(Train, and host your
own models)



EC2
(Trainium, Inferentia,
GPUs)



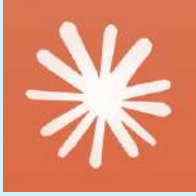
SageMaker AI



Notable LLMs
(Hosted in Bedrock)



Amazon Nova & Titan



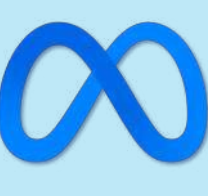
Anthropic Claude



A21 Labs



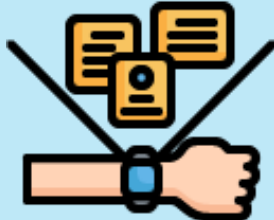
Cohere



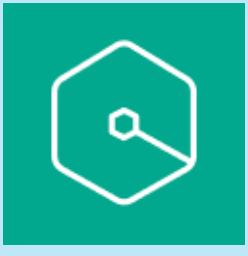
Meta Llama



Stable Diffusion



Coding & Productivity

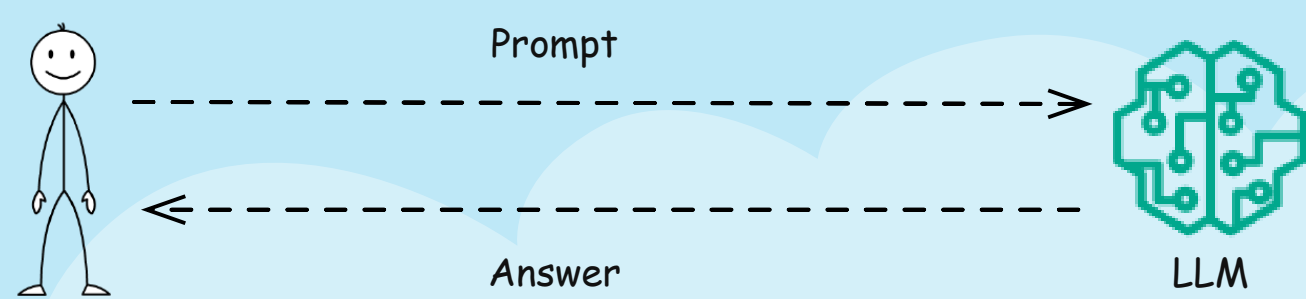


Amazon Q
(Coding and intelligence
Assist)

Agentic AI Evolution



Simple Prompting



Challenges

1. Static, pre-trained data
2. Not able to integrate project-specific data

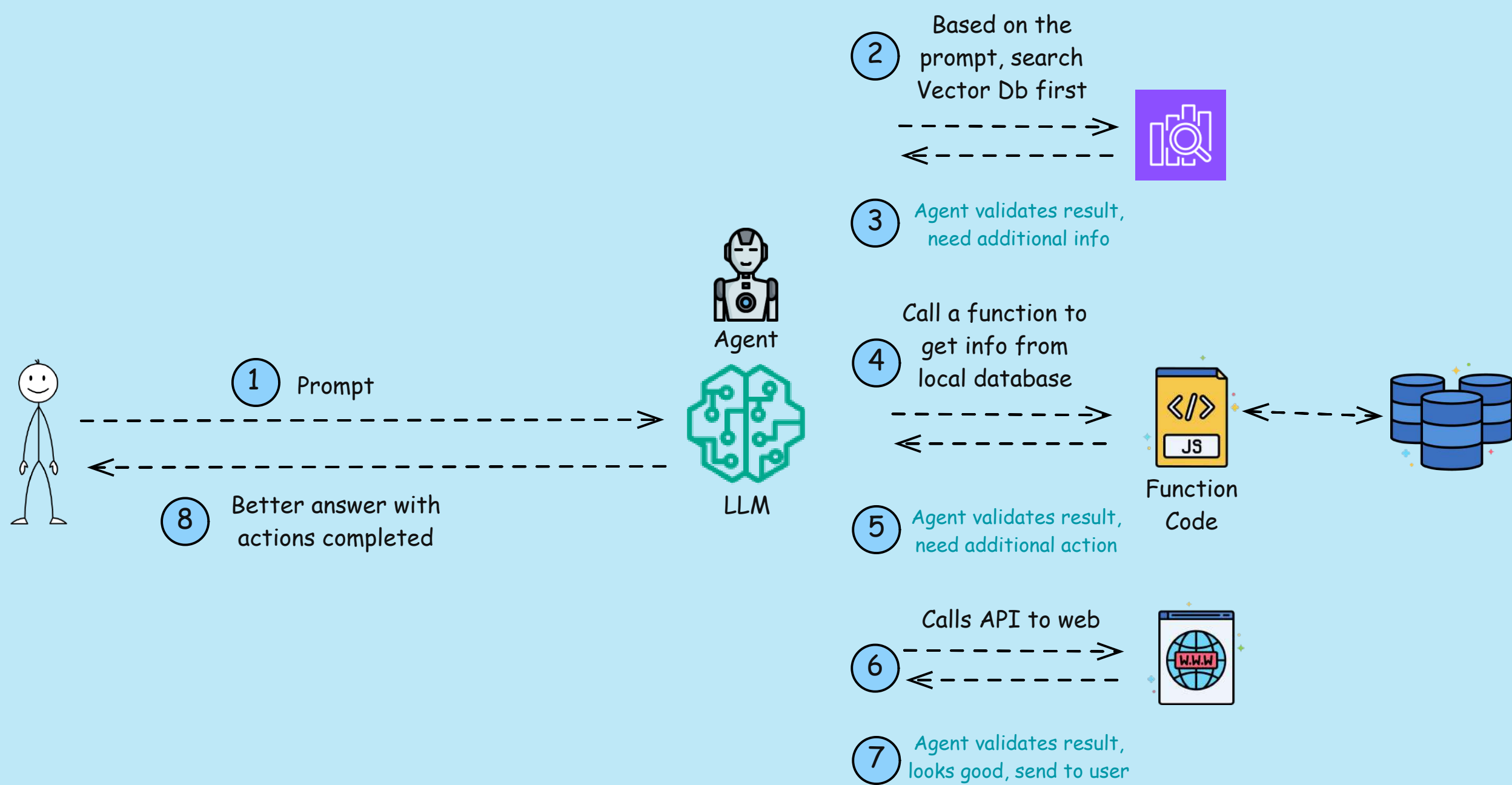
RAG (Retrieval Augmented Generation)



Challenges

1. Once RAG gives you an answer, there is no way to validate, iterate, and refine the answer
2. Non dynamic access to data, heavily biased towards embedded vector database

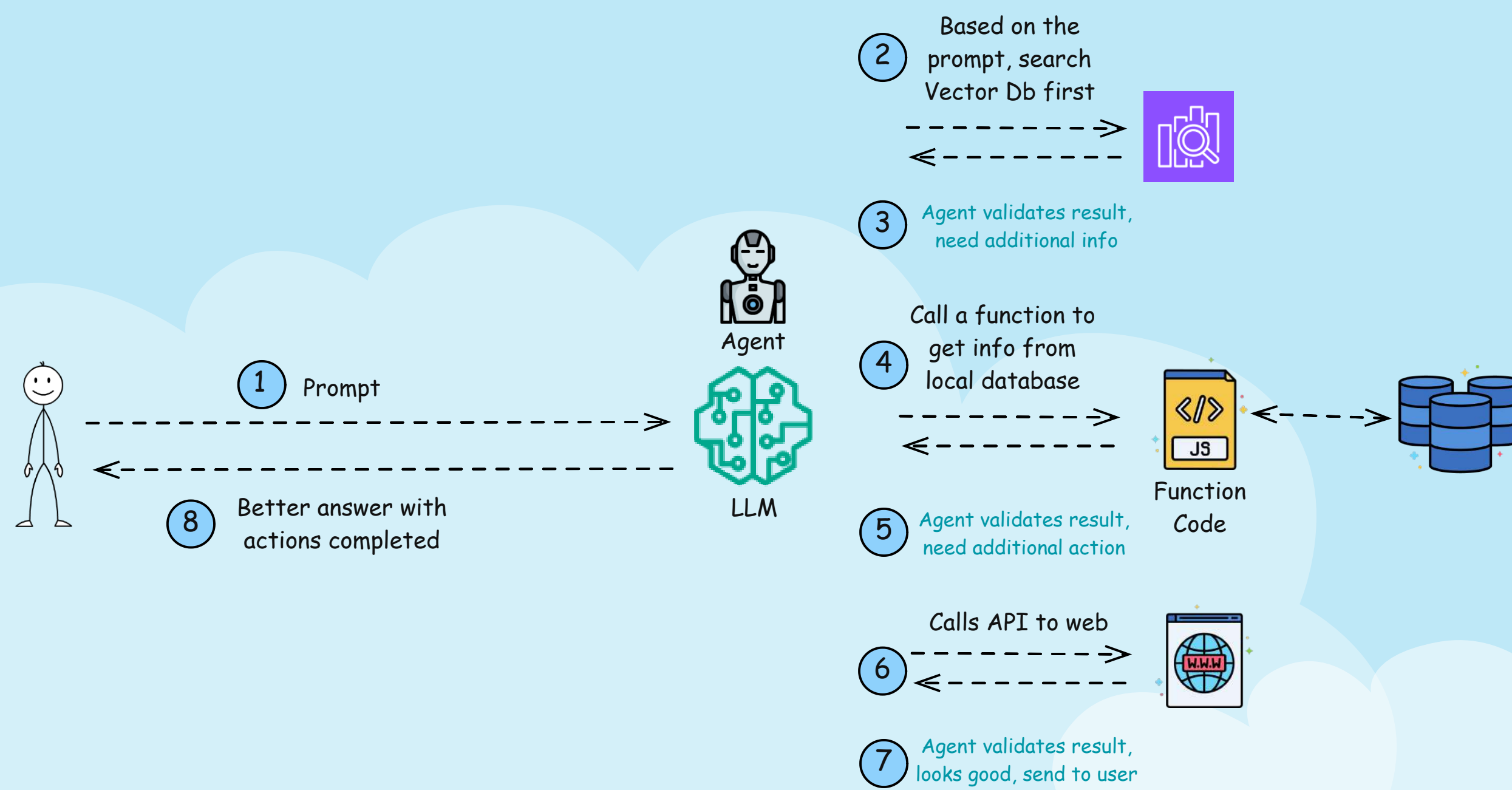
Agentic AI



Advantages

1. Because you can invoke code, you can literally do anything from the LLM. You can even have multiple functions, and LLM can dynamically invoke them based on prompt
2. Iterative flow - LLM agent validates result and iterate and refine
3. Given the flexibility of calling API or code, you can perform actions e.g. book vacation based on budget, send emails based on meeting recroding etc.
4. This is the fundamental flow, you can even invoke multiple Agents and LLMs using Agentic AI

Agentic AI



An AI agent is:

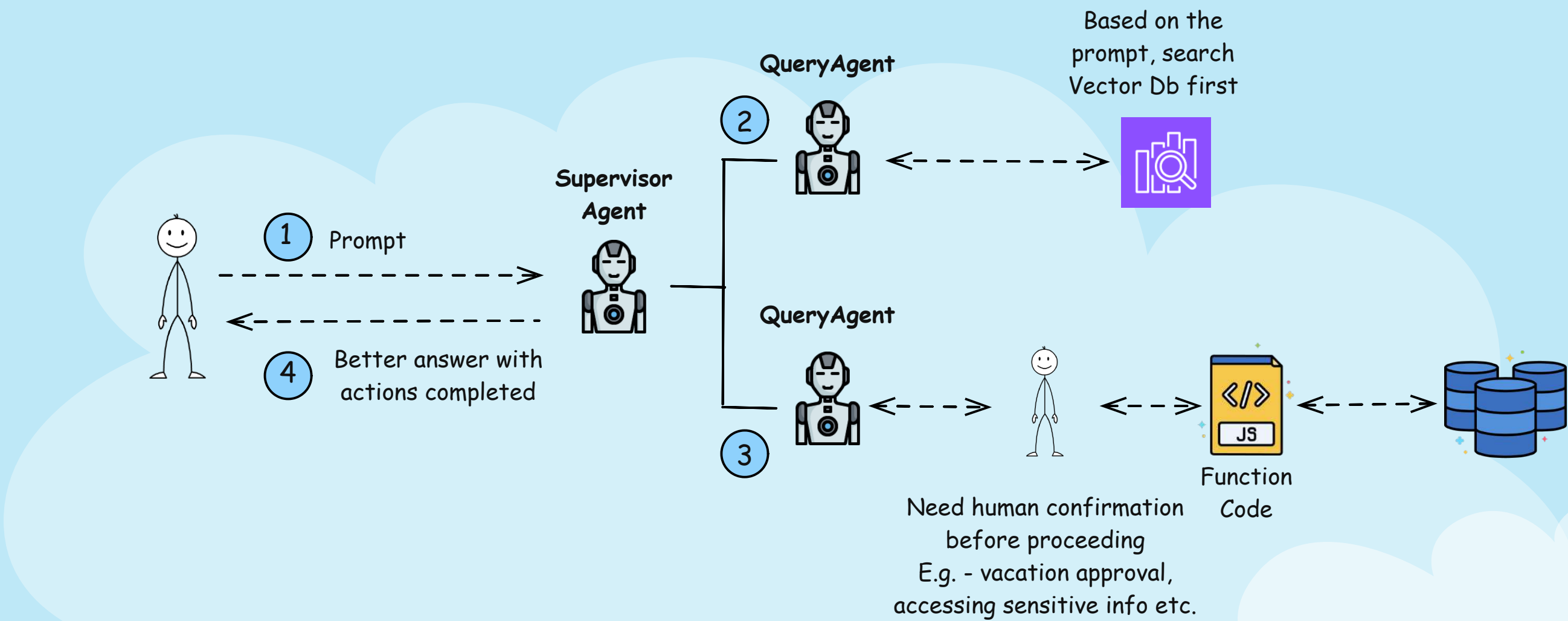
A program (or system) that takes input (observations or data)

Makes decisions using AI models (like LLMs)

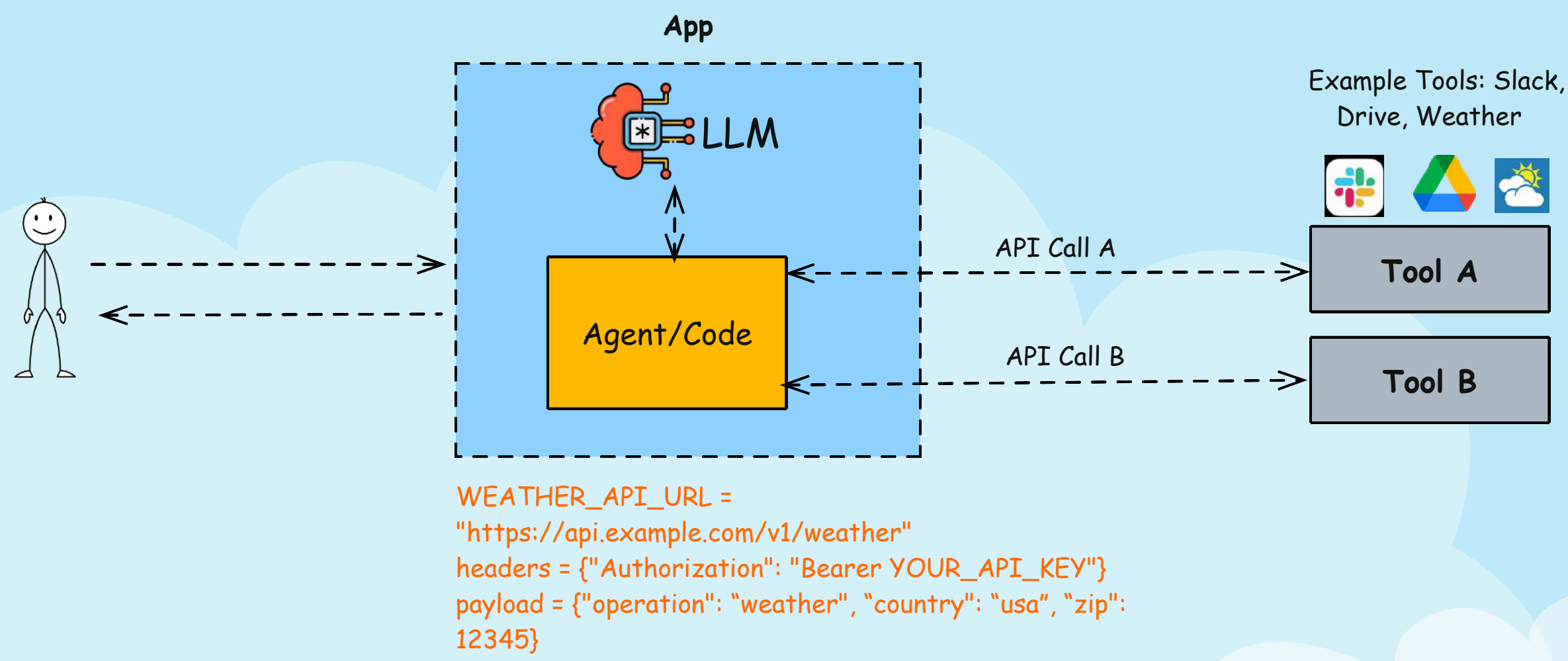
And performs actions (like calling tools, APIs)

Here is the biggest strength of AI agent - it can dynamically and autonomously call various tools, and go through iterations. For example, if the result of one tool call doesn't achieve the desired result, it will call another (or same) tool again. It is NOT just a static if-else loop, but a dynamic feedback loop, making it so powerful

Multi Agent Architecture - Human In The Loop



Challenges Before MCP (Model Context Protocol)



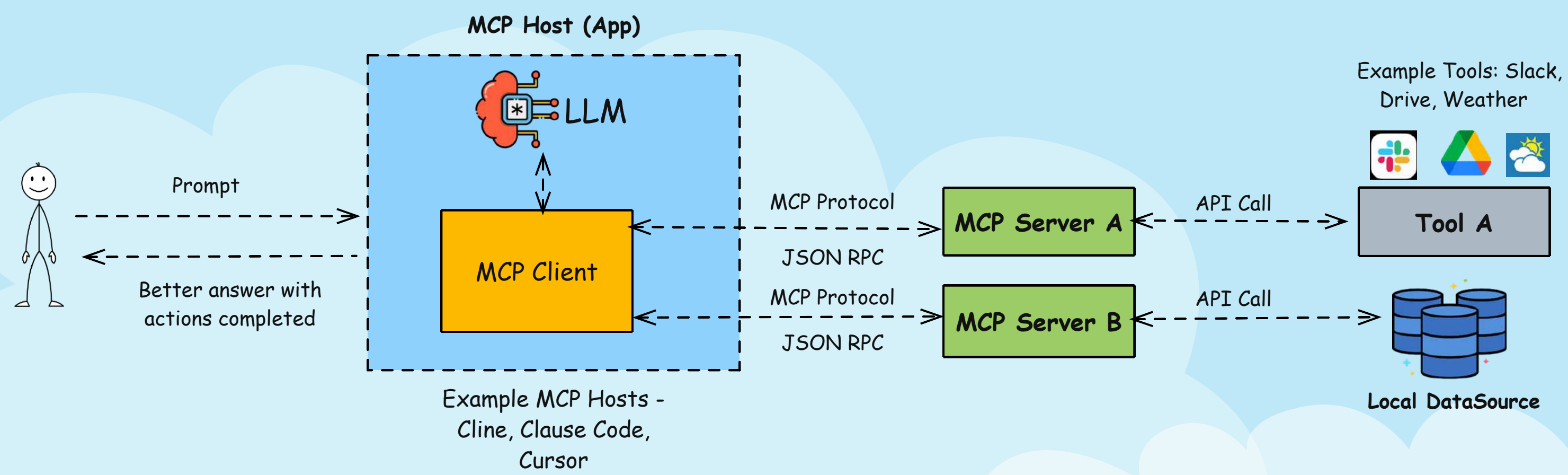
How does the agent code interact with the weather tool? Via API. And to do that, as shown, the agentic code, needs to know the API URL, required header information, and payload.

This works, but there are some challenges too:

- 1. If developers in the weather app changes the API schema, and adds/removes input/output fields, and agentic code doesn't know about it, the connection breaks
- 2. Imagine, in real-world, agents talk to numerous such tools. Each tool requires separate integration with custom code
- 3. It's a tedious task for the app developers to instrument all these separate tools inside the app code

Hence MCP (Model Context Protocol) was born to standardize the communication between Agents and tools. Let's find out about MCP next!

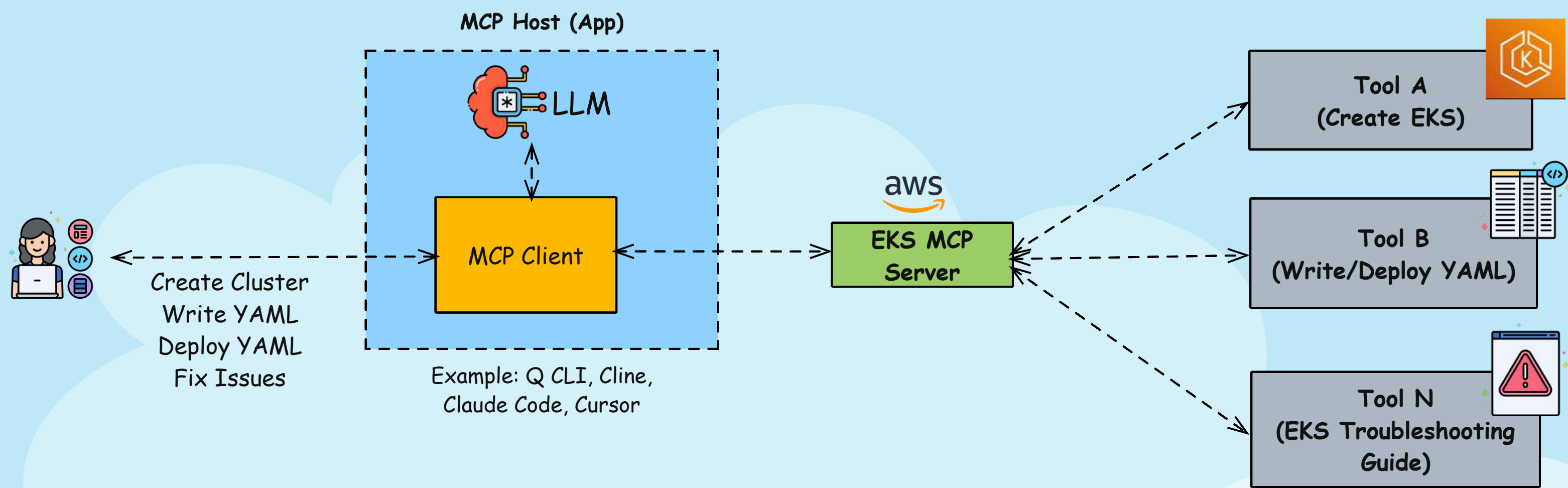
MCP (Model Context Protocol)



MCP standardizes the communication between the agentic code and tools (and local datasources, but tool is the most widely used). What does this mean?

1. An MCP client (think of a piece of code running inside the agent), connects to the MCP server, instead of connecting to the tool directly with a predefined API URL
2. The developers of each tool expose this MCP server
3. MCP client asks the server, "What can you do?". In response, the MCP server responds with the tool capability, description, and input/output schemas
4. IMPORTANT - this discovery is dynamic, and happening at runtime. If input/output field changes, this discovery call will reveal all the fields at runtime
5. The MCP client registers all these, and then can invoke the tool via the MCP server
6. The MCP server handles the connection to the tool. As a result, the code does NOT need to hardcode the API URLs like before
7. The developers of the tools give you the code for the MCP server. You still need to pass authn/z parameters (for e.g. token and team ID for Slack Tool, Access Key and Secret Access Key for AWS Tool etc) to the MCP server. BUT you can run this MCP server locally, or on the cloud. It's up to you

Official AWS EKS MCP Server



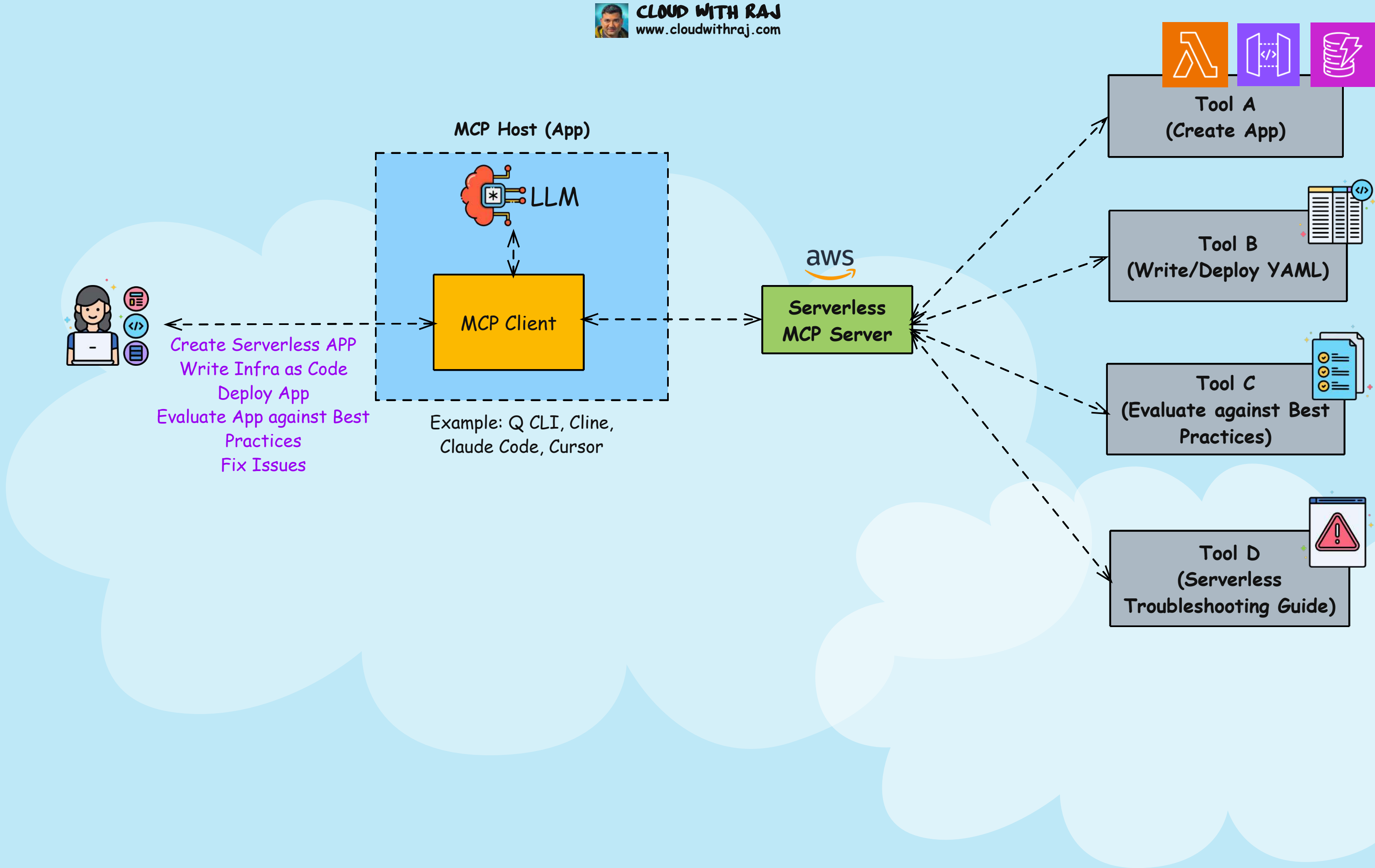
AWS just released an official EKS MCP Server and it's a gamechanger. Here's what it does:

- Creates EKS clusters autonomously
- Writes and edits Kubernetes manifests in your IDE
- Trained on years of real AWS EKS troubleshooting info. This MCP Server Fixes issues super fast
- All controlled with plain English

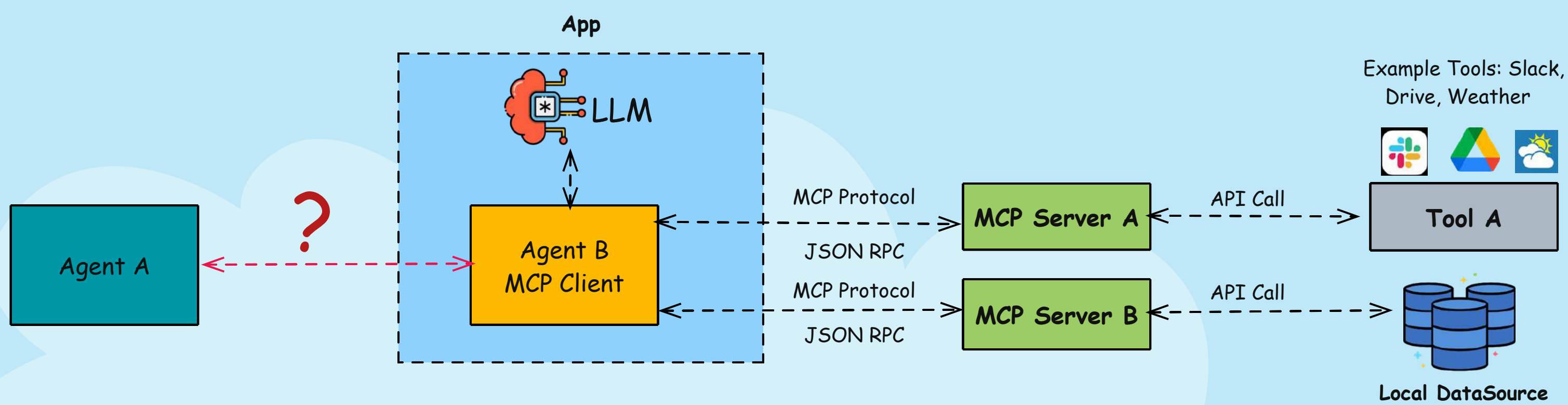
If you're a DevOps, SRE, or Cloud Engineer, learn this before your next interview.

Official AWS Serverless MCP Server

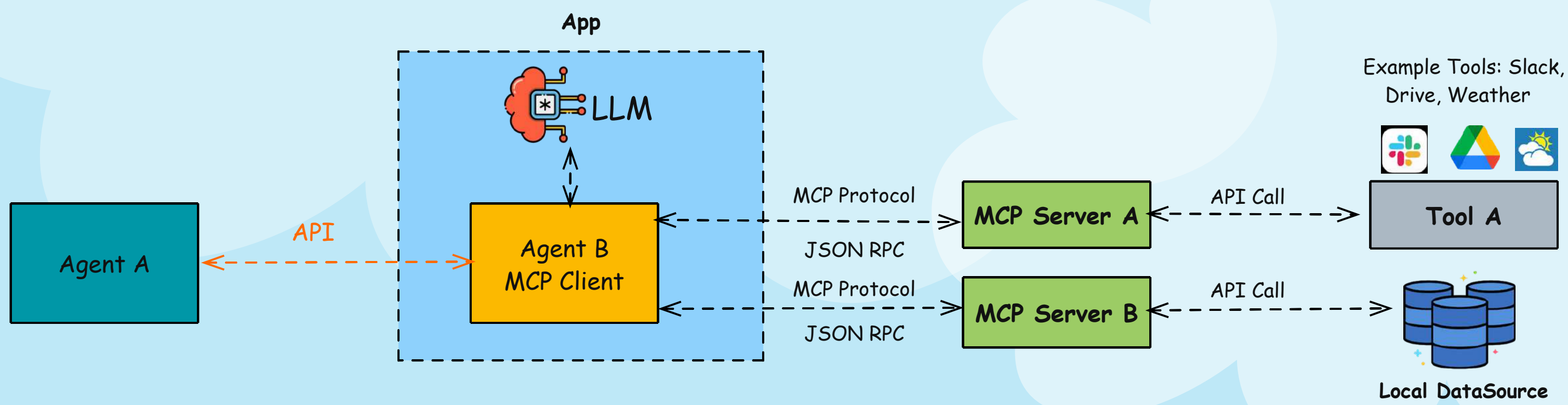
CLOUD WITH RAJ
www.cloudwithraj.com



Challenges Before A2A (Agent 2 Agent)



MCP handles the communication between the agent and tools (and local datasources). But how about agent-to-agent?

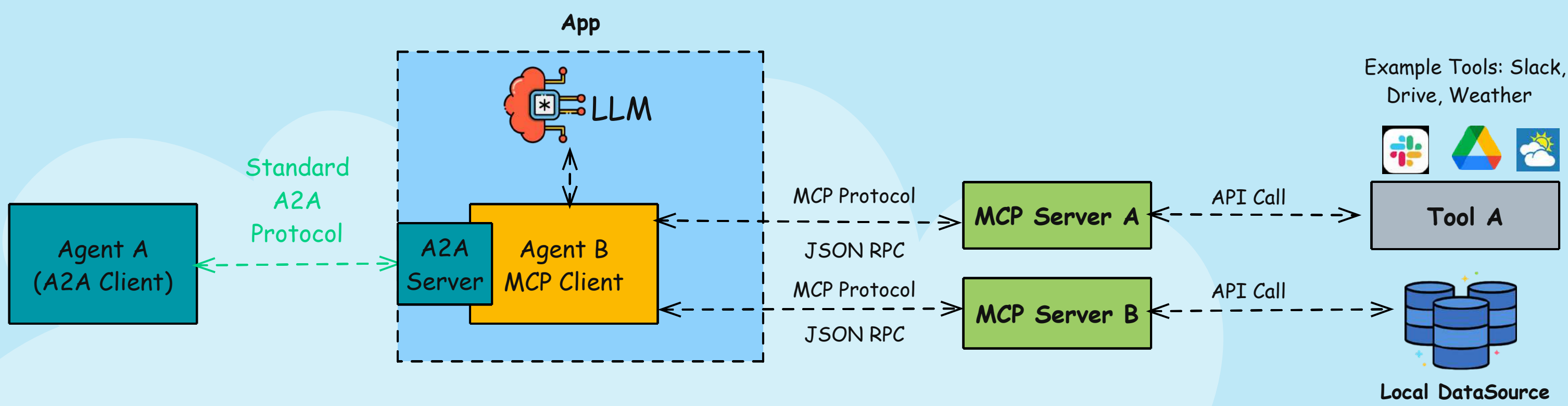


Similar to any other API call, Agent A will invoke the API URL of Agent B, and pass AuthN/Z parameters, and a payload. This brings similar challenges:

- Each agent needs to know what the other agent does beforehand
- Hardcode the connection URL and authn/z parameters
- Have to know how to submit prompts/tasks to the other agent

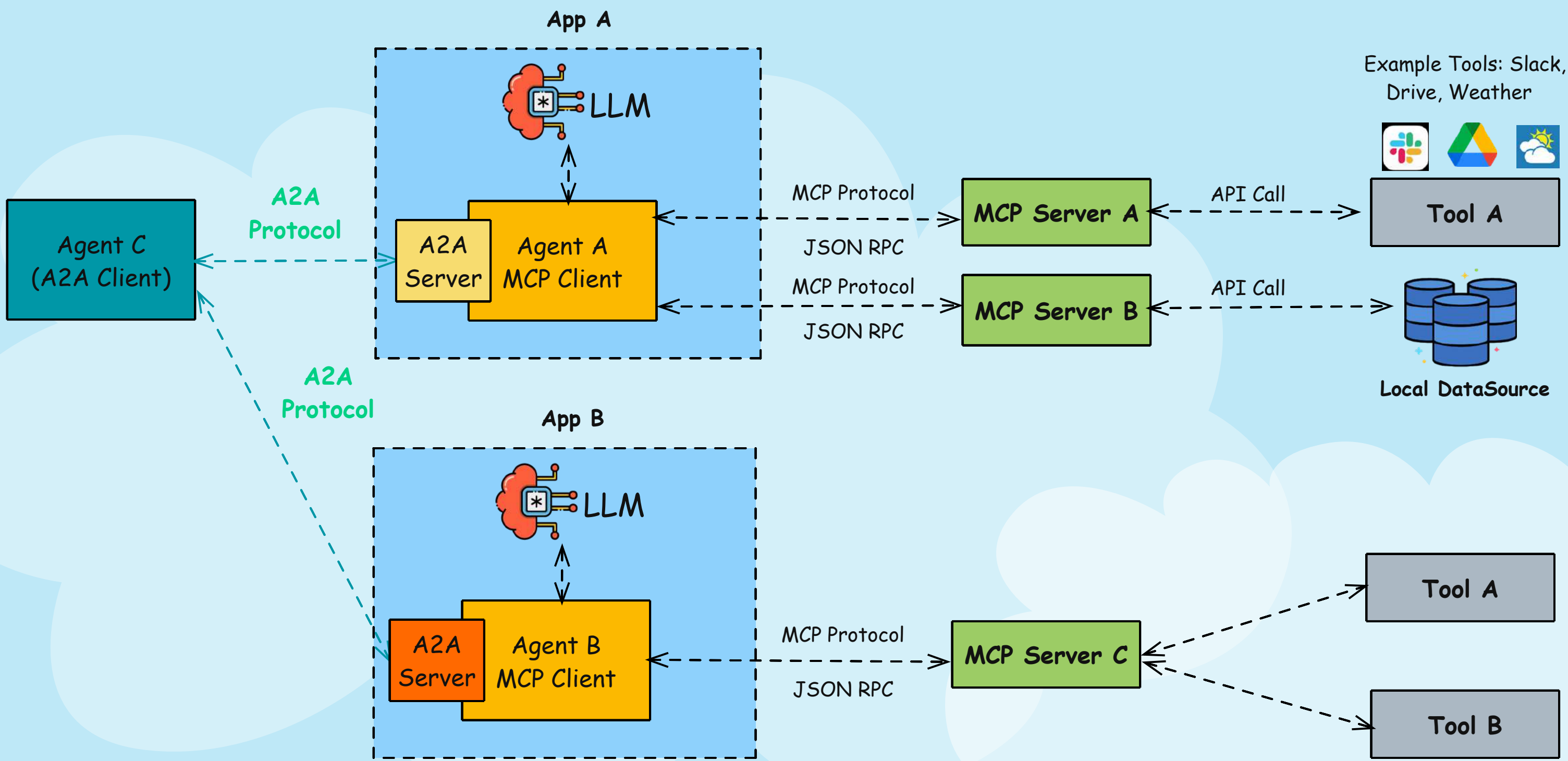
A2A (or Agent2Agent) standardizes the communication between agents. What does this mean? Let's find out next!

A2A (Agent 2 Agent)



1. Agent A, runs a A2A client, which connects to Agent B A2A Server
2. A2A server returns an Agent Card to Agent A
3. This Agent card includes capabilities of Agent B, endpoint URL, how to submit tasks/prompts, how to get notifications on task status, and authn/z mechanism
4. IMPORTANT - Note how dynamic this is (similar to MCP). Using this any agent can discover capabilities, and connection URL, and other info during runtime, and don't need to hardcode beforehand
5. A2A do NOT include payload schema at this time (MCP returns payload schema for the tools/datasources)

A2A (Agent 2 Agent) + MCP (Model Context Protocol)



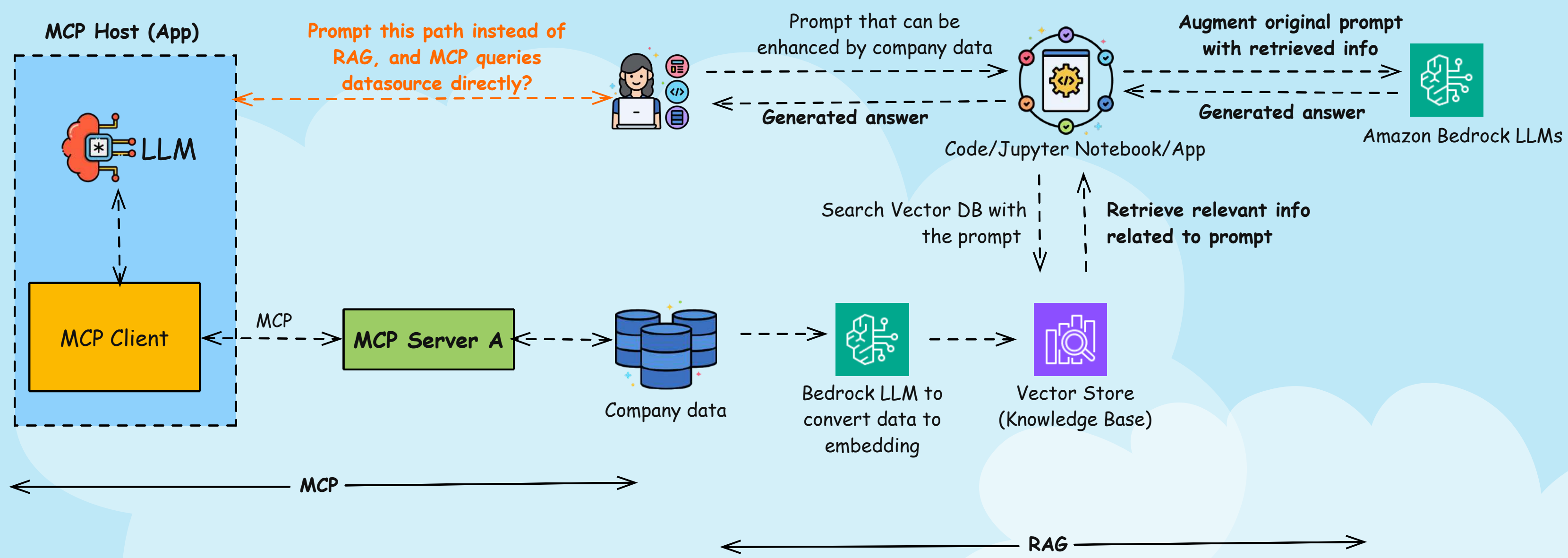
A2A + MCP Flow

1. Agent A, runs a A2A client, which connects to Agent B A2A Server
2. A2A server returns an Agent Card to Agent A
3. This Agent card includes capabilities of Agent B, endpoint URL, how to submit tasks/prompts, how to get notifications on task status, and authn/z mechanism
4. IMPORTANT - Note how dynamic this is (similar to MCP). Using this any agent can discover capabilities, and connection URL, and other info during runtime, and don't need to hardcode beforehand
5. A2A do NOT include payload schema at this time (MCP returns payload schema for the tools/datasources)

In summary, MCP standardizes the connection between LLM Agents and tools, and A2A standardizes the connection between two agents. They work hand in hand because they complement each other and do not compete with each other.

Detailed Video: 

Can MCP Replace RAG?



The obvious question on your mind is, well, if MCP can access the datasource, and the same datasource is being used in RAG, why don't I skip the whole RAG part, and just ask my question to the MCP Host App as shown above? There are a couple of problems with this approach:

- If you want MCP to connect to the database directly, that means the app team needs to give access to the database to the MCP server.
- If you have ever worked at an enterprise, you know teams don't want to give access to their actual database. You might think this is territorial (part of it is!), but there are also good reasons for it:
- If one team is granted access, more teams will be granted access
- As these teams access the actual database, it eats away at the read and write capacity for the business application to serve customers
- Database is expensive, and scaling up means more cost
- App team, managing the database, can't change the schema freely because it'd break the MCP flow

RAG actually solves this problem, how?

- RAG flow does NOT access the actual database, but the vector database. This way, RAG queries don't consume database capacity
- The embedding process typically runs nightly during off-hours
- Embedding into a vector database also indexes the data, ideal for text queries
- The vector database is not impacted even if the schema is changed in the underlying database

As we can see, RAG also has some strengths. And MCP is already powerful. What now? Turns out there is a middle ground, a best-of-both-worlds solution.

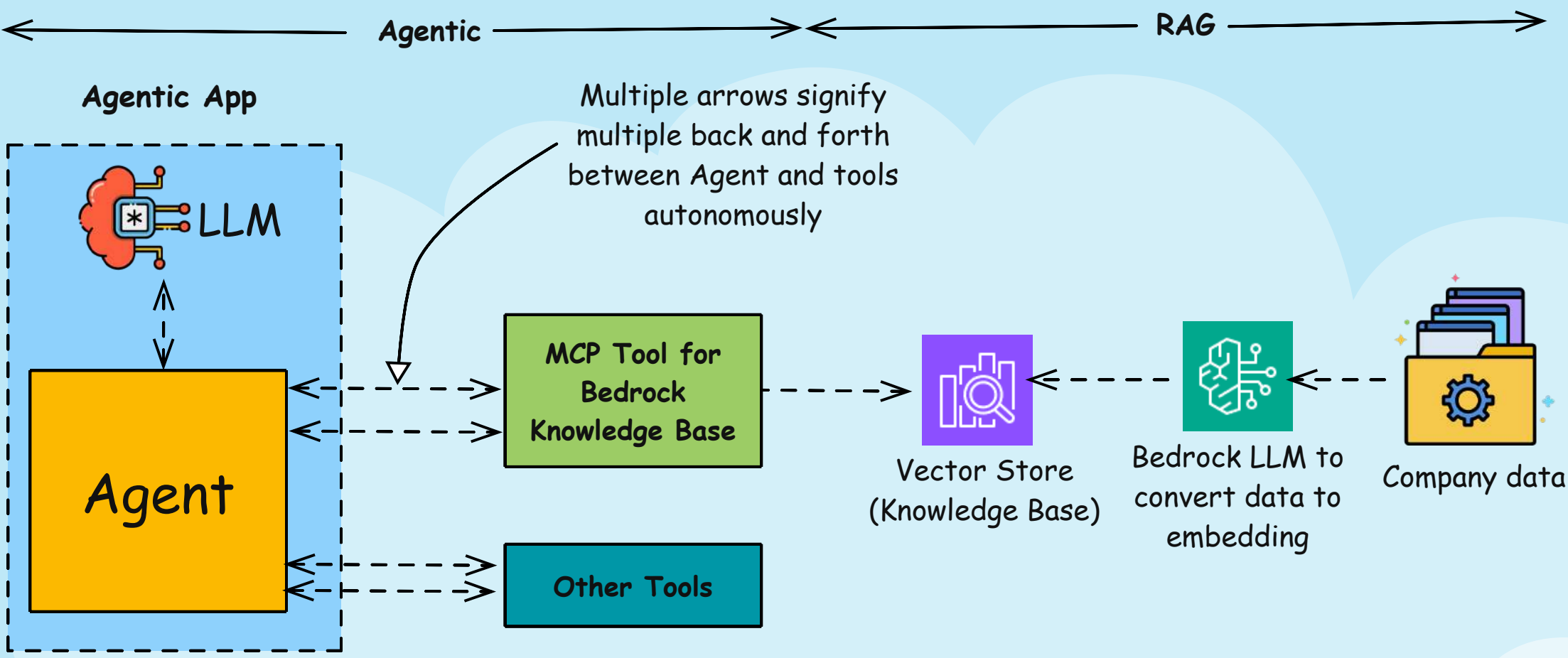
Sometimes you do want to query the MCP host because the MCP host has access to many tools, and even other agents! In those cases, if you need to utilize the vector database, MCP has a server that interacts directly with the vector database! For AWS, such MCP server can interact directly with Bedrock Knowledge Base which is basically the vector database.

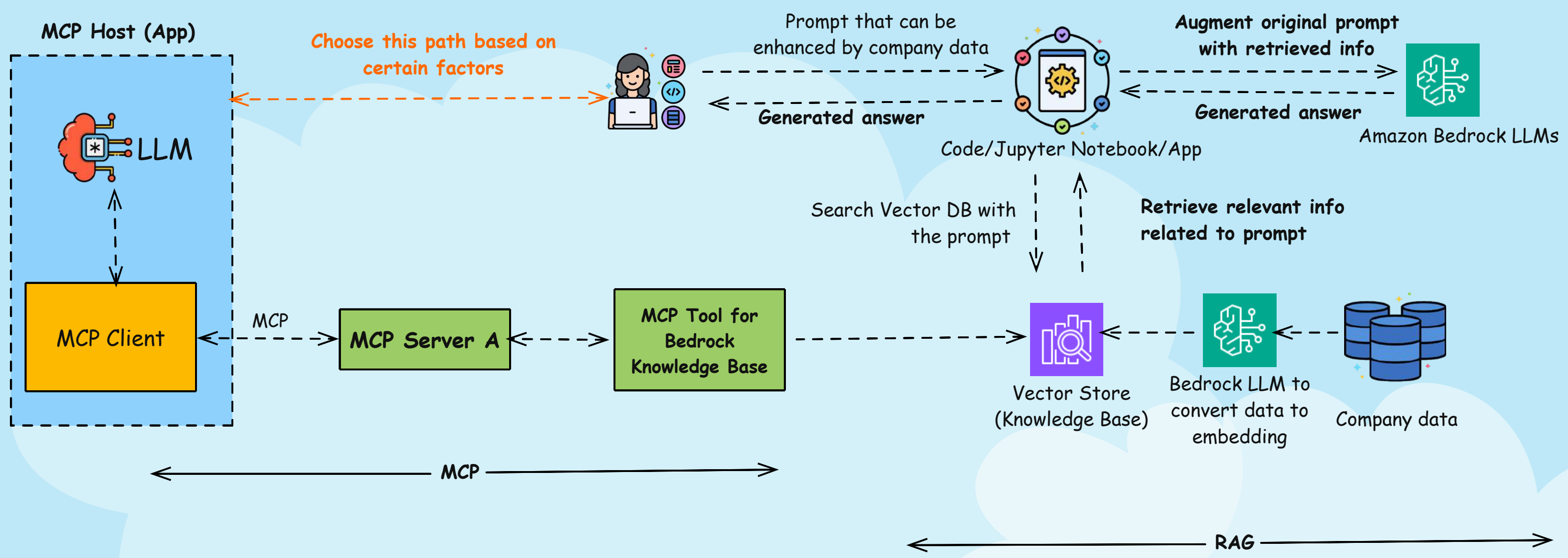
On the other hand, if you need the RAG flow, you can go the RAG route. RAG route is generally faster because it just queries the vector database. Meanwhile, for the MCP flow, handshaking is involved, as discussed above in the brief MCP overview. This back and forth introduces some latency.

In summary, RAG is going nowhere, and MCP complements RAG! If you get this question in your interview or projects, knock it out of the park!

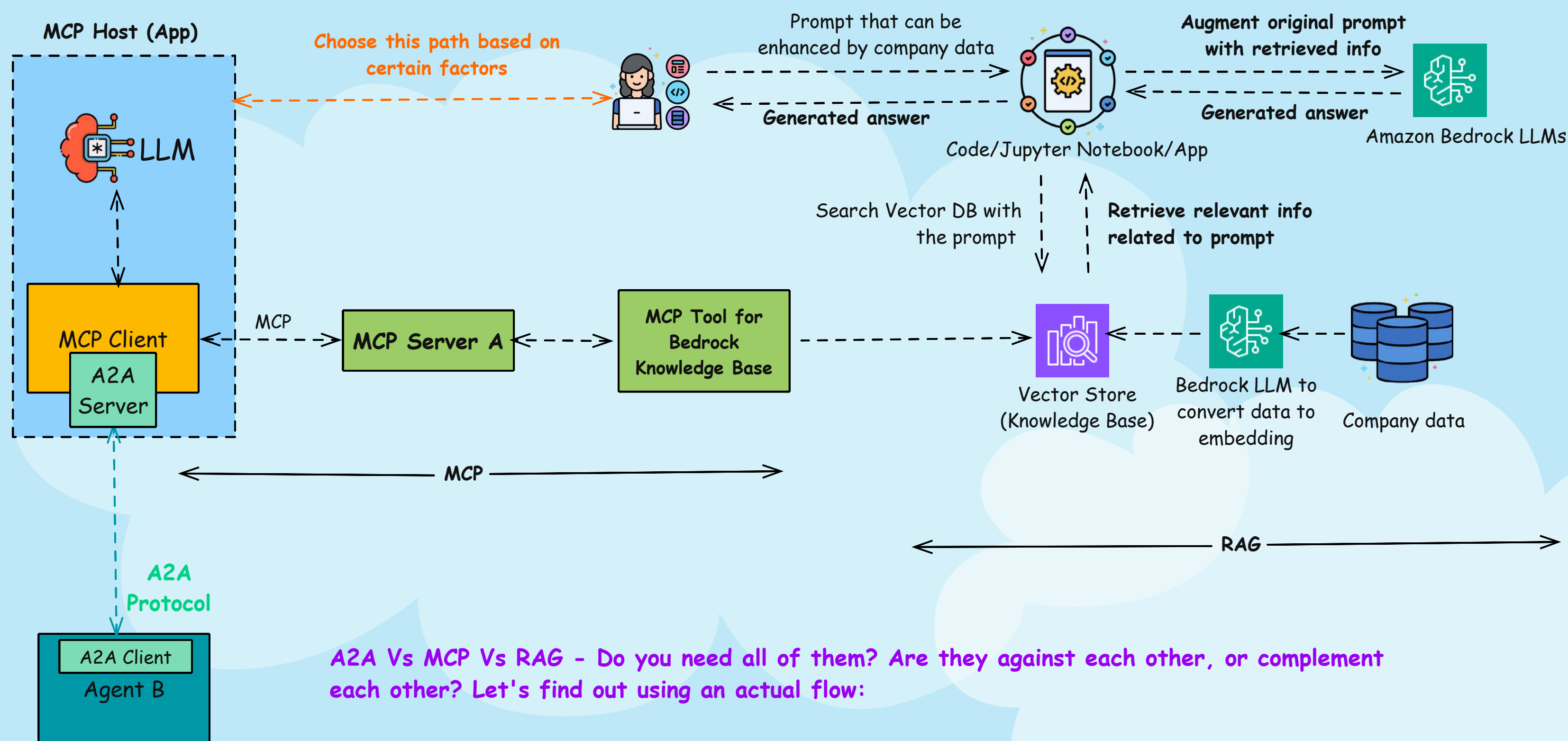
Agentic RAG

 **CLOUD WITH RAJ**
www.cloudwithraj.com





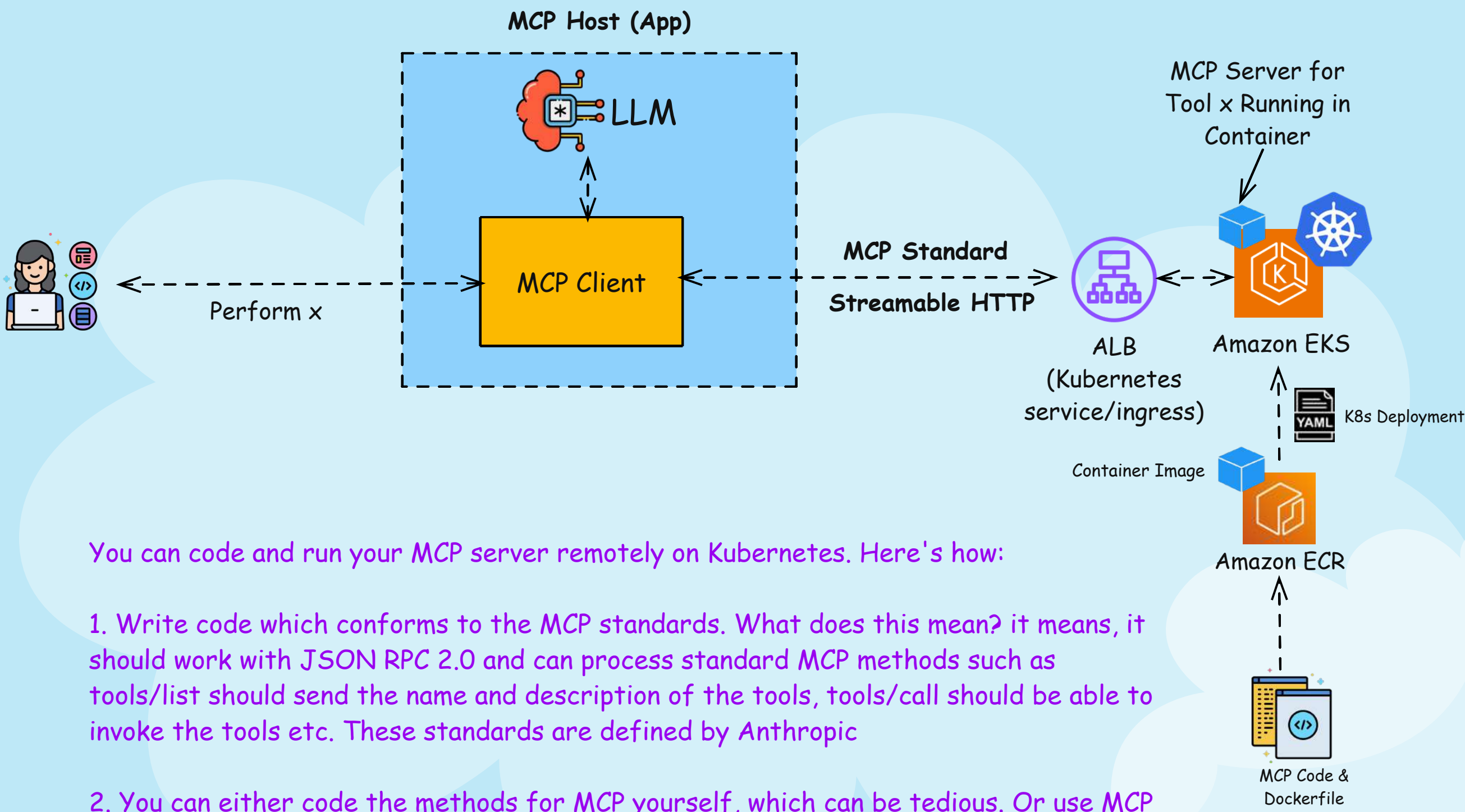
MCP with RAG with A2A



A2A Vs MCP Vs RAG - Do you need all of them? Are they against each other, or complement each other? Let's find out using an actual flow:

1. On the right, we have traditional RAG flow. Company data, converted to embeddings, stored in a vector store (Knowledge Base in Bedrock), and used in RAG
2. MCP actually makes RAG easier! Using MCP for Knowledge Bases, you can connect to this Knowledge Base directly
3. You can also bypass RAG and just invoke MCP Host when needed
4. A2A or Agent to Agent has nothing to do with connecting tools (such as Knowledge Base shown here). It facilitates one agent communicating with another.
5. IMPORTANT - Note how dynamic MCP, and A2A makes it. They can dynamically discover tools and agent capabilities, respectively, and act on it.

Remote MCP Server



You can code and run your MCP server remotely on Kubernetes. Here's how:

1. Write code which conforms to the MCP standards. What does this mean? it means, it should work with JSON RPC 2.0 and can process standard MCP methods such as tools/list should send the name and description of the tools, tools/call should be able to invoke the tools etc. These standards are defined by Anthropic
2. You can either code the methods for MCP yourself, which can be tedious. Or use MCP implementation with library like FastMCP
3. Run Dockerfile to create a container and save it in ECR
4. Deploy to the Kubernetes cluster such as in Amazon EKS, and expose it using a ALB via service or ingress. ALB and container support streamable HTTP out of the box, and works nicely
5. Invoke your MCP Server using the ALB Url. For Streamable HTTP, you need to initialize a session, and then use the sessionID for subsequent calls to do tool discovery, and tool calls

Demo with actual code snippet:

AWS Strands Agent

 **CLOUD WITH RAJ**
www.cloudwithraj.com



Strands (Python SDK)

No coding required for
common tools



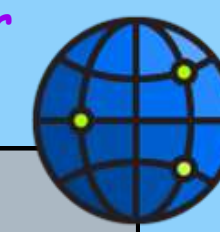
LLM

Bedrock hosted LLM or
external model

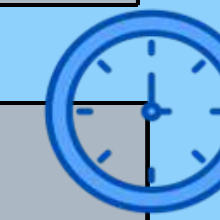
Automatically invoke
appropriate tool (no
description needed)

Agent(s)

Tool 1
Get lat long of NYC



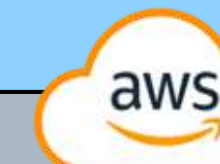
Tool 2
Get time using lat long



Tool 3
Get weather using lat
long



Tool 4
Get S3 buckets using
AWS creds



MCP Server

Strands has built in
MCP client to connect
to MCP Servers



1. What's the time in NYC?
2. What's the weather in NYC?
3. List the S3 buckets in my AWS account

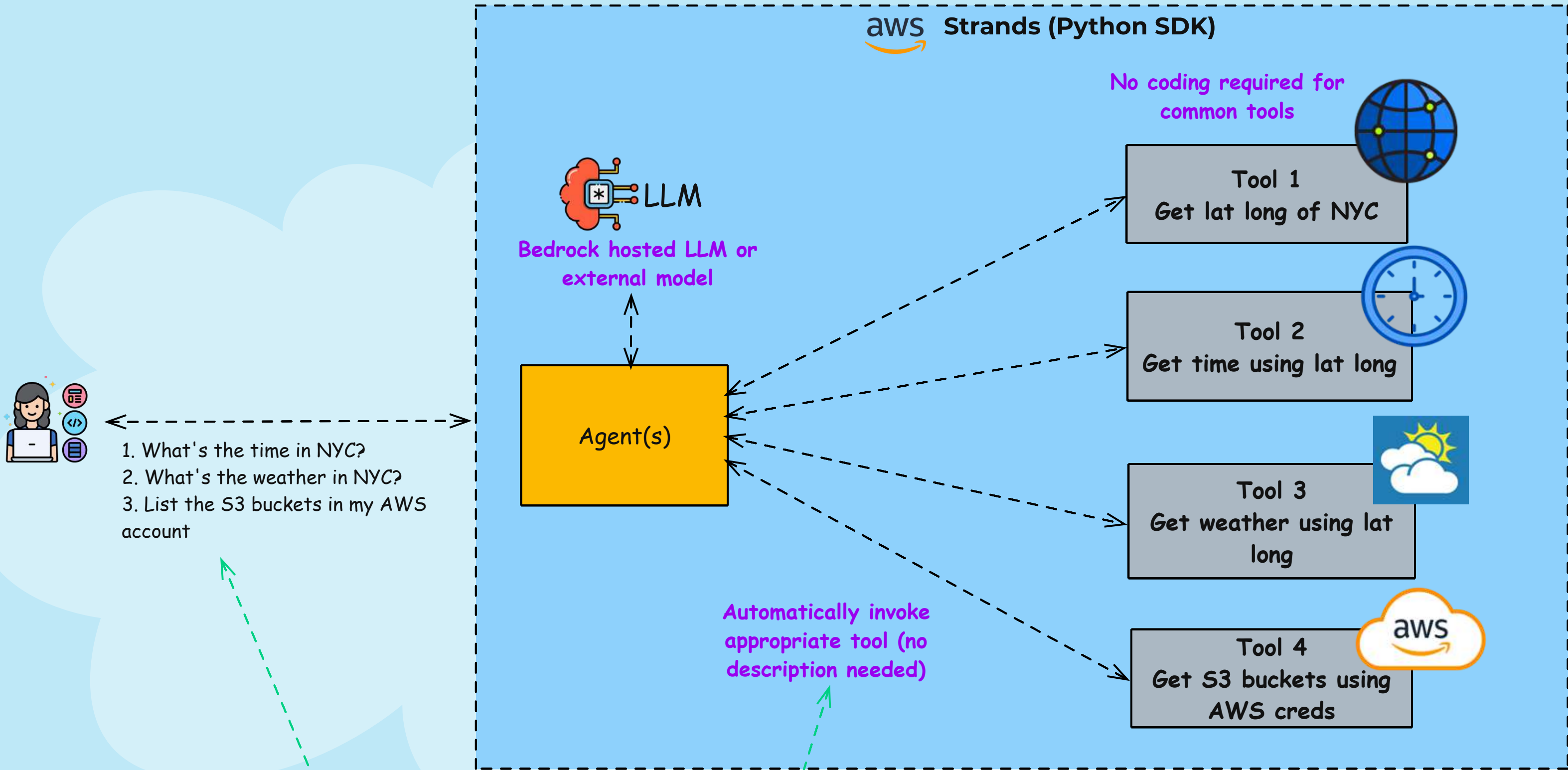
Think of AWS Strands as a single Python program that can implement powerful agentic workflows with a minimum amount of code. Why?

- Supports both Bedrock and external LLMs (e.g., Anthropic Claude)
- This one is awesome - it comes with 20+ prebuilt tools, that you do NOT have to code. For example: `current_time` (gives current time), `use_aws` (uses boto3 under the hood to list AWS resources), `http_request` (figures out API calls automatically)
- You can create your own custom tool
- It has MCP client built in, allowing you to connect with MCP Servers

Demo with actual code snippet:



AWS Strands Agent Code Snippet



```
1 from strands import Agent
2 from strands.models.anthropic import AnthropicModel
3 from strands.tools import current_time, http_request, use_aws
4
5 # Uses Bedrock Claude if no model given, can also use external model if specified
6
7 WEATHER_SYSTEM_PROMPT = """You are a weather assistant with HTTP capabilities. You can:
8 1. Make HTTP requests to the National Weather Service API
9 2. Process and display weather forecast data
10 3. Provide weather information for locations in the United States
11
12 When displaying responses:
13 - Format weather data in a human-readable way
14 - Highlight important information like temperature, precipitation, and alerts
15 - Handle errors appropriately
16 - Convert technical terms to user-friendly language
17
18 Always explain the weather conditions clearly and provide context for the forecast.
19
20 # Create agent with Claude 4
21 agent = Agent(
22     tools=[current_time, http_request, use_aws],
23     system_prompt=WEATHER_SYSTEM_PROMPT
24 )
25
26 response = agent("1. what is the time in new york city? 2. What is the weather in new york city? 3. list the s3 buckets in my aws account")
27 print(response)
28
```

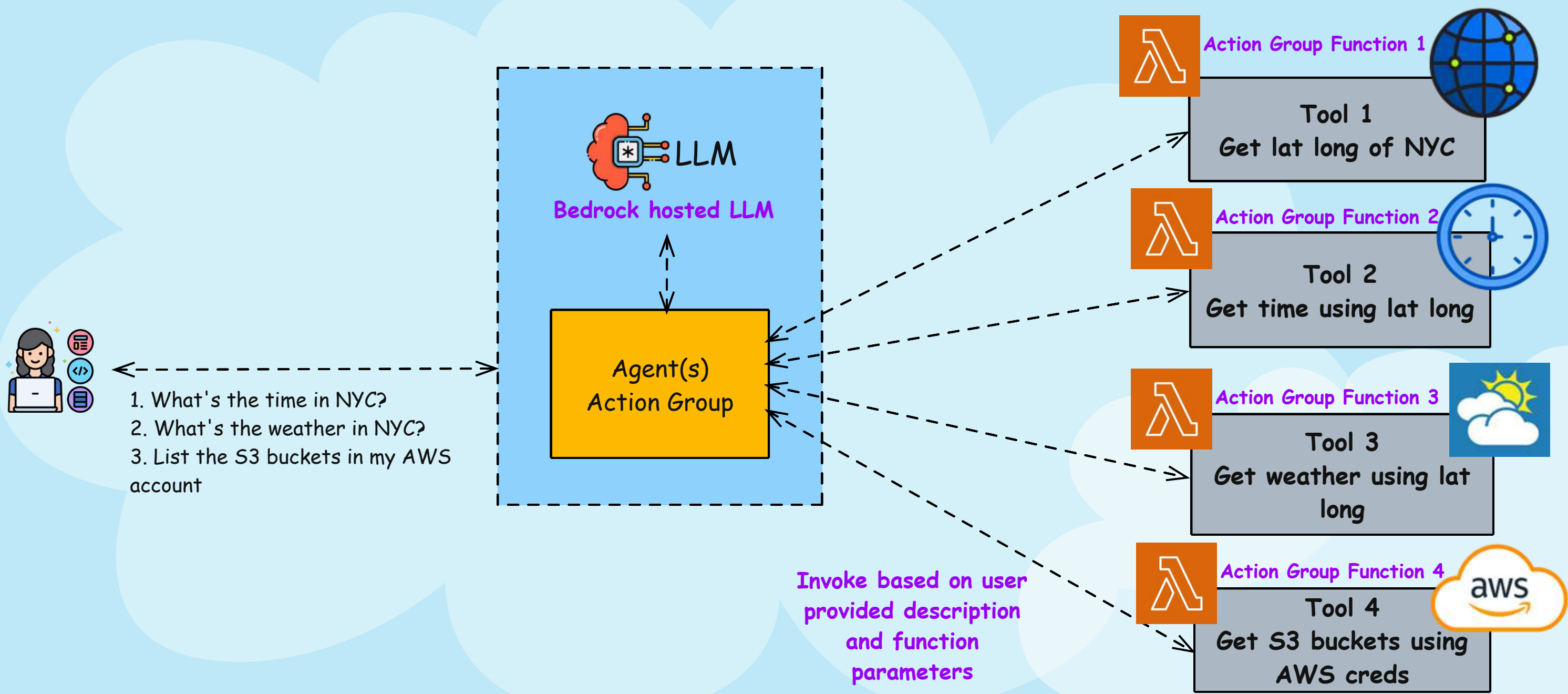
Note: I am just invoking pre-built tools and not coding

Demo with actual code snippet:

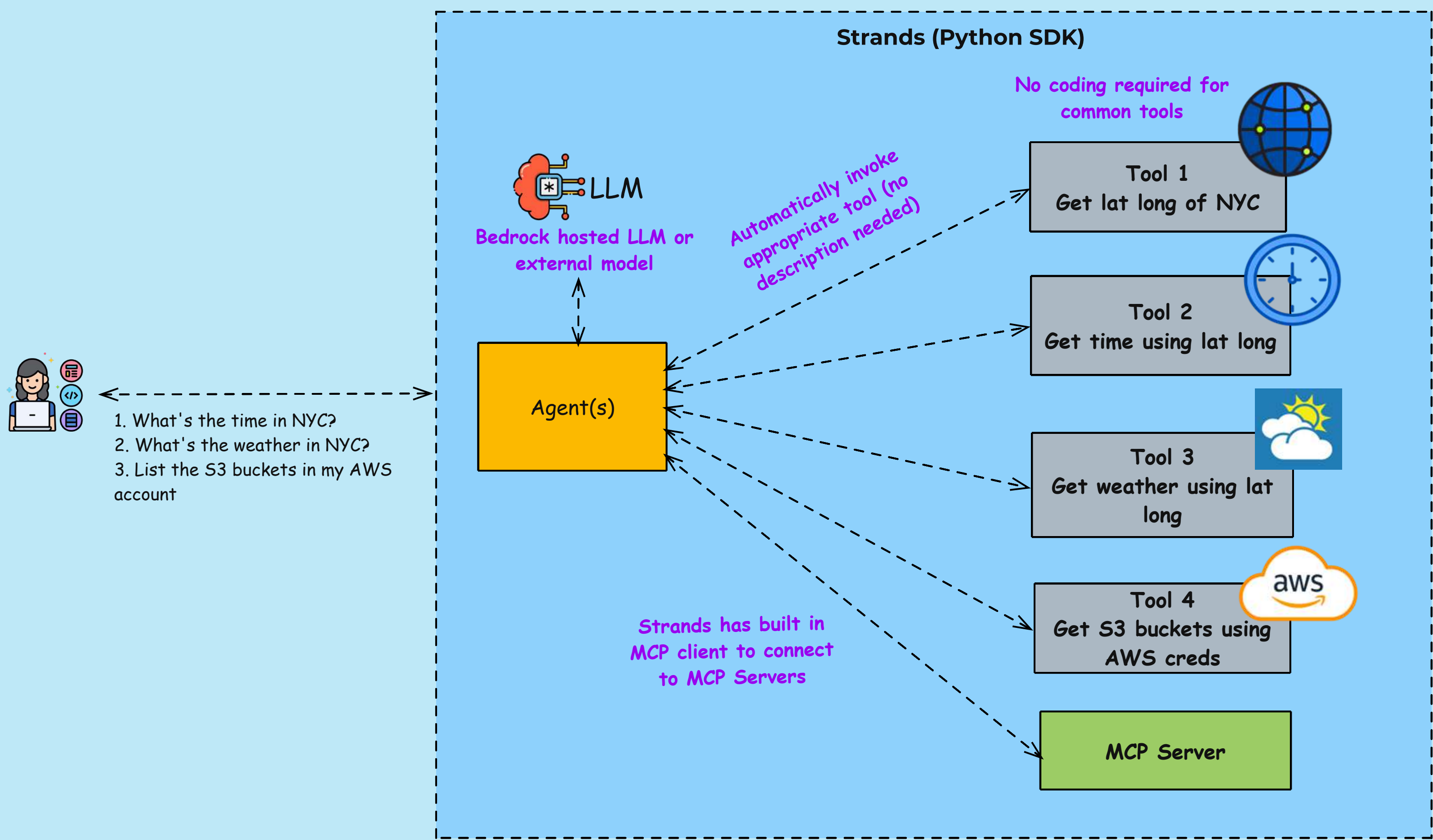
Strands Vs Bedrock Agents



Bedrock Agents

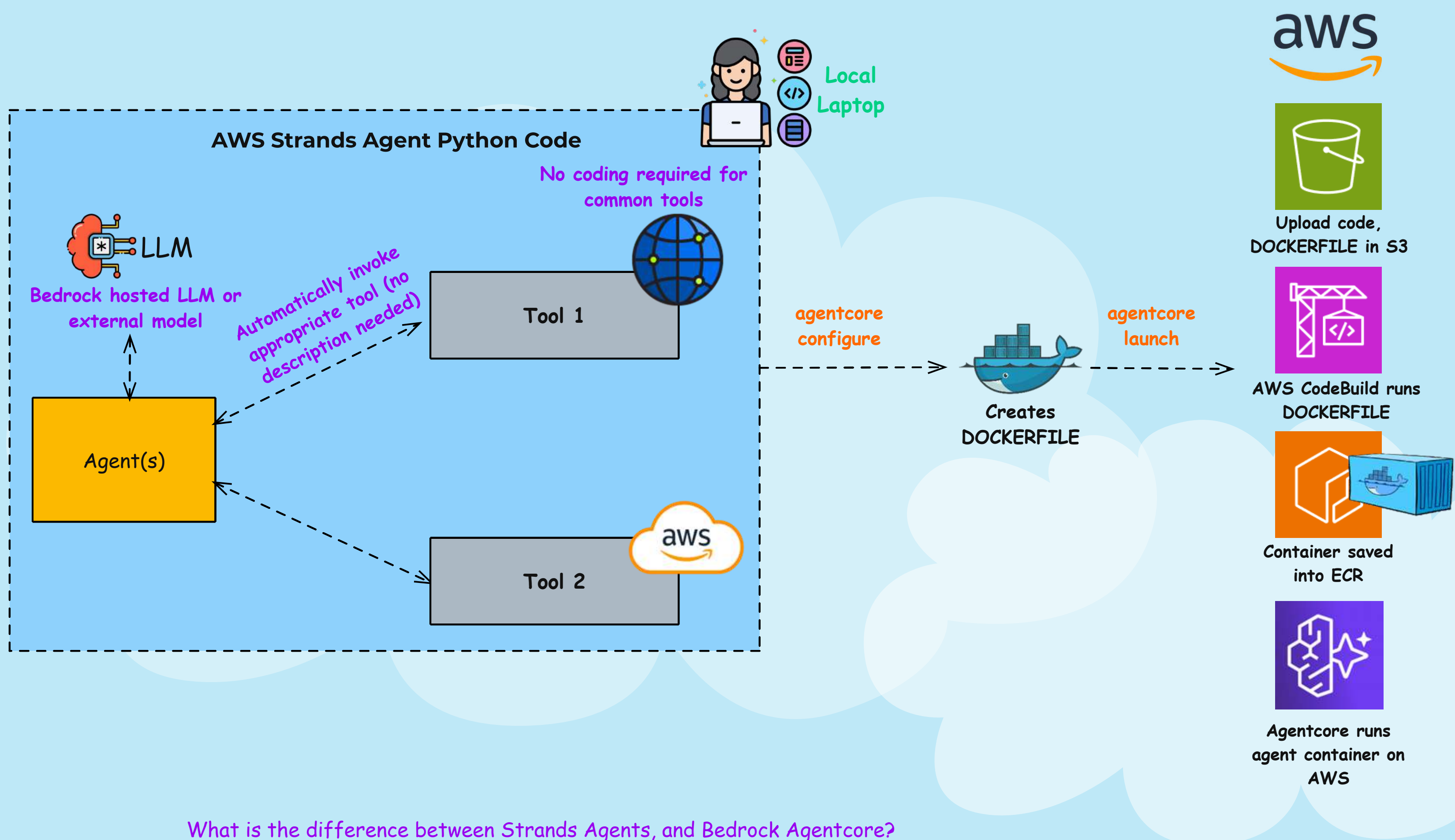


aws Strands Agents



Strands Vs Bedrock Agentcore

CLOUD WITH RAJ
www.cloudwithraj.com



What is the difference between Strands Agents, and Bedrock Agentcore?

Think of AWS Strands as a single Python program that can implement powerful agentic workflows with a minimum amount of code.

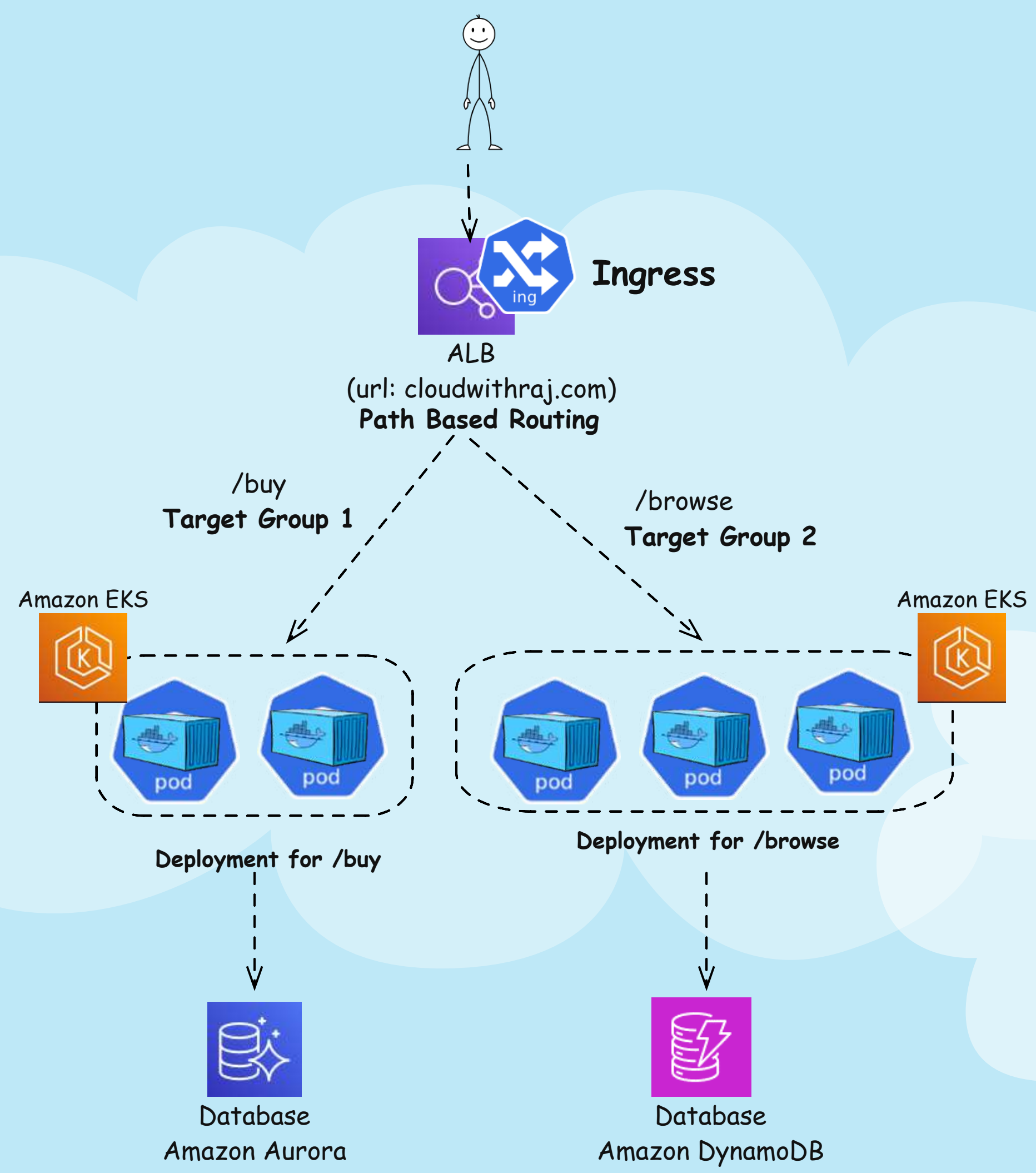
- Supports both Bedrock and external LLMs (e.g., Anthropic Claude)
- This one is awesome - it comes with 20+ prebuilt tools, that you do NOT have to code. For example: `current_time` (gives current time), `use_aws` (uses boto3 under the hood to list AWS resources), `http_request` (figures out API calls automatically)
- You can create your own custom tool
- It has MCP client built in, allowing you to connect with MCP Servers

At this point, you have your agent running in your laptop. Amazon Bedrock Agentcore helps you run this agent on AWS!

- Run command "agentcore configure" and it will create a DOCKERFILE to containerize your agent code
- Run command "agentcore launch" and it will upload and run your agent from local to Agentcore in AWS!
- The underlying infra is Serverless
- Logs, metrics, traces are automatically configured and provided
- AuthN/Z can be added easily

Demo with actual code snippet: 

Microservice on Kubernetes



Kubernetes Ingress NOT Deprecate

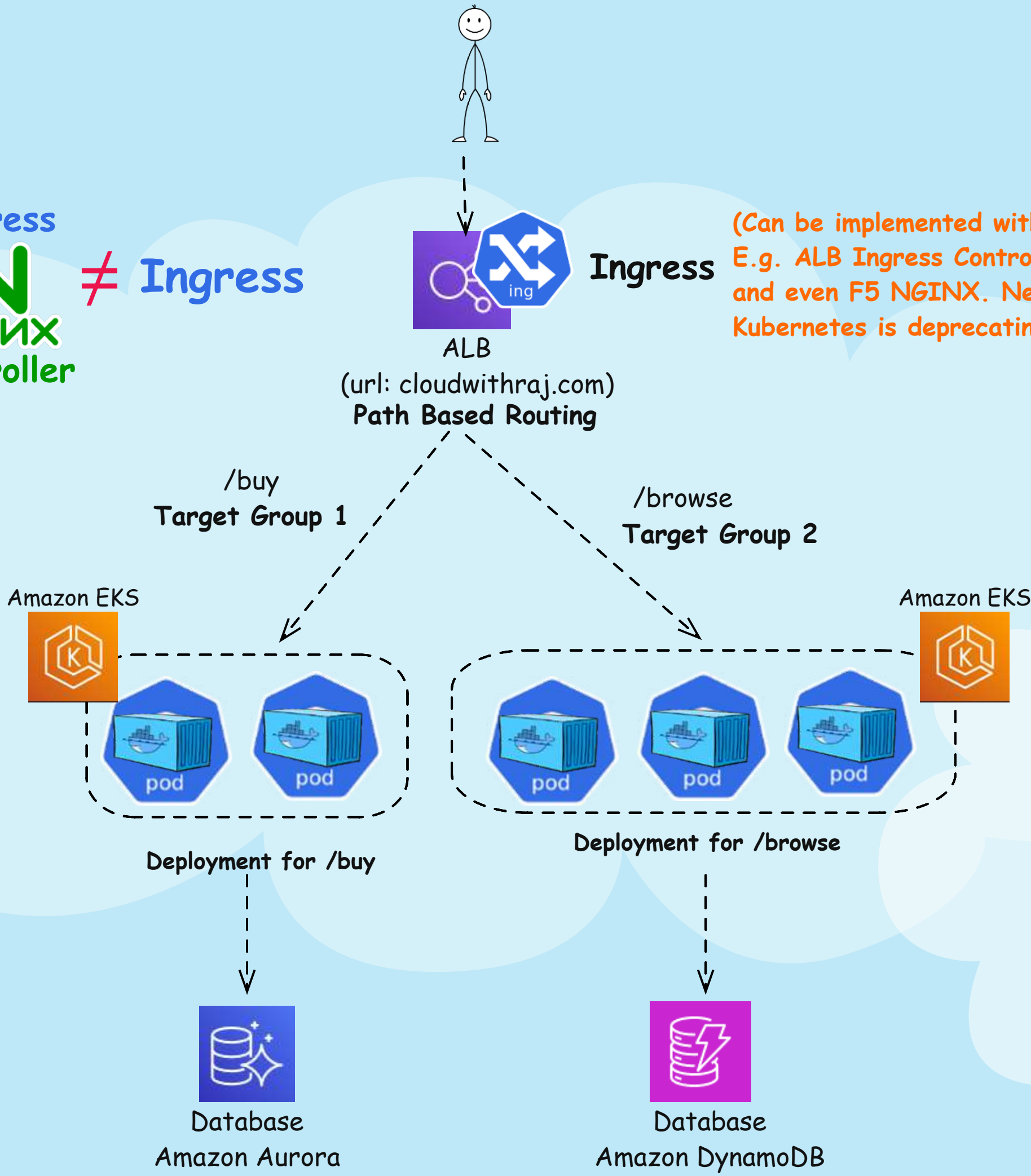
Ingress
NGINX
Controller

≠ Ingress



Ingress

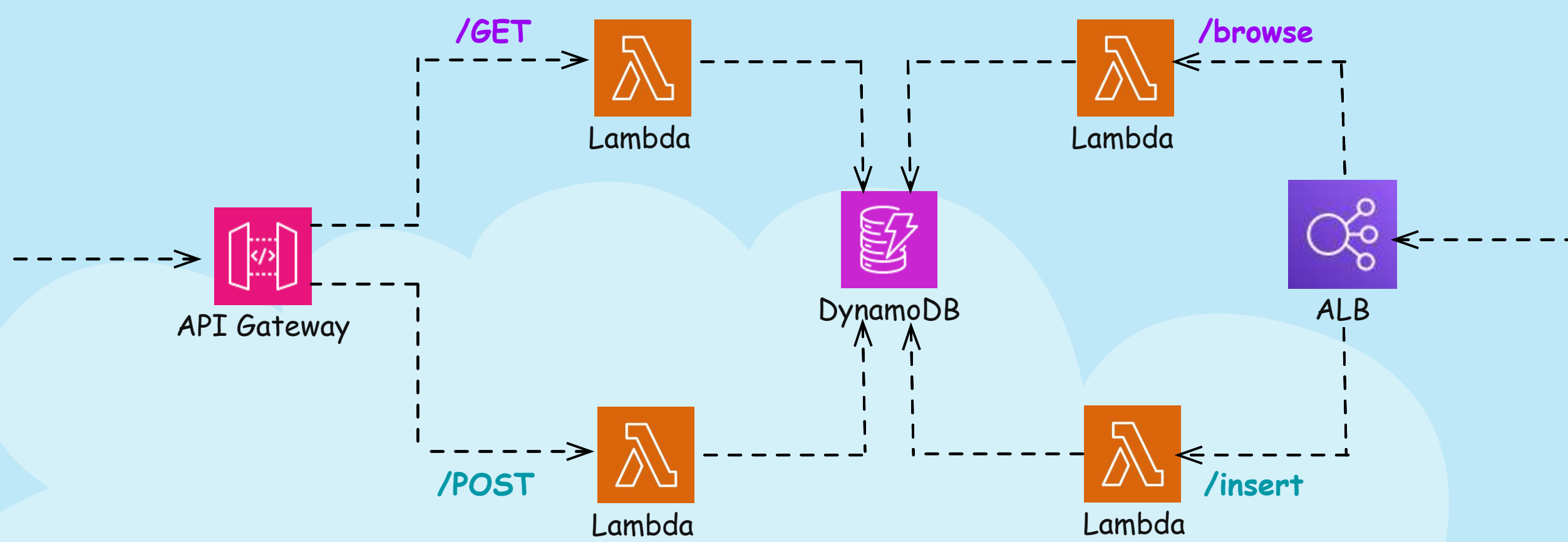
(Can be implemented with many ingress controllers
E.g. ALB Ingress Controller, Traefik, HAProxy,
and even F5 NGINX. Neither these, nor
Kubernetes is deprecating Ingress itself)



ALB VS API Gateway



CLOUD WITH RAJ
www.cloudwithraj.com



In AWS, two services often confuse architects: Application Load Balancer (ALB) and Amazon API Gateway. On the surface, both send traffic at OSI Layer 7 and scale automatically. But their use cases are very different. Let's break down the differences you need to know for real-world design and interviews.

1. Routing

API Gateway routes by HTTP method (e.g., GET → one Lambda, POST → another).

ALB routes by path (e.g., /browse vs /insert), independent of method, out of the box

2. Security

API Gateway enforces HTTPS by default with AWS-managed certs.

ALB defaults to HTTP - you must integrate with ACM to enable HTTPS.

3. Advanced Controls

API Gateway: Usage plans, quotas, rate limiting, request validation, response mapping, and DDoS protection built in.

ALB: Needs WAF/Shield for rate limiting and DDoS protection, backend code for validation.

4. Deployment

API Gateway: Canary deployments, multi-account/multi-region Lambda integration, OpenAPI spec import/export.

ALB: No canary support, region/account-bound targets, no simple spec migration.

5. Service Integration

API Gateway: Though integrates with many AWS services directly, doesn't integrate with ASG and Kubernetes directly

ALB: Integrates with EC2, ASG, Lambda. Native integration with Kubernetes, can act as an ingress or a Kubernetes service

6. Timeouts & Health Checks

API Gateway: 30s default timeout (can be increased with support request). No built-in health checks.

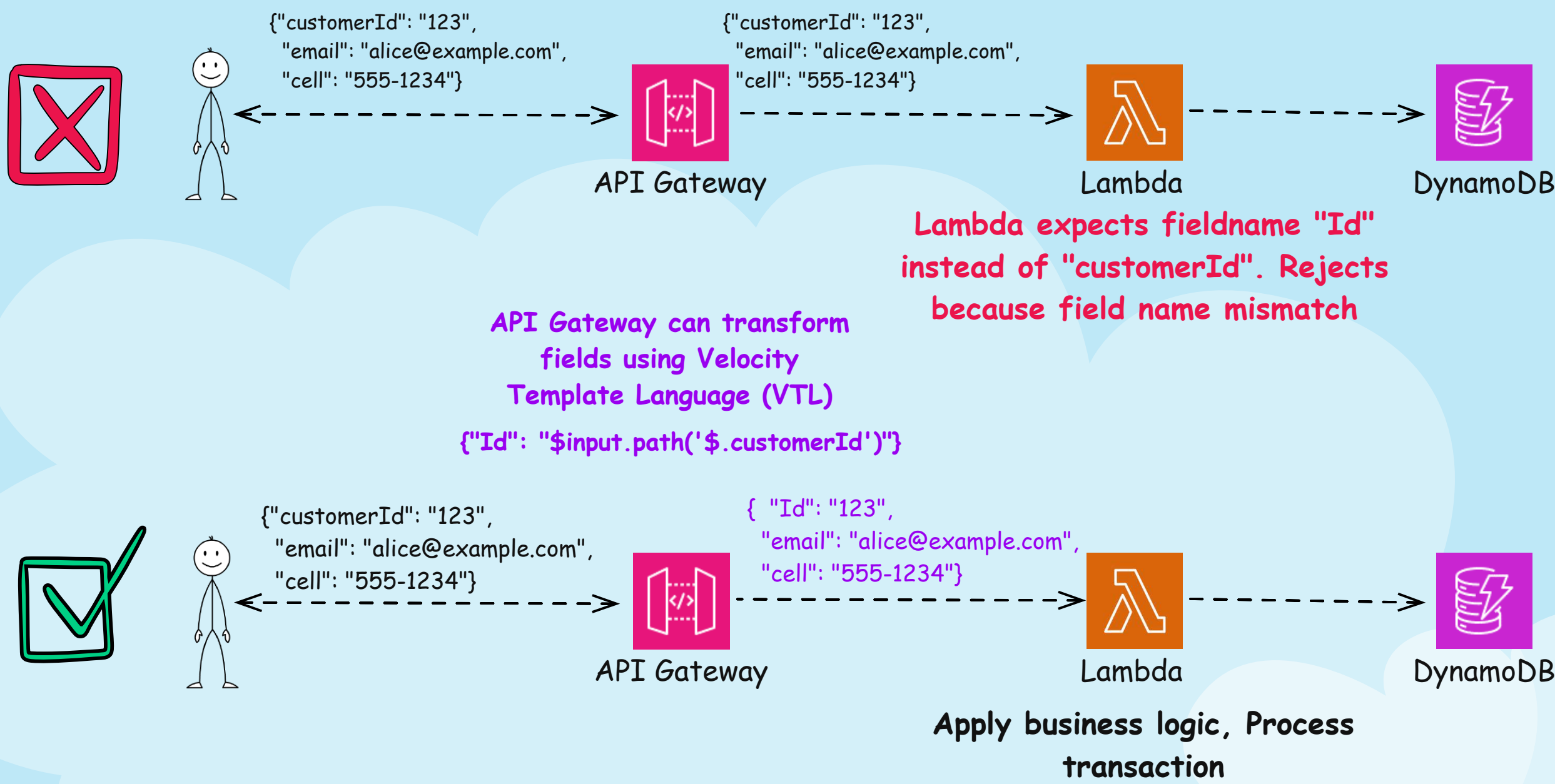
ALB: 4,000s timeout, health checks out-of-the-box.

7. Cost Model

API Gateway: Serverless, pay-per-use.

ALB: Incur charges even when idle, though relatively small at average traffic.

API Gateway Payload Transformation



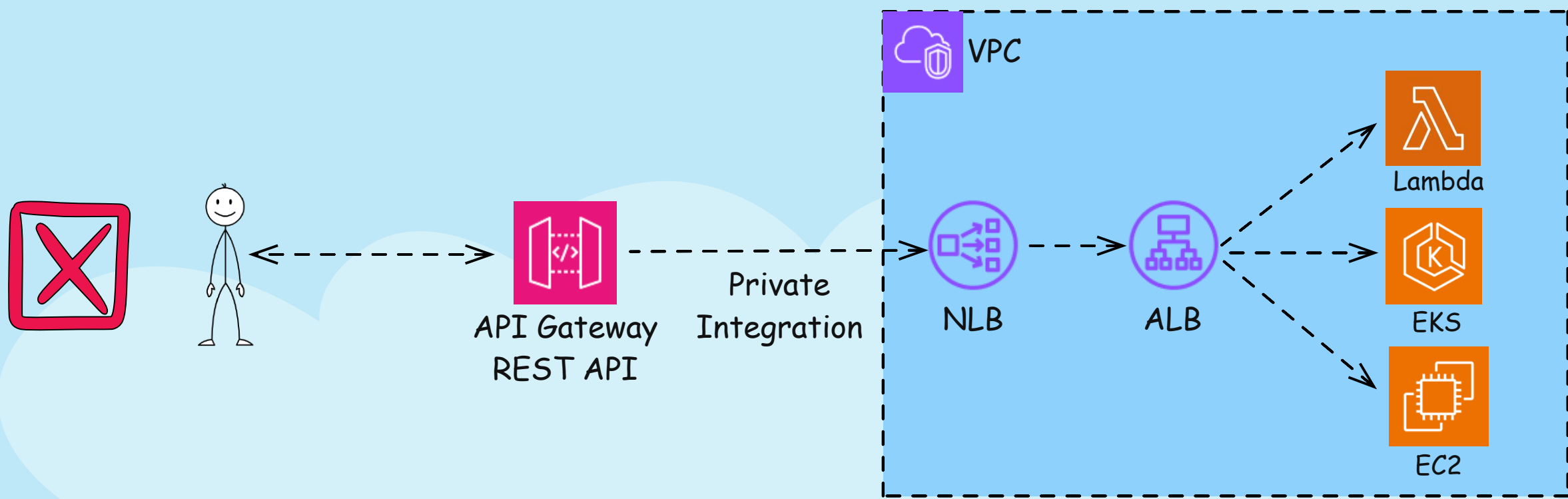
Did you know that API Gateway can also do payload transformation?

If the application is invoking the Lambda, sending the payload with certain field names, but your backend expects different names, you can achieve that in the Amazon API Gateway using Apache Velocity Template language.

In the above example, the producer is sending a fieldname "customerId", and the Lambda is expecting "Id". API Gateway is transforming the payload accordingly, and the Lambda is processing the transaction, instead of rejecting it.

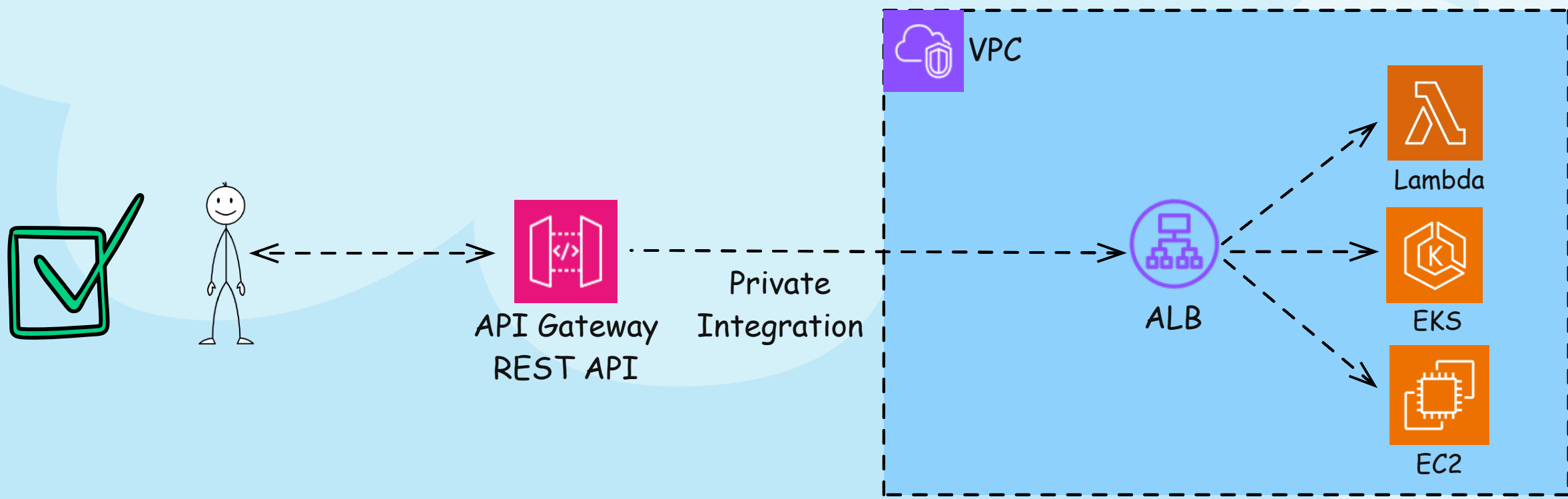
API Gateway REST API Private Integration

CLOUD WITH RAJ
www.cloudwithraj.com



REST API Private Integration could only terminate at NLB in VPC. If app is exposed through ALB, required NLB to ALB hop/complexity. HTTP API can terminate in ALB directly

Re:Invent 2025 Changed This!

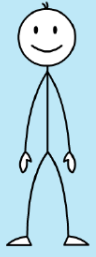


Now, REST (and HTTP API) don't need a NLB inbetween, they can go to ALB directly for Private API - cheaper, simpler, and faster

DynamoDB Global Secondary Index Change



BEFORE



Lambda



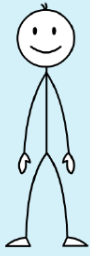
DynamoDB

GSI can only have two fields

Can only query based on Primary Key (Artist, SongTitle) or GSI (Genre, Year)

```
{
  "Artist": "Death Cab for Cutie", -----> Primary Key
  "SongTitle": "I will follow you into the dark", -----> Partition Key
  "AlbumTitle": "Plans", -----> Sort Key
  "SongRating": 4.9, -----> Global Secondary Index
  "Genre": "Indie rock", -----> Partition Key
  "Year": 2005 -----> Sort Key
}
```

AFTER Re:Invent 2025



Lambda



DynamoDB

GSI can only have upto FOUR fields (up from two) - 2 in Partition Key, 2 in Sort Key (Up from 1 each). No change in Primary Key

More flexibility in query

```
{
  "Artist": "Death Cab for Cutie", -----> Primary Key
  "SongTitle": "I will follow you into the dark", -----> Partition Key
  "AlbumTitle": "Plans", -----> Sort Key
  "SongRating": 4.9, -----> Global Secondary Index
  "Genre": "Indie rock", -----> Sort Key #1
  "Year": 2005 -----> Partition Key
  "Year": 2005 -----> Sort Key #2
}
```

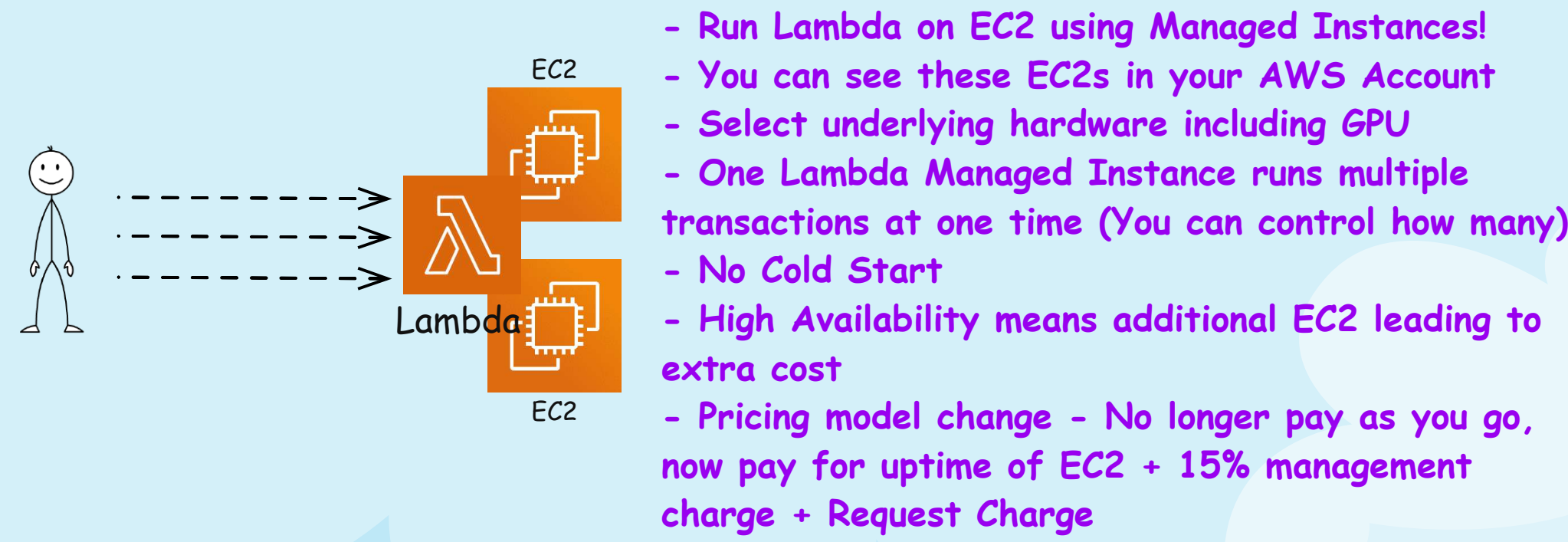
Lambda Managed Instances



BEFORE



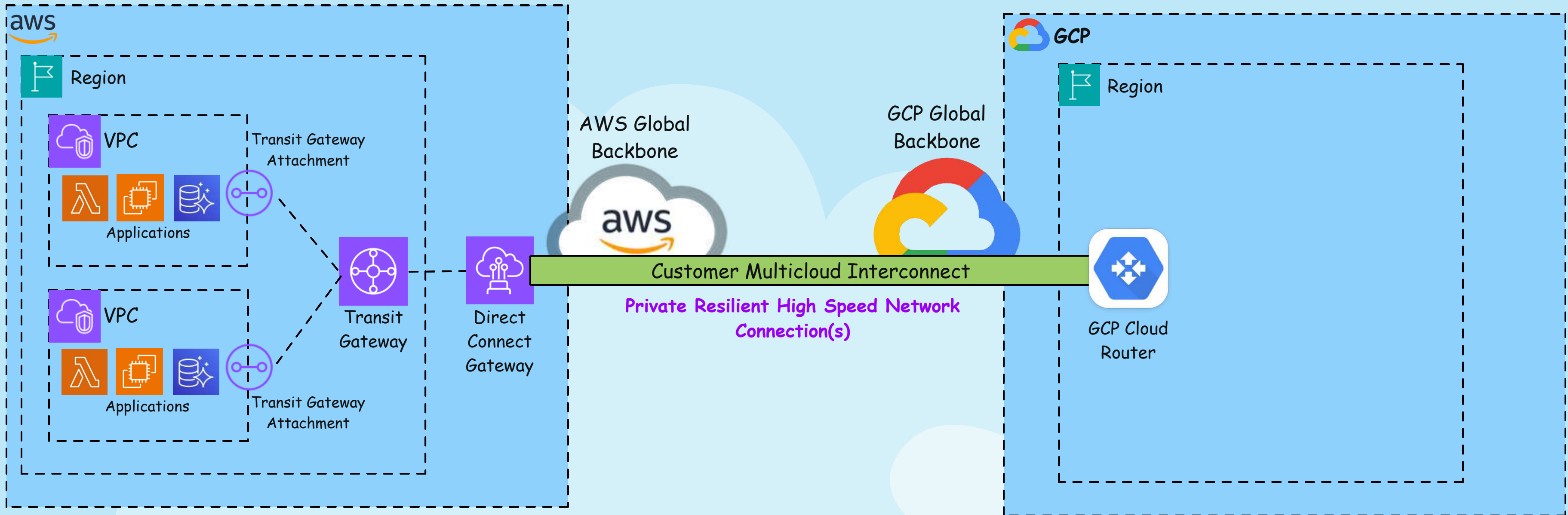
AFTER Re:Invent 2025



AWS Multicloud Interconnect



Re:Invent 2025 Announcement



AWS Interconnect enables you to create high-speed network connections between your AWS Virtual Private Clouds (VPCs) and your VPCs on other public clouds. The service is also designed to provision capacity in minutes, and traffic is encrypted.

In this example, on AWS side, Transit Gateway Attachments are attached to your VPCs. These Transit Gateway Attachments are associated with a Transit Gateway (Don't make this mistake of saying in interview that Transit Gateway is attached to VPC, it's NOT).

This Transit Gateway sends traffic to Direct Connect gateway. From here the new enhancements takes effect!

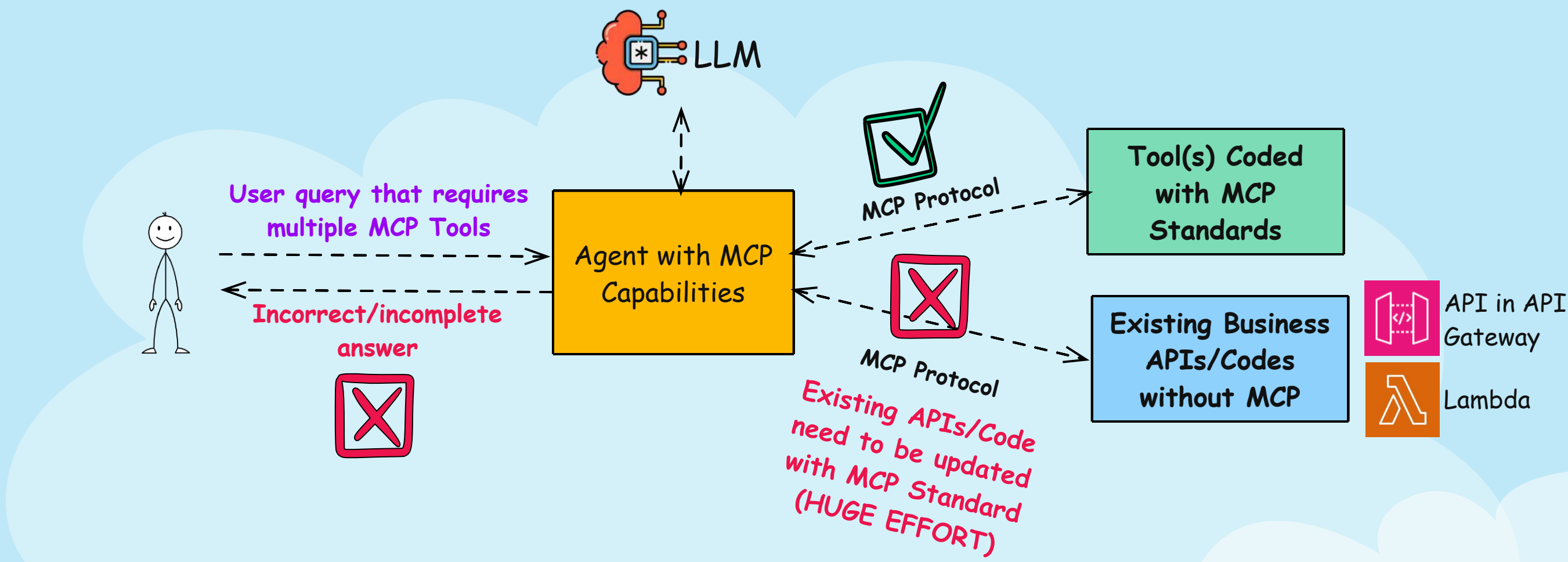
AWS and GCP (Azure coming later) has created dedicated high speed resilient network connections between AWS datacenters and Google datacenters. No traffic goes via public internet.

Traffic goes through this Customer Multicloud Interconnect Private connection, and can land in GCP network construct such as GCP Cloud Router.

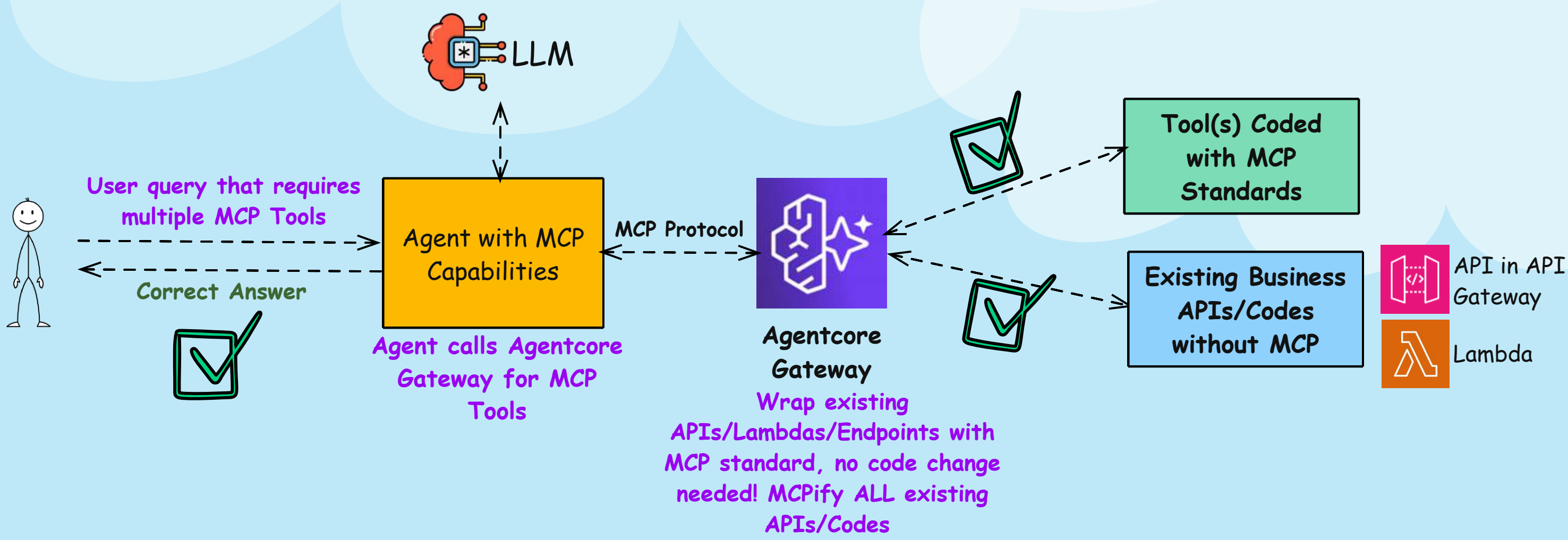
Bedrock Agentcore Gateway



BEFORE



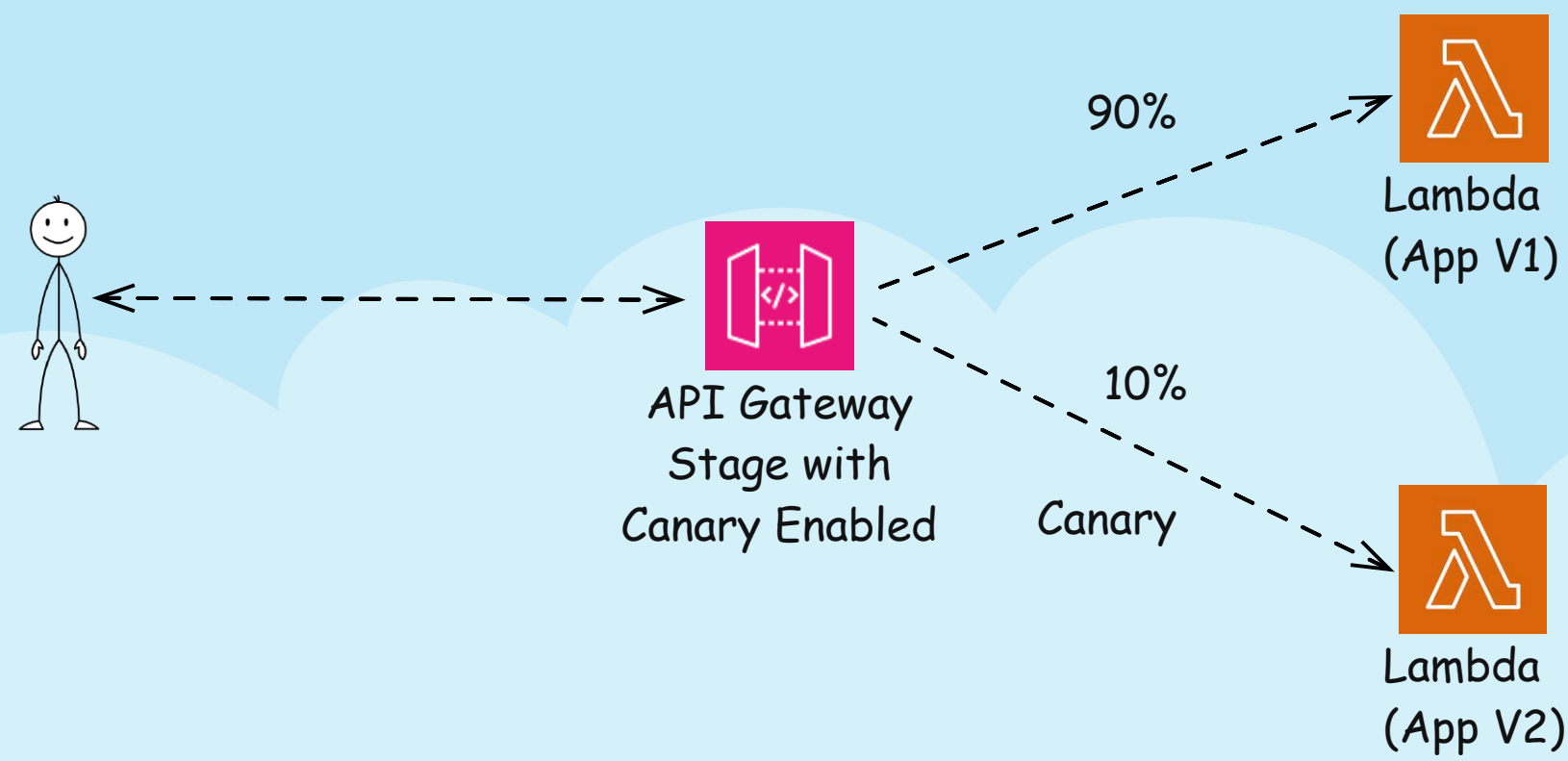
AFTER RE:INVENT 2025



Canary Deployment vs Blue/Green Vs Rolling Deployment

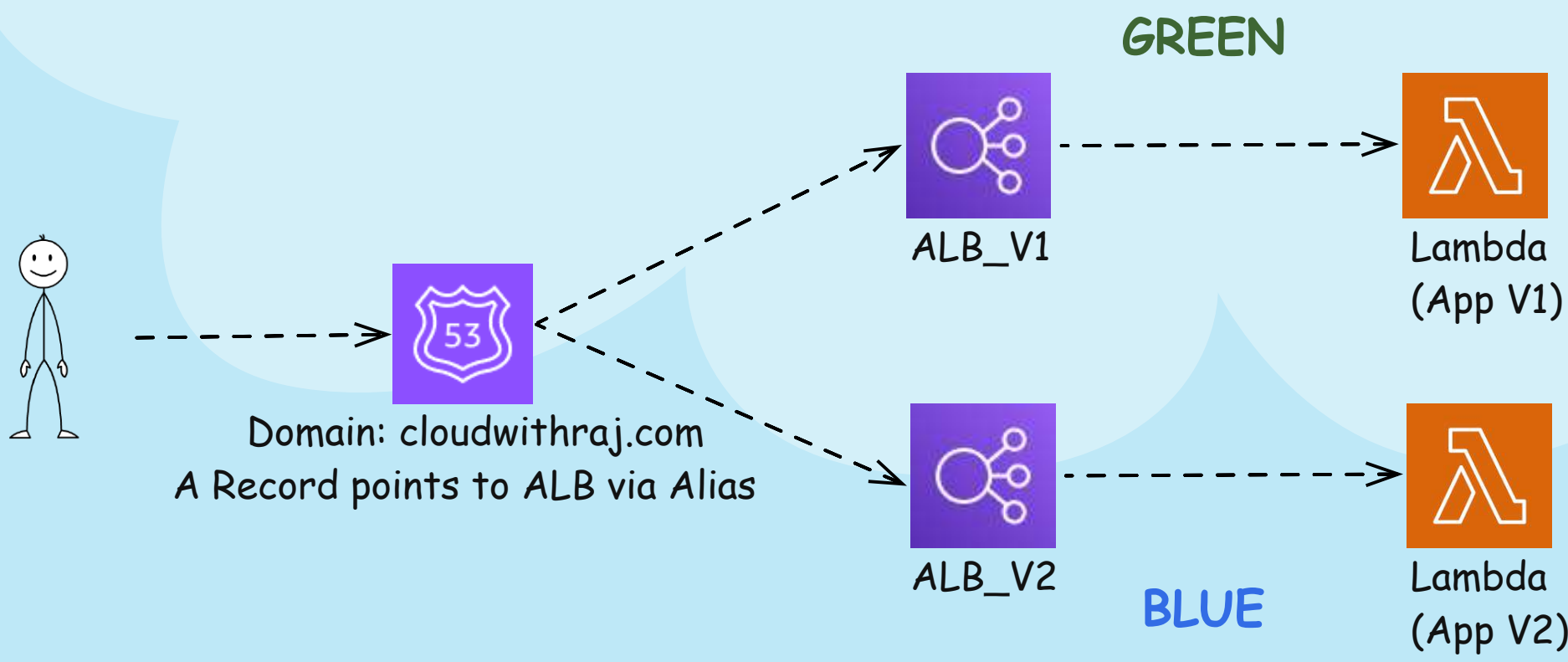


Canary Deployment



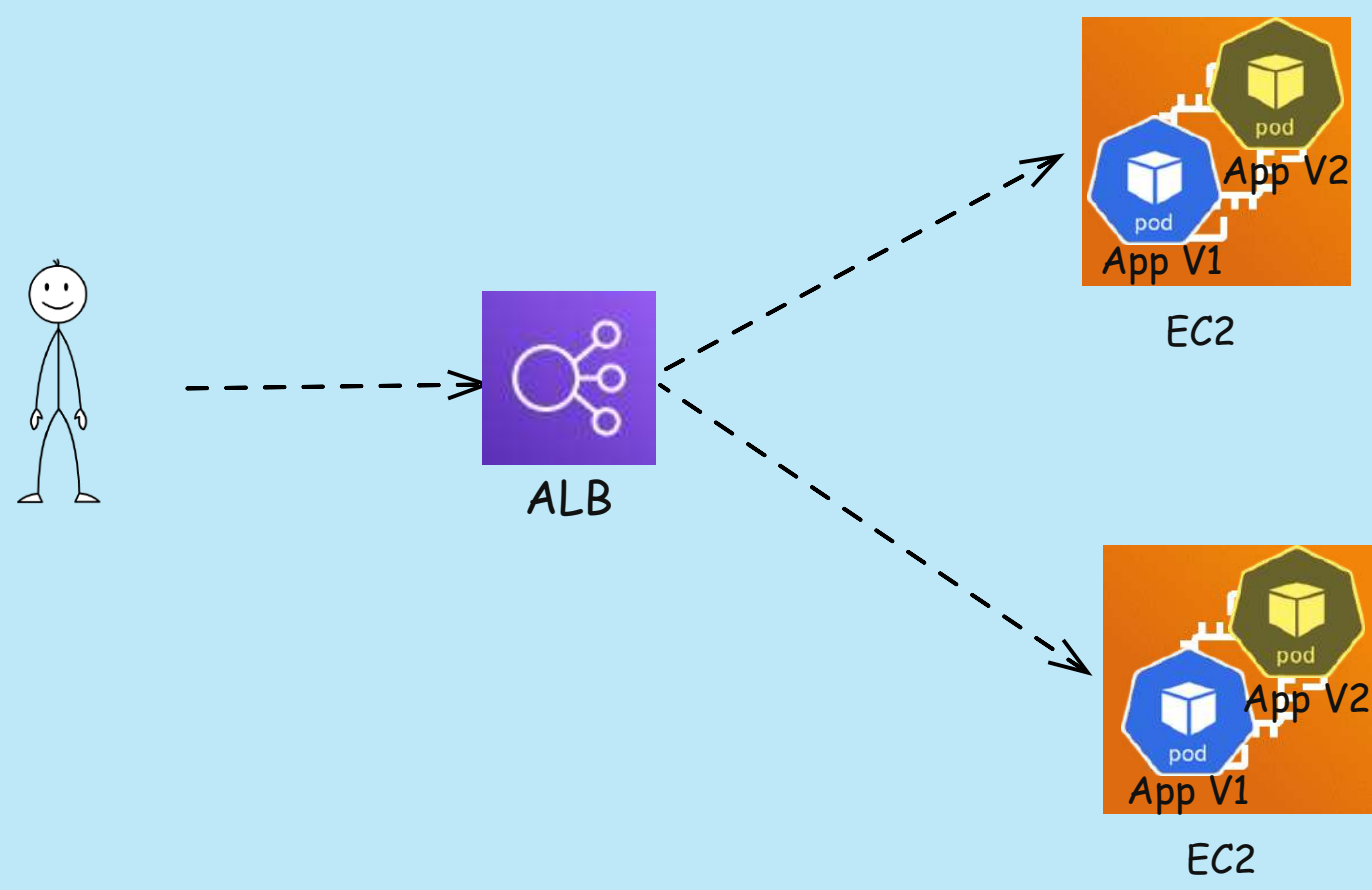
Traffic split between V1 and V2. Can be transitioned gradually to V2

Blue/Green Deployment



Traffic switched entirely from Green to Blue. Both running at same time. Can be rolled back and forth

Rolling Deployment



Replace old versions with new versions in small batches